

PROYEK AKHIR

**ANALISA PERBANDINGAN KINERJA LIBRARY CHART.JS, D3
DAN HIGHCHARTS UNTUK VISUALISASI DATA SENSOR IOT
PADA DASHBOARD BERBASIS LARAVEL**



Oleh:

AILSANISNANI ANUBHAWA

21/474745/SV/19024

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA**

2025

PROYEK AKHIR
ANALISA PERBANDINGAN KINERJA LIBRARY CHART.JS, D3
DAN HIGHCHARTS UNTUK VISUALISASI DATA SENSOR IOT
PADA DASHBOARD BERBASIS LARAVEL

Proyek Akhir
Program Studi Teknologi Rekayasa Perangkat Lunak

Diajukan untuk memenuhi salah satu syarat memperoleh gelar Sarjana Terapan
Teknik pada Program Studi Teknologi Rekayasa Perangkat Lunak Departemen
Teknik Elektro dan Informatika Sekolah Vokasi Universitas Gadjah Mada

Oleh:
AILSANISNANI ANUBHAWA
21/474745/SV/19024

PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2025

HALAMAN PENGESAHAN

PROYEK AKHIR

Judul : ANALISA PERBANDINGAN KINERJA LIBRARY CHART.JS, D3 DAN H
Nama : Ailsa Isnani Anubhawa
Program Studi : Teknologi Rekayasa Perangkat Lunak
Pembimbing : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.
Waktu Pengujian : Kamis, 26 Juni 2025, Lab Techno Gd Hy Sayap Utara Lt 2

Telah dipertanggungjawabkan dan diuji oleh Tim Penguji serta disetujui dan disahkan Sebagai syarat kelengkapan studi jenjang Sarjana Terapan Program Studi Teknologi Rekayasa Perangkat Lunak Sekolah Vokasi Universitas Gadjah Mada

Yogyakarta, 26 Juni 2025

Diterima dan disetujui oleh,

Ketua Penguji

Pembimbing/Sekretaris Penguji

Dr. Umar Taufiq, S.Kom., M.Cs.
NIP. 111198212201610101

Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.
NIP. 111199209201605201

Anggota Penguji,

Dinar Nugroho Pratomo, S.Kom., M.IM., M.Cs.
NIP. 111199407202002101

Mengetahui,

Ketua Departemen
Teknik Elektro dan Informatika

Ketua Program Studi
Teknologi Rekayasa Perangkat Lunak

Ir. Nur Rohman Rosyid, S.T., M.T., D.Eng., IPM
NIP. 111197510201206101

Dr. Umar Taufiq, S.Kom., M.Cs.
NIP. 111198212201610101

SURAT PERNYATAAN

Saya yang bertandatangan di bawah ini saya:

Nama : Ailsa Isnani Anubhawa
NIM : 21/474745/SV/19024
Tahun Terdaftar : 2021
Program Studi : Teknologi Rekayasa Perangkat Lunak
Fakultas : Sekolah Vokasi

Menyatakan bahwa dalam dokumen ilmiah Proyek Akhir ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Proyek Akhir ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 1 Juli 2025

Yang menyatakan,

Ailsa Isnani Anubhawa
21/474745/SV/19024

HALAMAN MOTTO

"It always seems impossible until it's done"
(Nelson Mandela)

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas segala rahmat, karunia, dan hidayah-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul "Analisis Perbandingan Kinerja Library Chart.js, D3, dan Highcharts Untuk Visualisasi Data Sensor IoT Pada Dashboard Berbasis Laravel" ini sebagai salah satu syarat untuk memperoleh gelar Sarjana Terapan pada Program Studi Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada. Berkenaan dengan hal tersebut, penulis menyampaikan ucapan terima kasih kepada yang terhormat:

1. Bapak Ir. Nur Rohman Rosyid, S.T., M.T., D.Eng., IPM., selaku Kepala Departemen Teknik Elektro dan Informatika Sekolah Vokasi Universitas Gadjah Mada, atas dukungan dan fasilitas yang diberikan selama proses studi dan penyusunan tugas akhir ini.
2. Bapak Dr. Umar Taufiq, S.Kom., M.Cs., selaku Ketua Program Studi Teknologi Rekayasa Perangkat Lunak sekaligus dosen penguji, atas arahan, masukan, dan evaluasi yang sangat berharga selama proses ujian tugas akhir.
3. Ibu Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D., selaku dosen pembimbing, yang telah membimbing dan memberikan banyak saran, motivasi, serta waktu dalam proses penyusunan tugas akhir ini, sehingga penulis dapat menyelesaikannya dengan baik.
4. Bapak Dinar Nugroho Pratomo, S.Kom., M.IM., M.Cs., selaku dosen penguji, yang telah memberikan masukan dan kritik yang membangun demi penyempurnaan laporan tugas akhir ini.
5. Bapak Yusron Fuadi, S.Sn., M.Sn., selaku Dosen Pembimbing Akademik, yang telah membimbing penulis selama masa studi.
6. Orang tua tercinta yang selalu memberikan doa, dukungan moril dan materiil, serta menjadi sumber semangat dan inspirasi dalam menyelesaikan pendidikan dan tugas akhir ini.
7. Rayendra Arya Daneswara yang memberi dukungan moriil dan telah kebersamai penulis dalam proses penyusunan Tugas Akhir ini.
8. Teman-teman TRPL atas kebersamaan, solidaritas, dukungan, dan semangat

yang telah menjadi bagian penting dalam perjalanan studi penulis, khususnya selama proses penyusunan tugas akhir ini.

Akhirnya, semoga segala bantuan yang telah diberikan oleh semua pihak dapat menjadi amalan yang bermanfaat dan mendapatkan buah karma baik di masa kini maupun di masa mendatang. Semoga Proyek Akhir ini menjadi informasi bermanfaat bagi pembaca atau pihak lain yang membutuhkannya.

Yogyakarta, 1 Juli 2025

Ailsa Isnani Anubhawa
21/474745/SV/19024

DAFTAR ISI

HALAMAN PENGESAHAN	i
HALAMAN PERNYATAAN BEBAS PLAGIASI	ii
HALAMAN MOTTO	iii
KATA PENGANTAR	iv
DAFTAR ISI	vi
DAFTAR GAMBAR	ix
DAFTAR TABEL	xi
INTISARI	xii
ABSTRACT	xiii
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	4
1.6 Sistematika Penulisan	4
BAB 2 LANDASAN TEORI	6
2.1 Tinjauan Pustaka	6
2.1.1 Studi Kasus <i>Library</i> JavaScript dan Visualisasi Data	6
2.1.2 Studi Analisis Perbandingan	10
2.2 Dasar Teori	17
2.2.1 Laravel	17
2.2.2 JavaScript	17
2.2.3 <i>Library</i> JavaScript	18
2.2.4 Chart.js	18
2.2.5 D3	18
2.2.6 Highcharts	18

2.2.7	Visualisasi	18
2.2.8	IoT	20
2.2.9	Dataset	20
2.2.10	Pengujian Performa	20
2.2.11	<i>Random Access Memory</i> (RAM)	22
2.2.12	<i>Chrome Task Manager</i>	22
2.2.13	<i>Use Case Diagram</i>	22
2.2.14	<i>Sequence Diagram</i>	23
BAB 3	METODOLOGI PENELITIAN	24
3.1	Bahan	24
3.2	Alat	33
3.2.1	Perangkat Lunak	33
3.2.2	Perangkat Keras	34
3.3	Tahapan Proyek Akhir	34
3.3.1	Studi Literatur	35
3.3.2	Perancangan <i>Website</i>	37
3.3.3	Implementasi	43
3.3.4	Evaluasi	43
3.3.5	Analisis dan Komparasi	46
BAB 4	HASIL DAN PEMBAHASAN	47
4.1	Perancangan <i>Frontend</i>	47
4.1.1	Halaman Utama	47
4.1.2	Halaman Visualisasi	49
4.2	Implementasi <i>Library</i>	54
4.2.1	Implementasi Pada Chart.js	54
4.2.2	Implementasi Pada D3	59
4.2.3	Implementasi Pada Highcharts	63
4.3	Pengukuran Performa Rendering	67
4.3.1	Chart.js	68
4.3.2	D3	69
4.3.3	highcharts	71
4.4	Pengukuran Penggunaan CPU	72
4.4.1	Chart.js	73
4.4.2	D3	74
4.4.3	Highcharts	75
4.5	Pengukuran <i>Footprint Memory</i>	76

4.5.1	Chart.js	77
4.5.2	D3	78
4.5.3	Highcharts	79
4.6	Analisis Skalabilitas	81
4.6.1	Performa <i>Rendering</i>	81
4.6.2	Alokasi CPU	84
4.6.3	Alokasi Memori	87
BAB 5	PENUTUP	90
5.1	Kesimpulan	90
5.2	Saran	91
DAFTAR PUSTAKA		92
LAMPIRAN A		L - 1
A	Lembar Perbaikan Proyek Akhir	L - 1
LAMPIRAN B		L - 3
B	Dokumentasi	L - 3

DAFTAR GAMBAR

2.1	Bentuk <i>Line Chart</i>	19
2.2	Bentuk <i>Bar Chart</i>	19
2.3	Bentuk <i>Scatter Plot</i>	20
3.1	Sensor BME280	26
3.2	Install Flask	27
3.3	<i>Library</i>	27
3.4	Inisialisasi Flask dan CORS	27
3.5	variabel	28
3.6	Membuat Sensor <i>Dummy</i>	29
3.7	API Endpoint	29
3.8	Menjalankan Program Threading	30
3.9	Menjalankan Sensor	30
3.10	Mounting Data	32
3.11	Membuat kolom timestamp	32
3.12	Membuat Endpoint API	33
3.13	Menjalankan Server	33
3.14	Alur Penelitian	35
3.15	use Case Diagram	38
3.16	<i>Sequence Diagram</i>	39
3.17	Halaman Utama	40
3.18	Menambahkan Formula	41
3.19	Menambahkan Ambang Batas	41
3.20	Menambahkan Ambang Batas	42
3.21	Menambahkan Ambang Batas	42
3.22	Mulai Pengukuran	44
3.23	Akhir Pengukuran	44
3.24	Pengukuran Footprint Memory	45
4.1	Generate API	47
4.2	Fungsi Generate API	48
4.3	Generate API	49
4.4	Fungsi Menyimpan Log Render	50
4.5	Fungsi Fetch Data	50
4.6	Memperbarui Data pada Chart	51
4.7	Potongan Kode Pada Button New Visualization	51

4.8	Fungsi untuk Membuat Visualisasi Baru.....	52
4.9	Fungsi untuk Populate Dropdown.....	53
4.10	Fungsi Delete Chart.....	54
4.11	Inisialisasi Variabel dan Pengukuran Performa.....	55
4.12	Potongan Kode untuk Menentukan Jenis Visualisasi.....	55
4.13	Potongan Kode untuk Pembuatan Chart.....	56
4.14	Potongan Kode untuk Pembuatan Chart.....	58
4.15	Membuat Sumbu Pada D3.....	60
4.16	Potongan Kode untuk Membuat <i>Tooltip</i>	60
4.17	Potongan Kode untuk Mengimplementasikan <i>Line Chart</i>	61
4.18	Potongan Kode untuk Mengimplementasikan <i>Bar Chart</i>	61
4.19	Potongan Kode untuk Mengimplementasikan <i>Scatter Plot</i>	62
4.20	Fungsi untuk Memperbarui Chart D3.....	63
4.21	Potongan Kode untuk Kontainer Grafik.....	64
4.22	Potongan Kode untuk Membuat Grafik Highcharts.....	65
4.23	Potongan Kode untuk Memperbarui Grafik Highcharts.....	67
4.24	Perbandingan Rendering Chart.js.....	68
4.25	Perbandingan Rendering D3.....	70
4.26	Perbandingan Rendering Highcharts.....	72
4.27	Potongan Kode untuk Mengukur Penggunaan CPU.....	73
4.28	Grafik Komparasi Chart.js.....	74
4.29	Grafik Komparasi D3.....	75
4.30	Grafik Komparasi Highcharts.....	76
4.31	Grafik Komparasi Chart.js.....	78
4.32	Grafik Komparasi D3.....	79
4.33	Grafik Komparasi Highcharts.....	80
4.34	Grafik Komparasi Render Pertama.....	82
4.35	Grafik Komparasi Render Kedua.....	83
4.36	Grafik Komparasi Render ketiga.....	83
4.37	Grafik Komparasi CPU Pertama.....	85
4.38	Grafik Komparasi CPU Kedua.....	85
4.39	Grafik Komparasi CPU Ketiga.....	86
4.40	Grafik Komparasi Memori Pertama.....	87
4.41	Grafik Komparasi Memori Kedua.....	88
4.42	Grafik Komparasi Memori Ketiga.....	89

DAFTAR TABEL

2.1	Perbandingan penelitian	8
2.2	Perbandingan Penelitian Library Visualisasi JavaScript	14
3.1	Struktur JSON dari Sensor	25
3.2	Struktur JSON dari Sensor	31
3.3	Spesifikasi Perangkat Lunak	33
3.4	<i>Spesifikasi Perangkat Keras yang Digunakan</i>	34
3.5	Perbandingan Chart.js, D3.js, dan Highcharts	36
4.1	Komparasi Render <i>Chart.js</i>	68
4.2	Komparasi Render D3	70
4.3	<i>Komparasi</i> Render Highcharts	71
4.4	Komparasi CPU <i>Chart.js</i>	73
4.5	Komparasi CPU <i>Chart.js</i>	74
4.6	Komparasi CPU pada <i>Highcharts</i>	76
4.7	Komparasi Memori <i>Chart.js</i>	77
4.8	Komparasi Memori <i>D3.js</i>	78
4.9	Komparasi Memori <i>Highcharts</i>	80
4.10	Komparasi Rendering Library Chart.js, D3.js, dan Highcharts	81
4.11	Komparasi Penggunaan CPU	84
4.12	Perbandingan Penggunaan Memori	87

INTISARI

Website telah berkembang menjadi platform yang kompleks, salah satunya yaitu dengan penggunaanya sebagai dasbor untuk menampilkan informasi dari perangkat IoT. Dalam hal ini, *website dashboard* berfungsi sebagai antarmuka visual untuk menampilkan data hasil pembacaan sensor IoT dan memvisualisasikan data-data tersebut. Untuk membuat visualisasi data IoT, terdapat beberapa *library* yang dapat digunakan dan setiap *library* memiliki kelebihan serta kekurangannya masing-masing. Dengan beberapa pertimbangan seperti performa *rendering*, fleksibilitas, dan implementasi, *library* dari bahasa pemrograman JavaScript dapat menjadi salah satu pilihan karena fleksibilitasnya yang dapat dijalankan di semua browser modern tanpa membutuhkan dependensi tambahan. Penelitian ini membandingkan tiga *library* JavaScript, Chart.js, D3.js, dan Highcharts, untuk membantu pengembang memilih yang paling sesuai dalam menangani data IoT yang terus-menerus diperbarui. Evaluasi penggunaan *library* visualisasi JavaScript diukur berdasarkan performa *rendering* data *real-time*, skalabilitas tampilan *chart*, dan efisiensi penggunaan CPU serta memori. Berdasarkan hasil pengujian tersebut menunjukkan bahwa Chart.js memiliki performa *rendering* yang baik, efisien dalam penggunaan memori dan CPU, serta menunjukkan stabilitas tinggi tanpa fluktuasi signifikan. D3.js unggul dalam bersifat *customizable* tetapi kurang stabil untuk data besar dan lebih boros memori serta CPU. Highcharts mampu menjaga kecepatan, stabilitas *rendering*, dan efisiensi memori pada beban data besar karena fitur optimasi internal, sehingga cocok untuk visualisasi IoT berskala besar. Hasil penelitian ini diharapkan dapat memberikan pengetahuan dan panduan dalam memilih *library* visualisasi yang optimal untuk meningkatkan performa *dashboard* dan efisiensi sistem.

Kata kunci: JavaScript, Highcharts, D3.js, Chart.js, *Performa Rendering*, CPU, *Footprint Memory library*

ABSTRACT

The website has evolved into a complex platform, one of which is its use as a dashboard for displaying information from IoT devices. In this context, a dashboard website serves as a visual interface to present data collected by IoT sensors and to visualize that data. To create IoT data visualizations, there are various libraries available, each with its own advantages and disadvantages. Considering factors such as rendering performance, flexibility, and its implementation, JavaScript libraries are a suitable choice because of their flexibility and compatibility with all modern browsers without requiring additional dependencies. This study compares three JavaScript libraries, Chart.js, D3.js, and Highcharts, to help developers choose the most appropriate one for handling continuously updated IoT data. The evaluation of these JavaScript visualization libraries focuses on real-time data rendering performance, chart scalability, and the efficiency of CPU and memory usage. The results show that Chart.js delivers good rendering performance, is efficient in memory and CPU usage, and maintains high stability without significant fluctuations. D3.js excels in customization but is less stable for large datasets and tends to consume more memory and CPU resources. Highcharts maintains rendering speed, stability, and memory efficiency under heavy data loads because of its built-in optimization features, making it suitable for large-scale IoT visualizations. It is hoped that the findings of this study will provide insights and guidance in selecting an optimal visualization library to improve dashboard performance and overall system efficiency.

Key words: JavaScript, Highcharts, D3.js, Chart.js, *rendering performance*, *CPU*, *Footprint Memory*, *library*

BAB I

PENDAHULUAN

1.1 Latar Belakang

Website mengalami perkembangan yang signifikan. Bukan hanya sebagai media untuk menampilkan informasi berupa teks atau gambar sederhana, saat ini, *website* telah menjadi platform dengan ekosistem yang kompleks dan diperuntukkan untuk berbagai hal. *Website* dapat dikembangkan dengan beberapa *framework* dan salah satu *framework* yang sering digunakan adalah Laravel. Laravel merupakan kerangka kerja PHP yang dapat diimplementasikan dalam berbagai proyek [1], salah satunya adalah sebagai dashboard yang menampilkan informasi dari perangkat lain seperti contohnya sistem IoT. Dalam sistem IoT, *website dashboard* memegang peranan sebagai antarmuka visual yang menampilkan visualisasi data penting dari perangkat IoT.

Dalam konteks ini, visualisasi data menjadi media yang sering digunakan untuk memahami dan mengambil insight dari data-data perangkat IoT yang memiliki ukuran besar serta memperoleh kesimpulan dalam pemrosesannya baik dalam bentuk *plot*, *chart*, atau bahkan animasi [2] [3]. Selain itu, melalui visualisasi data yang interaktif dan informatif, pengguna dapat dengan cepat mendeteksi masalah, menganalisis performa, serta melakukan penyesuaian atau pengaturan terhadap perangkat IoT tanpa perlu terlibat dalam proses teknis yang rumit.

Untuk membuat visualisasi data dari perangkat sistem IoT, terdapat banyak opsi *library* yang dapat diimplementasikan dengan beberapa bahasa pemrograman. Seperti contohnya *library* Plotly yang dapat diimplementasikan menggunakan bahasa pemrograman Python [3] atau *library* visualisasi seperti D3 dan Chart.js yang diimplementasikan dengan bahasa pemrograman JavaScript [4]. Terdapat beberapa *library* JavaScript yang dapat digunakan untuk memvisualisasikan data sensor IoT, beberapa diantaranya adalah Chart.js, D3.js, dan Highcharts yang ketiganya memiliki spesifikasi, kelebihan, dan kekurangan masing-masing. Chart.js adalah *library* visualisasi data berbasis JavaScript yang sederhana dan *open-source*. *Library* ini cocok untuk proyek kecil dan menengah dan mendukung berbagai jenis grafik seperti *bar*, *line*, dan *pie* [5]. D3 (atau D3.js) adalah *library* JavaScript gratis dan *open-source* yang digunakan untuk membuat visualisasi data. Dengan fitur-fiturnya, D3 mendukung fleksibilitas dalam membangun grafik yang interaktif dan dinamis. D3 menawarkan penggunaan *stylesheet* eksternal untuk mengubah tampilan grafik termasuk menyesuaikan dengan media *queries* untuk grafik responsif atau mode

gelap [6]. Sedangkan Highcharts adalah *library* chart yang banyak digunakan untuk memvisualisasikan data berskala besar dan kompleks serta mendukung grafik fitur yang memiliki performa tinggi [6]. Berdasarkan data yang diambil dari laman ossinsight.io saat data ini diakses, Chart.js, D3.js, dan Highcharts berada pada 10 besar *library* yang paling populer berdasarkan jumlah *stars* dan *pull request* pada github [7]. Oleh karena itu, untuk menentukan *library* yang tepat dalam proses visualisasi pada suatu *website*, terdapat beberapa hal yang menjadi pertimbangan. Misalnya, bahasa pemrograman yang digunakan, grafik interaktif atau statis, dengan animasi atau tanpa animasi, dan performa *rendering*. Dengan pertimbangan tersebut, JavaScript menjadi media paling fleksibel untuk menampilkan grafik statis maupun interaktif karena JavaScript telah diimplementasikan di setiap browser modern dan tidak memerlukan dependensi tambahan dalam penggunaannya. Selain itu, ketika digabungkan dengan CSS, HTML5, dan *library* yang tepat, kemampuan JavaScript menjadi semakin luas dan berkembang [4] [8].

Penelitian ini akan membandingkan tiga *library* JavaScript yaitu Chart.js, D3.js, dan Highcharts karena ketiga *library* tersebut termasuk kedalam 10 besar *library* charting JavaScript dan memiliki jumlah *pull request* di atas 1000 [7]. Selain karena popularitasnya yang sering digunakan oleh pengembang, alasan untuk membandingkan ketiga *library* ini adalah guna membantu pengembang memilih *library* visualisasi data yang paling sesuai dengan kebutuhannya untuk menangani data IoT yang secara terus menerus menghasilkan data baru dengan kecepatan yang tinggi [9]. Pemilihan *library* dievaluasi berdasar beberapa aspek utama, seperti performa *rendering* data *real-time*, skalabilitas chart, serta efisiensi dalam hal penggunaan sumber daya CPU dan memori. Evaluasi tersebut diukur berdasarkan efisiensi sistem dalam mengeksekusi suatu tugas, termasuk salah satunya adalah pengukuran performa *real-time* data rendering untuk mengukur waktu yang dibutuhkan oleh *website* dalam menampilkan visualisasi [4] serta analisis penggunaan memori dan CPU saat web diakses.

Pengujian dilakukan dengan studi kasus pengembangan dashboard untuk perangkat IoT. *Website* ini dibuat untuk mempermudah pengguna perangkat IoT dalam menganalisis data-data dari sensor IoT dalam bentuk visualisasi grafik yang mudah dipahami. Dengan adanya penelitian ini, diharapkan pengembang dapat memilih *library* yang paling optimal untuk visualisasi data, dan memastikan performa dashboard yang lebih baik dari segi user experience maupun efisiensi sistem.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan sebelumnya, maka dirumuskan beberapa rumusan masalah dalam karya tulis ini, sebagai berikut :

1. Bagaimana performa rendering *website* saat data IoT divisualisasikan dengan *library* Highcharts, Chart.js, atau D3?
2. Bagaimana penggunaan memori saat data dimuat dan divisualisasikan dengan *library* Highcharts, Chart.js, atau D3?
3. Bagaimana penggunaan CPU saat data dimuat dan divisualisasikan dengan *library* Highcharts, Chart.js, atau D3?
4. *Library* manakah yang memiliki performa paling baik untuk memvisualisasikan data IoT pada kasus ini?

1.3 Batasan Masalah

Agar penelitian tetap terfokus pada ruang lingkup yang telah ditetapkan, akan dijelaskan batasan-batasan yang perlu dipertimbangkan dalam pengembangan penelitian ini sebagai berikut:

1. Data yang digunakan pada percobaan ini merupakan dataset numerik dan tidak mencakup data IoT yang berupa gambar atau audio.
2. Penelitian ini dilakukan menggunakan perangkat keras dan perangkat lunak yang terbatas, dengan spesifikasi komputer tertentu yang digunakan untuk mengukur performa. Hasil penelitian ini mungkin tidak sepenuhnya representatif untuk perangkat keras atau perangkat lunak yang memiliki spesifikasi yang berbeda.
3. *Library* yang digunakan dalam penelitian ini hanya berfokus pada Highcharts, Chart.js, atau D3 serta hanya meneliti penggunaan *line chart*
4. Data yang digunakan sebagai bahan perbandingan pada penelitian ini merupakan data dummy yang dibuat dengan format seperti data sensor sebenarnya.
5. Pengujian ini menggunakan 5000 data uji untuk meninjau indikator-indikator yang telah ditentukan.

1.4 Tujuan Penelitian

Berikut adalah beberapa tujuan penelitian yang telah ditetapkan untuk memandu jalannya penelitian ini:

1. Mengetahui *library* yang paling efisien untuk memvisualisasikan data IoT

diantara *library* Highcharts, Chart.js, atau D3.js dengan pertimbangan parameter yang telah ditentukan.

2. Memperoleh cara terbaik untuk memaksimalkan penggunaan *library* tersebut.

1.5 Manfaat Penelitian

Adapun manfaat-manfaat yang diperoleh dari penelitian ini, yaitu:

1. Mempermudah pengembang perangkat lunak untuk mengembangkan dashboard IoT yang menampilkan visualisasi berupa *chart*.
2. Penelitian ini dapat memberikan wawasan tentang bagaimana *library* visualisasi mempengaruhi kinerja *runtime*, yang dapat digunakan untuk meningkatkan efisiensi dalam memvisualisasikan data.
3. Membantu meningkatkan pengalaman pengguna dalam interaksi dengan dashboard dan sistem IoT.

1.6 Sistematika Penulisan

Laporan proyek akhir ditulis dengan sistematika penulisan sebagai berikut.

1. Bab I Pendahuluan: menjelaskan latar belakang, rumusan masalah, tujuan, batasan masalah, serta sistematika penulisan dari laporan proyek akhir.
2. Bab II Tinjauan Pustaka: Bab ini menjelaskan referensi pendukung dari penelitian yang dilakukan dan perbandingan dengan penelitian-penelitian sejenis yang membahas mengenai perbandingan *library* JavaScript untuk merepresentasikan data-data IoT.
3. Bab III Metodologi Penelitian: Menjelaskan tentang alat dan bahan dalam penelitian, serta matrik evaluasi yang digunakan dalam membandingkan ketiga *library* JavaScript yang dimaksud.
4. Bab IV Hasil dan Pembahasan: Menjelaskan hasil pengujian dari *library* Chart.js, D3, dan Highcharts berdasarkan parameter-parameter yang digunakan.
5. Bab V Kesimpulan dan Saran: Menjelaskan hasil pengujian dari *library* Chart.js, D3, dan Highcharts berdasarkan parameter-parameter yang digunakan.
6. Daftar Pustaka: menjabarkan sumber-sumber yang digunakan dalam laporan proyek akhir.

Secara keseluruhan, sistematika penulisan dalam laporan proyek akhir sarjana terapan adalah susunan atau struktur dari laporan proyek akhir sarjana terapan yang

menjabarkan bagian-bagian yang harus ada dalam laporan proyek akhir sarjana terapan, yang meliputi Pendahuluan, Tinjauan Pustaka, Metode Penelitian, Hasil dan Pembahasan, Kesimpulan dan Saran, serta Daftar Pustaka. Sistematika penulisan yang baik akan membuat laporan proyek akhir sarjana terapan lebih mudah untuk dibaca dan dipahami.

BAB II

LANDASAN TEORI

2.1 Tinjauan Pustaka

Tinjauan Pustaka merupakan bagian yang memuat kajian terhadap berbagai sumber literatur yang relevan yang berkaitan dengan topik penelitian. Melalui bagian ini, penulis mengidentifikasi dan mengevaluasi hasil-hasil penelitian terdahulu guna memperoleh pemahaman yang komprehensif mengenai permasalahan yang dikaji. Selain itu, tinjauan pustaka berfungsi untuk menentukan ruang lingkup, serta posisi penelitian ini dalam konteks keilmuan yang lebih luas. Dengan demikian, bagian ini menjadi landasan teoritis yang memperkuat argumentasi dan arah penelitian yang dilakukan.

2.1.1 Studi Kasus *Library* JavaScript dan Visualisasi Data

Guna melakukan perbandingan terhadap Chart.js, Highcharts, dan D3.js, penulis meninjau sumber-sumber yang berkaitan dengan penggunaan *library-library* tersebut dalam memvisualisasikan data perangkat IoT berukuran besar, beberapa diantaranya adalah studi yang dilakukan oleh Vučetić [10] Manna dan Banerjee [11], Toppany et.al [12], dan Rothenhausler [13].

Pada jurnalnya yang berjudul “*Open Data Visualization by Using Javascript Libraries*”, Vučetić [10] melakukan penelitian mengenai visualisasi data open source menggunakan *library* JavaScript, dengan fokus pada data dari pemerintahan yang dapat diakses secara bebas. Setelah melakukan perbandingan terhadap beberapa *library* visualisasi, Vučetić memilih Highcharts sebagai *library* untuk menampilkan data grafik statistik yang memungkinkan pengguna melihat jumlah perpustakaan berdasarkan distrik. Untuk pembuatan peta Interaktif yang menampilkan lokasi perpustakaan di berbagai distrik, Vučetić menggunakan Leaflet.js.

Manna dan Banerjee [11] mengimplementasikan Highcharts dalam pengembangan visualisasi perangkat IoT. Dalam implementasinya, Highcharts digunakan sebagai alat visualisasi data *real-time* pada *user interface* (UI) berbasis web. Highcharts dipilih karena kemampuannya menampilkan data sensor secara interaktif, termasuk suhu ruangan, tingkat cahaya, dan aktivitas akses pintu berbasis RFID. Dalam sistem ini, Highcharts diintegrasikan dengan HTML dan JavaScript untuk menampilkan grafik dinamis yang diperbarui secara otomatis tanpa perlu *refresh* halaman. Grafik yang digunakan mencakup grafik perbandingan cahaya dan

waktu, grafik perbandingan suhu dan waktu, dan log akses RFID, yang dihasilkan dari data yang dikirimkan oleh sensor melalui MQTT broker di AWS. Dengan Highcharts, pengguna dapat memantau kondisi secara *real-time*, melihat detail titik data dengan fitur *hover* dan *zoom*, serta memahami tren perubahan lingkungan dengan lebih mudah.

Pada sebuah penelitian dengan judul “Integrasi Framework Bootstrap dan Chart.js untuk Visualisasi Data Sensor pada Sistem Hidroponik Berbasis Internet of Things (IoT)” yang dilakukan oleh Toppany et.al [12], peneliti mengembangkan sistem visualisasi data sensor berbasis Internet of Things (IoT) yang dapat diakses melalui website. Pengujian dan evaluasi sistem ini mencakup responsivitas website, kecepatan pemrosesan data, serta kestabilan sistem dalam menangani banyak pengguna. Hasilnya, website yang dikembangkan dapat memuat halaman dalam 2 detik dan menangani *user request* dengan latensi 354 ms. Sistem berhasil menampilkan data suhu, pH, dan TDS air secara *real-time* dalam bentuk grafik. Pengujian dengan 500 *user* dalam satu menit. Hal ini menunjukkan bahwa website dapat menangani beban tanpa mengalami *crash*. *Delay* dalam pengiriman data dari ESP-32 ke website rata-rata 12 menit 1 detik, dikarenakan sistem dirancang untuk melakukan *sampling* data secara berkala guna mencegah *overload* database.

Dalam penggunaan D3.js, Rothenhausler [13] melakukan studi untuk menganalisis performa D3.js dalam memvisualisasikan data pengungsi Ukraina akibat konflik bersenjata. Dalam penelitian ini, D3.js dijelaskan sebagai *library* yang memberikan kontrol penuh terhadap manipulasi elemen dalam *Document Object Model* (DOM) yang akan memudahkan pengembang untuk membuat *chart* yang fleksibel. Penelitian ini menguji performa D3.js dalam menangani jumlah elemen yang besar dalam sebuah diagram. Hasil dari penelitian ini menunjukkan, animasi pada D3.js bekerja dengan baik hingga batas tertentu. D3 melakukan manipulasi langsung pada DOM, sehingga dapat menyebabkan *lag* jika terlalu banyak elemen yang diperbarui secara bersamaan. Berdasarkan hasil pengujian terhadap kemampuan *render*, pada dataset hingga 1200 elemen, D3 bisa berjalan lancar, tetapi di atas itu, diperlukan optimasi tambahan atau beralih ke teknologi berbasis Canvas/WebGL untuk performa yang lebih baik.

Dalam melakukan pengambilan data, penulis perlu mempertimbangkan data yang akan digunakan agar sesuai dengan data lapangan. Dalam hal ini, penulis merujuk pada jurnal penelitian “Sistem Pemantauan Lingkungan Menggunakan Sensor BME280 Berbasis Internet of Things” [14]. Penelitian ini mengembangkan sistem pemantauan lingkungan berbasis *Internet of Things* (IoT) dengan

menggunakan sensor BME280 yang mampu mengukur parameter suhu, kelembaban, tekanan udara, dan ketinggian. Sistem ini menggunakan NodeMCU ESP8266 sebagai mikrokontroler yang bertugas mengumpulkan data sensor dan mengirimkannya ke platform ThingSpeak untuk penyimpanan dan visualisasi. Proses pengambilan data dilakukan secara berkala untuk memastikan keakuratan hasil pemantauan. Akuisisi data dari sensor BME280 dilakukan setiap 10 detik, dengan 10 sampel digunakan untuk kalkulasi Median Filter sebelum data dikirim ke server. Setelah proses filtering, data disimpan di platform ThingSpeak setiap ± 45 detik, karena adanya batasan pembaruan minimum per 15 detik di platform tersebut.

Penulis menggunakan rujukan jurnal dari Melo et al. dalam judul “*A Low-Cost IoT System for Real-Time Monitoring of Climatic Variables and Photovoltaic Generation for Smart Grid Application*” [15] untuk menentukan batas jumlah data maksimal pada halaman website. Jurnal ini membahas pengembangan dasbor pemantauan cuaca secara *real-time* melalui MQTT dan membuat visualisasi dengan Chart.js. Dalam pengembangannya, dashboard ini hanya menampilkan 1440 titik data untuk menjaga kelancaran *rendering*, kemudian mengganti data terlama saat terdapat titik baru.

Tabel 2.1 Perbandingan penelitian

Judul	<i>Library/ Tools</i>	Studi Kasus
<i>“Open Data Visualization by Using Javascript Libraries”</i> [10]	Leaflet.js, HighChart.js, Chart.js, dan D3.js.	<i>Library</i> Highcharts digunakan untuk memvisualisasikan data grafik yang menampilkan jumlah perpustakaan berdasarkan distrik.

Judul	Library/ Tools	Studi Kasus
<i>“Experimental Study on MQTT Protocol Based Home Automation System with Enhancement”</i> [11]	Highcharts	Highcharts digunakan dalam UI berbasis web untuk menampilkan data sensor secara <i>real-time</i> seperti suhu ruangan, tingkat cahaya, dan aktivitas akses pintu berbasis RFID, dan menyediakan grafik interaktif yang memungkinkan pengguna melihat tren data dari waktu ke waktu, serta mendukung pemantauan jarak jauh. grafik dapat diperbarui secara <i>real-time</i> dan Highcharts mampu menangani data dari berbagai sensor dengan latensi rendah.
<i>“Integrasi Framework Bootstrap dan Chart.js untuk Visualisasi Data Sensor pada Sistem Hidroponik Berbasis Internet of Things (IoT)”</i> [12]	Chart.js	- Memuat halaman dalam 2 detik dan menangani user request dengan latensi 354 ms. - Pengujian dengan 500 pengguna dalam 1 menit menunjukkan bahwa situs web dapat menangani beban tanpa mengalami <i>crash</i>. - Delay dalam pengiriman data dari ESP-32 ke website rata-rata 12 menit 1 detik, dikarenakan sistem dirancang untuk melakukan <i>sampling</i> data secara berkala guna mencegah <i>overload</i> database.

Judul	Library/ Tools	Studi Kasus
“D3.js and its potential in data visualization Creating a diagram showcase using Ukrainian refugee data” [12]	D3	D3 mampu menangani hingga 1200-1300 elemen animasi secara <i>smooth</i> , tetapi performa mengalami penurunan karena kompleksitas visualisasi dan spesifikasi perangkat yang digunakan.
“Sistem Pemantauan Lingkungan Menggunakan Sensor BME280 Berbasis Internet of Things” [12]	Sensor BME280	Durasi pengambilan data kurang lebih 45 detik. Hal tersebut meliputi interval antar data (10 detik) dan proses pengiriman data ke ThinkSpeak termasuk proses <i>median filter</i> serta perhitungan latensi(20-35 detik).
“A Low-Cost IoT System for Real-Time Monitoring of Climatic Variables and Photovoltaic Generation for Smart Grid Application” [12]	Chart.js	Jurnal ini memvisualisasikan data <i>real-time</i> dengan Chart.js dan membatasi data yang ditampilkan pada Chart.js sebanyak 1440 data untuk mengefisienkan performa <i>rendering</i> .

Berdasarkan kajian pustaka yang telah dilakukan, Chart.js, D3.js, dan Highcharts dapat diimplementasikan sebagai *chart* untuk menampilkan data-data IoT. Tiap-tiap *Library* ini memiliki kelebihan dan kekurangannya masing-masing. Oleh karena itu, *library-library* tersebut akan diujikan dengan data *dummy* yang telah disamakan formatnya dengan data IoT asli yang merujuk pada jurnal-jurnal yang telah disebutkan sebagai referensi untuk menguji kemampuannya dalam menampilkan data dalam jumlah tertentu.

2.1.2 Studi Analisis Perbandingan

Penulis meninjau berbagai sumber pustaka akademik yang membahas topik serupa yaitu perbandingan antara *library* visualisasi JavaScript, sebagai landasan sekaligus bahan perbandingan untuk melakukan penelitian ini. Studi-studi ini melakukan penelitian dengan berbagai metode dan berbagai *library* JavaScript selain ChartJS, Highcharts, dan D3.js.

Danielyan [16] melakukan penelitian untuk meninjau perbandingan *library*

visualisasi JavaScript dalam judul "*Analysis of Data Visualization Tools and Approaches*". Studi ini menganalisis berbagai *tools* visualisasi data berdasarkan aspek biaya, kemudahan penggunaan, kemampuan, serta dokumentasi dan dukungan. *Library* yang diujikan pada penelitian ini adalah D3.js, Chart.js, Chartist.js, FusionCharts, dan Google Charts. Hasilnya, pada parameter popularitas, D3.js menjadi *library* paling populer dengan lebih dari 80.000 GitHub *stars*. Dari parameter kemudahan penggunaan, Chart.js, Chartist.js, dan Google Charts cenderung lebih mudah karena berbasis *chart object constructor*, dan D3.js menjadi *library* yang paling *customizable* namun membutuhkan lebih banyak kode. Dari parameter jenis *chart*, FusionCharts menyediakan lebih dari 95 jenis *chart* bawaan, termasuk grafik geospasial. Google Charts mendukung lebih dari 25 tipe *chart* akan tetapi membutuhkan koneksi internet dalam implementasinya.

Gokhale dan Mahajan [17] dalam penelitiannya yang berjudul "*Comparative Study of Data Visualization Tools*" membandingkan berbagai alat visualisasi data berdasarkan beberapa aspek utama, yaitu ketersediaan *open-source*, kebutuhan pemrograman, bahasa pemrograman yang digunakan, serta jenis output grafis yang dihasilkan. Dari segi ketersediaan, beberapa *library* bersifat *open-source* dan dapat digunakan secara gratis, seperti D3.js, Chart.js, Leaflet, Dygraphs, Google Charts, dan Timeline.js. Sementara itu, terdapat pula *library* yang bersifat komersial dan memerlukan lisensi, seperti Microsoft Excel, Highcharts, dan FusionCharts, yang umumnya menawarkan fitur lebih lengkap bagi pengguna. Dalam hal kebutuhan pemrograman, Microsoft Excel dan Timeline.js, dapat digunakan tanpa pemrograman, sehingga lebih mudah diakses oleh pengguna yang tidak memiliki latar belakang teknis. Sebaliknya, *library* seperti D3.js, Chart.js, Highcharts, Leaflet, Dygraphs, FusionCharts, dan Google Charts memerlukan pemahaman bahasa pemrograman, khususnya JavaScript. Sementara itu, Microsoft Excel menggunakan *Visual Basic for Applications* (VBA) untuk kebutuhan analisis data yang lebih kompleks. Dari segi jenis output grafis yang dihasilkan, Microsoft Excel, D3.js, Chart.js, Highcharts, FusionCharts, dan Google Charts banyak digunakan untuk menghasilkan berbagai jenis grafik, seperti *bar chart*, *line chart*, dan *pie chart* sementara D3.js, Highcharts, Google Charts, dan Dygraphs, juga dapat digunakan untuk memvisualisasikan *bar chart*, *line chart*, dan *pie chart* dengan fitur yang lebih interaktif. Leaflet dimanfaatkan sebagai visualisasi berbasis peta, sedangkan Timeline.js lebih cocok untuk menampilkan data dalam bentuk *time-series* yang interaktif.

Dalam penelitiannya yang berjudul "*Scalability of JavaScript Libraries for Data*

Visualization”, Persson [4] melakukan analisis dengan tujuan mengetahui apakah ukuran dataset dan/atau jenis grafik mempengaruhi waktu *render* pada *library* JavaScript yang dibandingkan dan apakah tipe *chart* mempengaruhi performa *rendering*. Hasilnya, ukuran dataset mempengaruhi waktu *render* pada semua jenis *chart*. Hasil ANOVA-test menunjukkan *p-value* sebesar 0,61 dengan *confidence level* 95% yang berarti tidak ada perbedaan yang signifikan dalam waktu *render* antara grafik garis dan diagram batang ketika dataset terdiri dari 100 titik data. Hasilnya serupa untuk set data 5 ribu titik dengan nilai *p-value* 0,54 dan *confidence level* 95% dan dengan demikian tidak ada perbedaan yang signifikan perbedaan waktu *render* antara diagram garis dan batang ketika dataset terdiri dari lima ribu titik.

Bennba juga turut melakukan penelitian mengenai *library* Javascript untuk visualisasi dalam judul tesisnya “*Comparison of D3.js and Chart.js as Visualization Tools*” [18]. Dalam tesisnya, Bennba melakukan perbandingan terhadap Chart.JS dan D3.js untuk melakukan uji performa pada waktu eksekusi, *load time*, dan penggunaan memori. Uji performa dilakukan dengan memvisualisasikan data menggunakan *line chart* dan untuk membuat grafik garis dengan dataset berukuran 10, 100, 1.000, 10.000, 100.000, dan 1.000.000 data. Hasilnya, D3.js lebih cepat dibandingkan Chart.js dalam hal waktu eksekusi, terutama pada dataset besar. Sementara itu, Chart.js mengalami *crash* saat mencoba me-*render* 1.000.000 data. Sedangkan dalam hal penggunaan memori, jika setiap titik data di-*render* sebagai elemen SVG individual, D3 mengalami peningkatan konsumsi memori secara drastis.

Penelitian serupa juga dilakukan oleh Bostromm, dkk [3] dalam judul “*A Performance Investigation into JavaScript Visualization Libraries With the Focus on Render Time and Memory Usage*”. Penelitian ini bertujuan untuk menentukan apakah ukuran dataset mempengaruhi waktu *render* dan penggunaan memori, baik di dalam *library* itu sendiri maupun di antara semua *library* JavaScript yang dipilih serta meneliti apakah terdapat perbedaan waktu *render* yang signifikan ketika terdapat peningkatan poin data. Hasilnya, ukuran dataset mempengaruhi waktu *render*. Semua *library* mengalami peningkatan waktu *render* seiring bertambahnya jumlah data, kecuali dalam beberapa kasus anomali, seperti ECharts yang menunjukkan waktu *render* lebih rendah di kategori 2 dibanding kategori 1. Selain itu, ANOVA test menunjukkan nilai *p* kurang dari 0.05 ketika membandingkan waktu *render* antara *line chart* dan *bar chart*, baik untuk dataset kecil (100 data) maupun besar (5000 data), sehingga tidak ada perbedaan signifikan antara kedua tipe *chart* ini.

Perbandingan performa *render* dan waktu respon juga diteliti oleh Millqvist

dan Bolin [19] dalam tesisnya yang berjudul “*A Comparison of Performance and Scalability of Chart Generation for Javascript Data Visualization Libraries*”. Studi ini membandingkan performa lima *library* JavaScript, yaitu Chart.js, ApexCharts, Billboard.js, ToastUI, dan Chartist dengan fokus penelitian untuk mengukur waktu respon dalam *me-render pie charts, bar charts, line charts, dan scatter plots*. Pengujian dilakukan dengan parameter waktu respon dari setiap *library* dalam *me-render* grafik, skalabilitas berdasarkan ukuran dataset dari 100 hingga 1.000.000 data, serta normalisasi grafik. Hasil dari penelitian ini menunjukkan bahwa Chart.js dapat memproses hingga 1.000.000 data tanpa mengalami *crash*. Chartist memiliki performa *render* yang cepat tetapi gagal *me-render* dataset lebih dari 100.000 data. Sedangkan ApexCharts dan Billboard mengalami kendala eksponensial seiring dengan meningkatnya waktu respon. Dari kelima *library* yang diujikan, Chart.js adalah pustaka paling stabil yang mampu menangani semua ukuran dataset tanpa mengalami *crash*.

Kim et al [20], dalam jurnal penelitiannya yang berjudul “*Beyond Alternative Text and Tables: Comparative Analysis of Visualization Tools and Accessibility Methods*”, meninjau tentang penggunaan *library* JavaScript, termasuk Highchart dan Tableau dalam pengembangan *website* yang mendukung aksesibilitas dan skalabilitas. Penulis melakukan analisis terhadap 39 alat visualisasi data berbasis web, termasuk 30 *library* pemrograman dan 9 *framework* visualisasi dengan metode pencarian web, filtrasi data, dan evaluasi fitur aksesibilitas. Dari hasil evaluasi, Highcharts adalah salah satu dari sedikit alat yang mendukung pembaruan data *real-time* yang memungkinkan kontrol jumlah data yang ditampilkan sehingga kompleksitas *render* dapat berkurang.

Berdasarkan tinjauan pustaka yang telah dikaji sebelumnya, perbandingan penelitian tersebut dijelaskan dalam tabel berikut :

Tabel 2.2 Perbandingan Penelitian Library Visualisasi JavaScript

Judul	Library	Metric Komparasi	Hasil
<i>Analysis of Data Visualization Tools and Approaches</i> [18]	D3.js, Chart.js, Chartist.js, Google Chart	Popularitas <i>Library</i> , Kemudahan Penggunaan, Pendekatan grafis (SVG atau Canvas), <i>Built-in Charts & Customization</i>	D3.js lebih dari 80.000 GitHub <i>stars</i> dan 123 kontributor. Chart.js dengan lebih dari 40.000 stars dan 284 kontributor. Chartist.js dengan 11.000 stars dan 67 kontributor.
<i>Comparative Study of Data Visualization Tools</i> [19]	Microsoft Excel, D3.js, Chart.js, Highcharts, Leaflet, Dygraphs, FusionCharts, Google Charts, Timeline.js	Ketersediaan <i>open-source</i> , kebutuhan pengetahuan pemrograman, format output grafis	Library <i>open-source</i> : D3, Chart.js, Leaflet, Dygraphs, Google Charts, Timeline.js. D3, Chart.js, Highcharts, Leaflet, Dygraphs, Google Charts, FusionCharts perlu kemampuan <i>coding</i> . Mayoritas berbasis JavaScript. Microsoft Excel hanya chart statis, D3 mendukung chart interaktif, Chart.js fokus animasi chart, Highcharts, Dygraphs, Google Charts unggul dalam interaktivitas, Leaflet untuk peta interaktif, FusionCharts mendukung chart dan peta, Timeline.js untuk timeline interaktif.

Judul	Library	Metric Komparasi	Hasil
<i>Scalability of JavaScript Libraries for Data Visualization</i> [4]	Apexcharts, Frappe Charts, TeeCharts JS, jQuery	ANOVA-test pada ukuran dataset	Ukuran dataset mempengaruhi waktu render di semua <i>chart</i> . Tidak ada perbedaan signifikan waktu <i>render</i> antara diagram garis dan batang pada dataset 5.000 titik.
<i>Comparison of D3.JS and Chart.JS as Visualization Tools</i> [20]	D3.JS, Chart.JS	Waktu eksekusi, <i>load time</i> , penggunaan memori	D3 lebih cepat dari Chart.js untuk dataset besar. Chart.js <i>crash</i> saat merender 1.000.000 data. Chart.js lebih efisien dalam memori jika tiap titik data dirender sebagai elemen SVG individual.
<i>A performance investigation into JavaScript visualization libraries with the focus on render time and memory usage</i> [3]	D3, ECharts, CanvasJS, Chartist, Highcharts, Plotly	ANOVA-Test dan Tukey-test pada waktu <i>render</i>	Ukuran dataset mempengaruhi waktu <i>render</i> . Semua <i>library</i> mengalami peningkatan waktu <i>render</i> seiring bertambahnya data, meskipun ada beberapa anomali.

Judul	Library	Metric Komparasi	Hasil
<i>A Comparison of Performance and Scalability of Chart Generation for Javascript Data Visualization Libraries</i> [21]	Chart.js, ApexCharts, Chartist, Billboard	Performa render	Chart.js mampu memproses hingga 1.000.000 data tanpa <i>crash</i> . Chartist cepat tetapi gagal merender dataset lebih dari 100.000 data. ApexCharts dan Billboard mengalami peningkatan waktu respon secara eksponensial.
<i>Beyond Alternative Text and Tables: Comparative Analysis of Visualization Tools and Accessibility Methods</i> [22]	Highcharts, Vega-Lite, Visa Chart, amCharts, FusionCharts, AnyChart	Aksesibilitas Chart, cara menampilkan data real-time	Highcharts memiliki dukungan aksesibilitas terbaik berupa navigasi keyboard multi-level, mode kontras tinggi, dan sonifikasi. Mendukung pembaruan data <i>real-time</i> dengan kontrol jumlah data agar kompleksitas render dapat dikurangi.

Berdasarkan penelitian-penelitian sebelumnya, setiap studi memiliki metrik penilaiannya masing-masing dalam melakukan komparasi, seperti kemudahan penggunaan, performa *rendering*, ketersediaan jenis *chart*, waktu eksekusi, konsumsi memori, serta metrik lainnya sebagaimana dijelaskan pada paragraf dan tabel sebelumnya. Oleh karena itu, dalam penelitian ini, penulis melakukan komparasi terhadap tiga *library* visualisasi berbasis JavaScript, yaitu Chart.js, D3.js, dan Highcharts, dengan fokus pada studi kasus *dashboard website* untuk menampilkan data sensor IoT secara *real-time*.

Pengujian dilakukan dengan mengacu pada metrik-metrik yang telah digunakan dalam penelitian terdahulu, yaitu performa *rendering* untuk memastikan kecepatan dan kelancaran tampilan data yang terus diperbarui, penggunaan memori untuk

melihat efisiensi dalam penanganan elemen-elemen visual yang dihasilkan, serta alokasi CPU untuk meninjau sejauh mana beban pemrosesan memengaruhi kinerja sistem secara keseluruhan. Selain itu, skalabilitas masing-masing *library* juga diamati untuk memastikan kemampuan visualisasi dalam menangani penambahan volume data sensor IoT yang dapat berkembang signifikan dalam implementasi nyata. Dengan pendekatan ini, diharapkan hasil pengujian dapat menjadi acuan praktis bagi pengembang dalam memilih *library* yang paling sesuai untuk membangun *dashboard* IoT yang responsif, stabil, dan efisien.

2.2 Dasar Teori

Bagian dasar teori berperan untuk menjelaskan berbagai konsep dan istilah yang berkaitan dengan topik penelitian. Penjelasan ini penting untuk memahami konteks dan arah penelitian secara lebih jelas serta mengetahui acuan teoritis yang digunakan peneliti dalam menyusun kajian.

2.2.1 Laravel

Laravel merupakan kerangka kerja aplikasi web berbasis PHP yang dirancang untuk pengembang web yang membutuhkan sintaks yang mudah digunakan. Laravel juga menawarkan berbagai kebutuhan awal yang secara otomatis membuat rute, kontroler, dan tampilan yang diperlukan untuk registrasi dan autentikasi pengguna dalam aplikasi. Dengan fitur-fitur tersebut, Laravel memungkinkan pengembang untuk membangun aplikasi web yang kuat dan *scalable* dengan sintaks yang efisien [21].

2.2.2 JavaScript

JavaScript adalah bahasa pemrograman yang sering digunakan pada halaman web untuk mengolah, memanipulasi, atau memvalidasi data. JavaScript memungkinkan untuk membuat tampilan yang interaktif, mulai dari animasi sederhana hingga aplikasi web kompleks berbasis *real-time*. JavaScript menjadi salah satu bahasa pemrograman yang paling umum digunakan di dunia dengan tingkat penggunaan mencapai lebih dari 98,9% dari semua situs web yang ada [22]. Ekosistem Javascript juga luas termasuk *library* seperti jQuery dan *framework* modern seperti React.js, Angular, dan Vue.js, yang semakin mempermudah pengembangan web. Dalam konteks penelitian ini, JavaScript dalam Laravel digunakan untuk meningkatkan interaktivitas dan pengalaman pengguna di sisi klien untuk membangun *Fetch* API yang akan mengambil data dari API tanpa perlu *me-refresh* halaman.

2.2.3 *Library* JavaScript

Library JavaScript merupakan kumpulan kode yang bisa digunakan kembali dengan menyediakan fungsionalitas bawaan dalam beberapa cara untuk mempercepat pengembangan aplikasi sehingga pengembang tidak perlu mengimplementasikan fungsionalitas dasar yang sama untuk setiap situs web atau program. Sebagai gantinya, mereka dapat menggunakan kembali kode dari pustaka yang dipilih.

2.2.4 Chart.js

Chart.js adalah *library* JavaScript *open-source* yang fleksibel untuk membuat berbagai jenis grafik pada web modern. Pustaka ini mendukung beberapa jenis grafik, termasuk *bar chart*, *line chart*, *area*, *pie chart*, *bubble chart*, *radar chart*, dan *polar area*. Chart.js dirancang untuk memudahkan pengembang dalam membuat visualisasi data yang interaktif dan responsif dengan menggunakan elemen kanvas HTML5 [5].

2.2.5 D3

D3 (atau D3.js) adalah pustaka JavaScript gratis dan *open-source* yang digunakan untuk membuat visualisasi data. Dengan pendekatan berbasis web standar, D3 memberikan fleksibilitas dalam membangun grafik yang interaktif dan dinamis. D3 menawarkan penggunaan *stylesheet* eksternal untuk mengubah tampilan grafik (termasuk menyesuaikan dengan media queries untuk grafik responsif atau mode gelap). Model evaluasi D3 yang sinkron dan imperatif membuat D3 sering kali lebih mudah untuk di-*debug* dibandingkan dengan *framework* yang memiliki *runtime* asinkron yang kompleks [6].

2.2.6 Highcharts

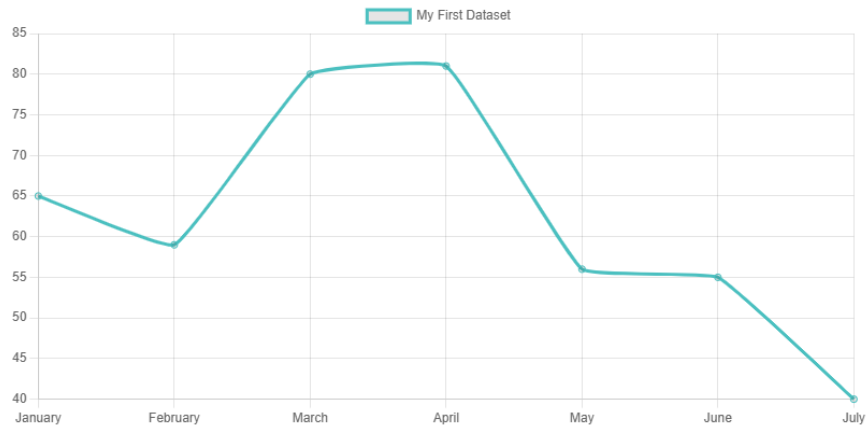
Highcharts adalah pustaka JavaScript yang memungkinkan pengembang membuat berbagai jenis grafik interaktif untuk aplikasi web. Pustaka ini mendukung berbagai jenis grafik seperti garis, batang, area, pai, dan banyak lagi. Highcharts dirancang untuk dapat diimplementasikan di semua browser modern dan kompatibel dengan perangkat seluler [23].

2.2.7 Visualisasi

Dalam bukunya, Unwin [24] menjelaskan cara utama untuk menyajikan data statistik adalah melalui representasi visual, yang disebut diagram, bagan, atau grafik. Representasi ini memberikan interpretasi data yang terorganisir, cepat, dan kuat, sehingga lebih mudah dipahami. Sehingga tujuan utama dari visualisasi data adalah untuk menampilkan data dan statistik serta menginterpretasikan tampilan tersebut guna memperoleh informasi. Dilansir dari laman Chart.js, terdapat beragam bentuk visualisasi. Beberapa diantaranya adalah :

A. *Line Chart*

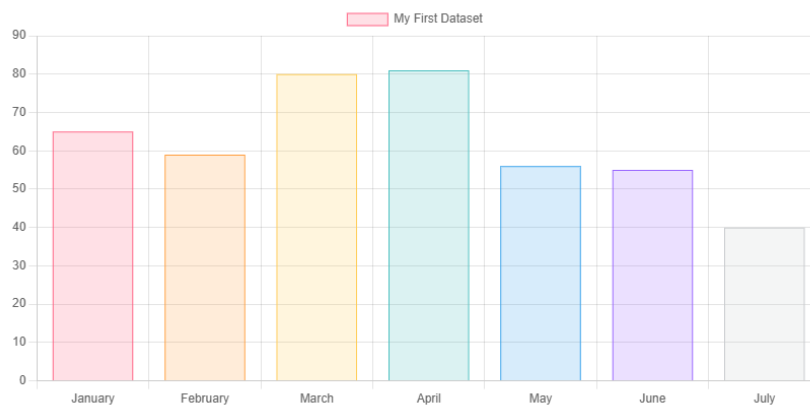
Line Chart adalah metode untuk memplot titik data pada sebuah garis. Grafik ini sering digunakan untuk menunjukkan tren data atau perbandingan antara dua set data [5].



Gambar 2.1 Bentuk *Line Chart*

B. *Bar Chart*

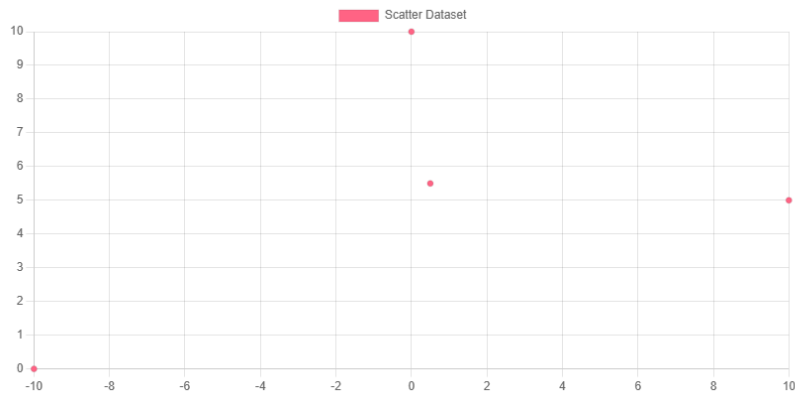
Bar Chart adalah metode untuk menampilkan nilai data dalam bentuk batang vertikal. Grafik ini seringkali digunakan untuk menunjukkan tren data serta membandingkan beberapa set data secara berdampingan [5].



Gambar 2.2 Bentuk *Bar Chart*

C. *Scatter Plot*

Scatter plot merupakan salah satu jenis grafik yang membandingkan nilai data numerik dari dua variabel menggunakan titik yang berada dalam koordinat (x,y). Biasanya, jenis grafik ini digunakan untuk membandingkan nilai antar variabel [5]



Gambar 2.3 Bentuk *Scatter Plot*

2.2.8 IoT

Internet of Things (IoT) adalah konsep yang bertujuan untuk memperluas manfaat konektivitas internet yang selalu terhubung. IoT mengacu pada sistem di mana berbagai objek di dunia nyata dapat saling berkomunikasi dalam sebuah jaringan yang terintegrasi dengan internet sebagai perantara utamanya. Secara umum, cara kerja IoT berlandaskan pada tiga elemen utama, yaitu perangkat fisik yang telah dilengkapi modul IoT, perangkat koneksi seperti modem dan *router* nirkabel untuk menghubungkan ke internet, serta pusat data berbasis *cloud* yang berfungsi sebagai tempat penyimpanan aplikasi dan basis data. Modul IoT ini akan mengumpulkan serta mengirimkan data dalam rentang waktu tertentu sesuai dengan konfigurasi yang telah ditetapkan [25].

2.2.9 Dataset

Secara umum, dataset didefinisikan sebagai kumpulan data yang dikumpulkan dan disusun dalam suatu format tertentu untuk digunakan dalam analisis lebih lanjut. Definisi dataset didefinisikan sesuai konteks penggunaannya, seperti dalam ilmu komputer, statistik, atau penelitian ilmiah [8]. Pada konteks penelitian ini, dataset ini merupakan data yang dihasilkan secara terus menerus dari perangkat IoT. Format data yang akan digunakan adalah JSON dan akan diuji cobakan sebagai data yang akan ditampilkan dalam visualisasi dan menentukan hasil evaluasi nantinya.

2.2.10 Pengujian Performa

Performa mengacu pada proses evaluasi, pengukuran, dan penilaian terhadap efisiensi dalam menyelesaikan suatu tugas. Dalam bidang komputasi, performa menggambarkan efektivitas sebuah komputer dalam menjalankan suatu proses. Indikator utama dalam mengukur performa adalah jumlah waktu dan sumber daya yang digunakan untuk menyelesaikan tugas tersebut [26]. Dalam penelitian ini,

performa yang diukur antara lain adalah *rendering*, *scalability*, dan penggunaan *resource* CPU dan memori.

A. *Rendering*

Rendering merupakan proses pembuatan tampilan visual dalam bentuk gambar atau model 3D. Terdapat dua jenis metode *rendering* yang dapat dilakukan, yaitu *Hardware rendering* dan *Software Rendering*. *Software rendering* memanfaatkan CPU untuk memproses dan menampilkan gambar, sementara *hardware rendering* menggunakan GPU untuk mempercepat proses tersebut [27].

B. CPU

Berbeda dari implementasi perangkat keras lainnya, CPU merupakan perangkat yang sangat fleksibel karena dapat diprogram melalui perangkat lunak. CPU dianggap sebagai komponen dengan konsumsi energi terbesar dalam sebuah komputer. Oleh karena itu, dalam setiap penelitian, pengukuran penggunaan CPU selalu diperhitungkan untuk memperkirakan energi yang digunakan hanya oleh sebuah program komputer. Pengukuran penggunaan CPU dapat ditinjau melalui *Task Manager* atau dengan skrip yang dibuat melalui terminal [28].

C. *Footprint Memory*

Footprint memory adalah total jumlah memori yang digunakan oleh suatu program, aplikasi, atau proses pada saat berjalan (*runtime*). Memori *footprint* digunakan untuk menggambarkan konsumsi memori dari sebuah aplikasi terhadap sistem komputer. Komponen memori *footprint* meliputi:

1. *Private memory* yang digunakan hanya pada tab spesifik
2. *Heap memory* yang menyimpan objek dan struktur data dinamis.
3. *Stack memory* yang digunakan untuk menyimpan variabel lokal dan kontrol eksekusi fungsi.
4. *Buffer* dan *cache* internal yang menyimpan data sementara dari tab *website*.

D. Scalabilitas

Konsep skalabilitas menjelaskan kemampuan suatu sistem untuk beradaptasi dengan penambahan elemen atau objek, menangani peningkatan beban kerja secara efisien, serta dapat diperluas sesuai kebutuhan. Dalam proses perancangan sistem, skalabilitas sering menjadi faktor utama yang harus dipenuhi untuk mengukur kemampuan sistem dalam menerima sumber daya [29].

2.2.11 *Random Access Memory(RAM)*

Memori adalah komponen dalam komputer yang berfungsi untuk menyimpan program dan data. Memori memiliki berbagai jenis, teknologi, struktur, dan kinerja yang bervariasi, tergantung pada cara informasi disimpan, dibaca, dan ditulis. CPU memiliki tiga tingkat *cache*, yang juga dikenal sebagai RAM internal. RAM internal membantu meningkatkan kinerja prosesor yang memungkinkan instruksi dikirim ke CPU lebih cepat dibandingkan jika diambil dari RAM eksternal. Saat CPU membutuhkan data, pencarian pertama dilakukan di *cache* Level 1 (L1), kemudian di Level 2 (L2), dan terakhir di Level 3 (L3). Jika data tidak ditemukan dalam *cache*, CPU akan mengambilnya dari DRAM [30]. Salah satu bagian dari RAM yang digunakan untuk memproses alokasi memori dinamis adalah *heap memory*. *Heap memory* akan menunjukkan distribusi memori di antara objek JavaScript pada halaman dan node DOM terkait. Penggunaan RAM ini dapat dipantau pada komputer, salah satunya melalui *Chrome Task Manager* pada kolom memori *footprint*.

2.2.12 *Chrome Task Manager*

Task Manager merupakan ekstensi Chrome yang dapat digunakan untuk memantau dan mengelola berbagai proses yang berjalan di dalam browser. Beberapa hal yang diukur dan ditampilkan oleh *Chrome Task Manager* antara lain:

1. Penggunaan CPU menunjukkan seberapa banyak prosesor yang digunakan oleh setiap proses.
2. *Memory Footprint* mengukur jumlah memori yang digunakan oleh setiap proses, termasuk tab, ekstensi, dan plugin. Hal ini membantu dalam mengidentifikasi proses yang memakan banyak RAM.
3. *Network* memantau aktivitas jaringan dari setiap proses sehingga penggunaan bandwidth dapat terpantau.
4. *Process ID* merupakan id yang dapat digunakan untuk pelacakan dan diagnostik lebih lanjut.

2.2.13 *Use Case Diagram*

Use case diagram digunakan untuk menggambarkan bagaimana sistem berfungsi dari sudut pandang penggunanya. Diagram ini menunjukkan pola interaksi yang biasa terjadi antara pengguna dan sistem. Dalam pembahasannya, pengguna sering disebut sebagai aktor, yakni pihak yang berperan langsung dalam menggunakan sistem untuk mencapai suatu tujuan. Aktor dapat berupa perorangan,

sebuah organisasi, atau sistem eksternal, yang berinteraksi dengan sistem atau aplikasi secara langsung [31].

2.2.14 *Sequence Diagram*

Sequence diagram adalah salah satu diagram *UML (Unified Modeling Language)* yang digunakan untuk memperlihatkan alur komunikasi antar objek dalam sistem seiring berjalannya waktu. Diagram ini menyusun urutan interaksi berdasarkan skenario tertentu dan mengaitkannya dengan operasi yang dimiliki oleh masing-masing objek dalam *class diagram*. Diagram ini cukup populer karena bentuk visualnya yang mudah dipahami dan kemampuannya untuk menggambarkan sebagian perilaku sistem secara spesifik. Terdapat beberapa elemen di dalam *sequence diagram* [32]. Elemen-elemen tersebut antara lain :

1. Aktor adalah pihak di luar sistem, contohnya adalah pengguna, yang berinteraksi langsung dengan sistem untuk menjalankan suatu proses.
2. Objek atau *instance* adalah bagian yang menggambarkan komponen dalam sistem yang berperan dalam suatu skenario atau alur tertentu.
3. *Lifeline* adalah garis vertikal yang menunjukkan objek aktif selama proses interaksi berlangsung.
4. Pesan ditampilkan dalam bentuk panah antar objek yang merepresentasikan komunikasi, baik yang berjalan secara sinkron maupun asinkron.
5. *Activation bar* adalah blok kecil di atas *lifeline* yang menunjukkan bahwa objek tersebut sedang menjalankan aktivitas tertentu pada waktu itu.
6. *Fragments* bersifat opsional dan biasanya digunakan untuk menunjukkan percabangan alur, seperti kondisi pilihan, aksi opsional, atau perulangan.

BAB III

METODOLOGI PENELITIAN

3.1 Bahan

Dalam proses penelitian, diperlukan data yang akan digunakan sebagai bahan pengujian untuk mengukur dan mengevaluasi kinerja metode yang diterapkan. Bahan uji yang digunakan merupakan data *dummy* yang dirancang dengan struktur menyerupai dataset asli yang diperoleh dari perangkat sensor IoT. Meskipun bukan data asli, dataset ini tetap mempertahankan variabel dan pola data yang umumnya ditemukan pada sensor IoT. Dengan menggunakan dataset *dummy*, penelitian ini dapat mensimulasikan kondisi real dari hasil sensor tanpa harus bergantung pada data aktual, sehingga memungkinkan pengujian yang lebih fleksibel dan berulang. Selain itu, pendekatan ini juga membantu dalam memahami bagaimana sistem bekerja dalam berbagai skenario yang mungkin terjadi pada sistem berbasis IoT, termasuk skenario yang sulit diperoleh dari data sensor asli. Pembuatan data IoT ini merujuk pada jurnal-jurnal yang membahas mengenai penggunaan sensor baik sebagai *single-sensor* maupun *multiple-sensor*. Penulis merancang data tersebut dalam format API yang mengacu pada penelitian yang dilakukan oleh Triawan dkk. [14] serta Kasera dan Acharjee [33].

a. Dummy Single Sensor BME380

Berdasarkan data yang diperoleh dari jurnal berjudul “Sistem Pemantauan Lingkungan Menggunakan Sensor BME280 Berbasis Internet of Things” [14], struktur data dalam format JSON yang digunakan adalah sebagai berikut :

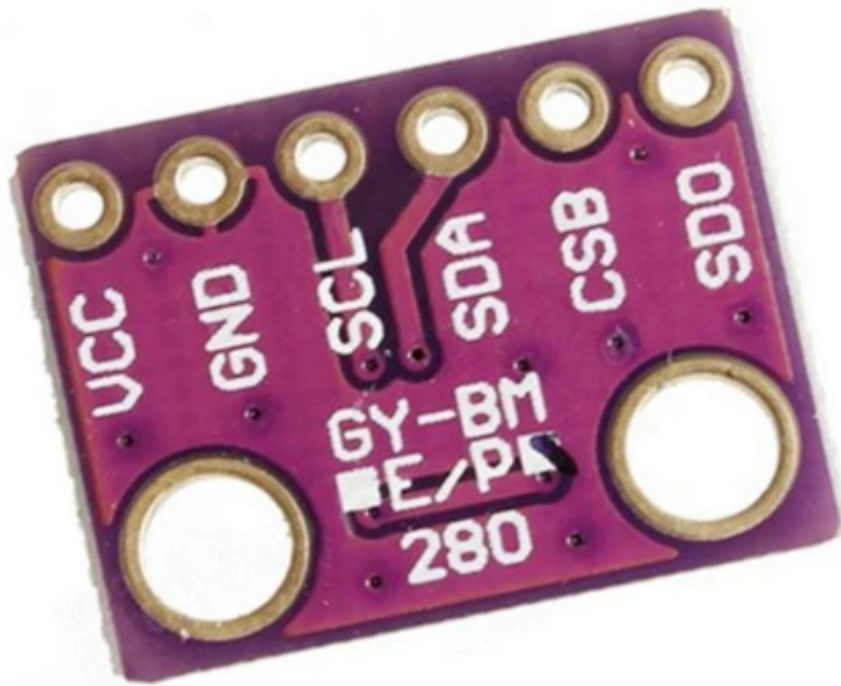
Tabel 3.1 Struktur JSON dari Sensor

Struktur JSON:	
<pre>[{ "altitude": 73.42, "humidity": 50.34, "pressure": 1013.04, "temperature": 29.36, "timestamp": "2025-03-05 12:45:42" }, { "altitude": 86.22, "humidity": 59.96, "pressure": 1019.2, "temperature": 27.94, "timestamp": "2025-03-05 12:45:57" }, { "altitude": 28.59, "humidity": 46.03, "pressure": 1015.18, "temperature": 24.29, "timestamp": "2025-03-05 12:46:12" }, { "altitude": 80.11, "humidity": 53.01, "pressure": 1005.49, "temperature": 26.54, "timestamp": "2025-03-05 12:46:27" }]</pre>	
Parameter	Keterangan
altitude	Posisi vertikal (ketinggian) suatu objek dari suatu titik tertentu (datum). Dalam hal ini adalah mdpl.
humidity	Kelembaban yang ditangkap dari sensor.
pressure	Tekanan udara yang ditangkap dari sensor.
temperature	Suhu yang ditangkap dari sensor.
timestamp	Waktu saat data ter- <i>generate</i> .

Pada implementasinya, data dari referensi asli diolah menggunakan bahasa pemrograman Python hingga data menjadi API dalam format JSON. Tahapan pengolahan data *dummy* tersebut dijelaskan sebagai berikut :

1. Mengidentifikasi jenis sensor yang akan digunakan. Dalam hal ini, sensor tersebut adalah sensor BME280. Sensor ini merupakan modul sensor yang dapat mengukur kelembaban, suhu, tekanan barometrik, dan ketinggian.

Pada penerapannya, sensor ini dapat digunakan dalam banyak lini, beberapa diantaranya adalah pertanian, perkebunan, dan penerapan lainnya yang berhubungan dengan cuaca. Suryana dalam penelitiannya menjelaskan sensor BME280 mudah digunakan karena tidak memerlukan komponen tambahan lain dan telah memiliki fitur *pre-calibrated* [34].



Gambar 3.1 Sensor BME280

2. Menentukan rentang nilai yang akan dijadikan data *dummy* dengan merujuk pada penelitian yang membahas mengenai implementasi sensor BME280 pada studi kasus pengukuran stasiun cuaca dalam 3 hari berturut-turut. Mengacu pada penelitian tersebut, rentang data yang dihasilkan sensor dalam 10 detik berkisar sekitar 28,47°C - 34,46 °C untuk suhu, 50,78% - 59,17% untuk kelembaban, 1008,27 hPa - 1009,24 hPa untuk tekanan udara, dan 40,14 - 63,67 m untuk ketinggian [14].
3. Membuat file berformat Python dengan nama sensor-data.py.
4. Menginstal Flask dengan perintah `pip install flask-cors`. Flask merupakan salah satu *framework* Python yang ringan dan mudah dikustomisasi serta dapat dimanfaatkan dalam pengembangan website. Flask menyediakan fitur yang dapat disesuaikan dengan kebutuhan

pengembang tanpa harus berpatokan dengan standar atau struktur sebuah *framework* [35]. Pada konteks ini, Flask dimanfaatkan sebagai *framework* untuk pembuatan data API *dummy* yang akan digunakan untuk pengujian. Alasan pemilihan *framework* ini adalah karena ukurannya yang kecil dan fleksibilitasnya sehingga tidak membebani memori serta dapat dikonfigurasi dengan mudah.

```
PS D:\VAPID> pip install flask-cors
Requirement already satisfied: flask-cors in c:\python311\lib\site-packages (5.0.1)
Requirement already satisfied: flask<0.9 in c:\python311\lib\site-packages (from flask-cors) (3.0.3)
Requirement already satisfied: Werkzeug>=0.7 in c:\python311\lib\site-packages (from flask-cors) (3.0.3)
Requirement already satisfied: Jinja2>=3.1.2 in c:\python311\lib\site-packages (from flask-cors) (3.1.4)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\python311\lib\site-packages (from flask-cors) (2.2.0)
Requirement already satisfied: click>=8.1.3 in c:\python311\lib\site-packages (from flask-cors) (8.1.7)
Requirement already satisfied: blinker>=1.6.2 in c:\python311\lib\site-packages (from flask-cors) (1.8.2)
Requirement already satisfied: MarkupSafe>=2.1.1 in c:\python311\lib\site-packages (from Werkzeug>=0.7->flask-cors) (2.1.5)
Requirement already satisfied: colorama in c:\python311\lib\site-packages (from click>=8.1.3->flask-cors) (0.4.6)

[notice] A new release of pip is available: 24.0 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Gambar 3.2 Install Flask

5. Menerapkan beberapa *library*, seperti berikut :

```
1  from flask import Flask, jsonify
2  from flask_cors import CORS
3  import random
4  import threading
5  import time
6  from collections import OrderedDict # Gunakan OrderedDict
```

Gambar 3.3 Library

- a. *jsonify*: Mengubah data menjadi format JSON untuk dikirim ke *client*.
- b. *CORS*: Mengizinkan permintaan dari domain berbeda (*Cross-Origin Resource Sharing*). Digunakan untuk menghubungkan port 5000 dari python dan port 8000 dari HTTP.
- c. *random*: Digunakan untuk menghasilkan data acak untuk membuat simulasi sensor.
- d. *threading*: Digunakan untuk menjalankan fungsi secara paralel memperbarui data sensor secara terus-menerus.
- e. *time*: Mengatur *delay* dalam proses pembaruan data.

6. Inisialisasi Flask dan Konfigurasi CORS

```
8  app = Flask(__name__)
9  CORS(app)
```

Gambar 3.4 Inisialisasi Flask dan CORS

7. Variabel `sensor_data_list` digunakan untuk menyimpan data sensor yang terus diperbarui.

```
10  
11 sensor_data_list = []  
12 sensor_id = 1  
13
```

Gambar 3.5 variabel

8. Membuat fungsi untuk membuat data sensor yang menghasilkan dan menyimpan data setiap 10 detik dalam rentang data 28.47°C - 34.53°C untuk suhu, 59.13% - 50.78% untuk humidity, 1008 hPa - 1011 hPa untuk pressure, dan 40.16 m - 63.67m untuk *altitude*. Sensor ini dibuat untuk terus me-*looping* pembuatan data dengan struktur:
- a. Id nilai unik untuk mengidentifikasi data
 - b. *Timestamp* dalam format standar ISO 8601
 - c. *Temperature* dalam satuan derajat celcius hingga derajat celcius
 - d. *Humidity* dalam satuan RH
 - e. *Pressure* dalam satuan HPa
 - f. *Altitude* dalam satuan meter

```

1 last_values = {
2     "temperature": round(random.uniform(28.47, 34.53), 2),
3     "humidity": round(random.uniform(50.78, 59.13), 3),
4     "pressure": round(random.uniform(1008, 1011), 3),
5     "altitude": round(random.uniform(40.16, 63.67), 3)
6 }
7
8 def generate_next_value(last_value, min_value, max_value, max_change):
9     change = round(random.uniform(-max_change, max_change), 3) # Random dengan batas perubahan
10    new_value = round(last_value + change, 3)
11    return max(min_value, min(max_value, new_value))
12
13
14 def generate_sensor_data():
15     global sensor_id, last_values, start_time
16
17     while True:
18         elapsed_time = (time.time() - start_time) / 60 # Hitung waktu dalam menit
19
20         # Periksa apakah sudah mencapai 45, 90, 135 menit, dst.
21         if (elapsed_time >= 120): # Setelah 2 jam, ubah maksimal 2
22             temp_change = 2.0
23         elif int(elapsed_time) % 90 == 45: # Setelah 45, 90, 135 menit, dst.
24             temp_change = round(random.uniform(0.1, 0.2), 3)
25         else:
26             temp_change = 0.04 # Normal: ±0.04 setiap 10 detik
27
28         new_data = OrderedDict([
29             ("id", sensor_id),
30             ("timestamp", time.strftime("%Y-%m-%d %H:%M:%S")),
31             ("temperature", generate_next_value(last_values["temperature"], 28.47, 34.53,
temp_change)),
32             ("humidity", generate_next_value(last_values["humidity"], 50.78, 59.13, 0.004)),
33             ("pressure", generate_next_value(last_values["pressure"], 1008, 1011, 0.001)),
34             ("altitude", generate_next_value(last_values["altitude"], 40.16, 63.67, 0.008))
35         ])
36
37         # Simpan nilai terbaru sebagai referensi untuk iterasi selanjutnya
38         last_values = new_data
39
40         sensor_data_list.append(new_data)
41         print(f"Data baru ditambahkan: {new_data}")
42         sensor_id += 1
43         time.sleep(10) # Update setiap 10 detik
44

```

Gambar 3.6 Membuat Sensor *Dummy*

9. Membuat API *Endpoint*. API *endpoint* ini akan digunakan untuk mengambil data dan mengurutkannya dalam format JSON dengan *library* jsonify.

```

32 @app.route('/sensor-data', methods=['GET'])
33 def get_sensor_data():
34     """Mengembalikan semua data sensor dengan urutan tetap"""
35     return jsonify(sensor_data_list) # jsonify akan tetap menjaga urutan OrderedDict
36
37 if __name__ == '__main__':
38     thread = threading.Thread(target=generate_sensor_data, daemon=True)
39     thread.start()
40     app.run(host="127.0.0.1", port=5000, debug=True)
41

```

Gambar 3.7 API Endpoint

10. Menjalankan program *threading* pada fungsi `generate_sensor_data` secara berkala, paralel dengan server Flask untuk menangani *request*.

```

32 @app.route('/sensor-data', methods=['GET'])
33 def get_sensor_data():
34     """Mengembalikan semua data sensor dengan urutan tetap"""
35     return jsonify(sensor_data_list) # jsonify akan tetap menjaga urutan OrderedDict
36
37 if __name__ == '__main__':
38     thread = threading.Thread(target=generate_sensor_data, daemon=True)
39     thread.start()
40     app.run(host="127.0.0.1", port=5000, debug=True)
41

```

Gambar 3.8 Menjalankan Program Threading

11. Menjalankan server tersebut dengan `python sensor-data.py`.

```

PS D:\API> python sensor-data.py

```

Gambar 3.9 Menjalankan Sensor

b. *Multiple Sensor*

Data yang dijadikan acuan sebagai contoh data yang mengintegrasikan beberapa sensor pengukuran adalah data yang merujuk pada jurnal “A Comprehensive IoT Edge Based Smart Irrigation System for Tomato Cultivation” [33]. Dataset ini berisi hasil pengambilan data dari sensor IoT pada kultivasi tomat yang mengukur faktor-faktor yang mempengaruhi pertumbuhan tomat. Terdapat beberapa sensor yang diterapkan pada pengujian ini, seperti diantaranya sensor BME280, modul NRF24LO1, raspberry pi, mikrokontroler ESP32, ESP8266, LoRaWAN, dan modul serta perangkat keras lain untuk memperoleh data dari lapangan. Data-data tersebut kemudian dikumpulkan menjadi data berformat csv dan diubah menjadi data dengan format JSON sebagai berikut :

Tabel 3.2 Struktur JSON dari Sensor

Struktur JSON:	
<pre>[{ "crop_coefficient": 1.24, "crop_coefficient_stage": "Mid stage", "days_of_planted": 65, "evapotranspiration": 578.6195453, "humidity_[%]": 77.15, "id": 1, "nitrogen_[mg/kg]": 63, "ph": 5.86, "phosphorus_[mg/kg]": 91, "potassium": 112, "reference_evapotranspiration": 185.2125075, "soil_moisture": 651.1, "solar_radiation_ghi": 147.88, "temperature_[_c]": 27.52, "timestamp": "Mon, 23 Jun 2025 18:16:35 GMT", "wind_speed": 2.89 }, r]</pre>	
Parameter	Keterangan
crop_coefficient	Koefisien tanaman (Kc) menggambarkan kebutuhan air relatif tanaman pada fase pertumbuhan tertentu
crop_coefficient_stage	Tahapan pertumbuhan tanaman saat ini
days_of_planted	Hari sejak tanaman ditanam
evapotranspitarion	Jumlah air yang hilang akibat penguapan
humidity	Kelembapan udara
nitrogen	Kandungan nitrogen pada tanah
ph	Tingkat keasaman tanah
phosphorus	Kandungan fosfor pada tanah
potassium	Kandungan kalium di dalam tanah
reference_evaporation	nilai standar berdasarkan tanaman acuan
soil_mosture	kelembapan tanah
solar_radiation_ghi	Radiasi
temperature	Suhu udara saat pengukuran
timestamp	Waktu saat data diambil
windspeed	Kecepatan angin

Tahapan yang dilakukan dalam menampilkan data tersebut menjadi data *real-time* yang siap uji antara lain :

1. Mengunduh sumber data hasil pengukuran dalam bentuk csv.
2. Memanfaatkan *framework* Flask untuk menampilkan data tersebut dan menggunakan *library* pandas untuk mengolah data csv.

3. Melakukan data *preprocessing* dengan mengubah semua huruf menjadi huruf kecil, mengganti spasi dengan *underscore* agar mudah dipahami, serta melakukan teknik *oversampling* pada data agar data bertambah dari jumlah data aslinya. Teknik *random oversampling* adalah teknik untuk menambahkan data dari kelas minoritas secara acak. Penambahan ini dilakukan berulang-ulang hingga jumlah data pada dataset setara dengan jumlah data yang diinginkan [36]. Teknik *random oversampling* diterapkan dengan menggunakan `df.sample()` untuk membuat data menjadi lebih banyak tanpa memperhatikan distribusi label.

```
1
2 csv_file = 'tomato irrigation dataset.csv'
3
4 # Load CSV
5 df = pd.read_csv(csv_file)
6 df.columns = [col.strip().lower().replace(" ", "_") for col in df.columns]
7
8 # Upsample ke 5000 baris
9 desired_size = 20000
10 df = df.sample(n=desired_size, replace=True, random_state=42).reset_index(
    drop=True)
11
12 # Tambahkan ID unik
13 df.insert(0, 'id', range(1, len(df) + 1))
```

Gambar 3.10 Mounting Data

4. Membuat kolom *timestamp* agar dapat menampilkan data secara *real-time* selama pengujian.

```
1
2 csv_file = 'tomato irrigation dataset.csv'
3
4 # Load CSV
5 df = pd.read_csv(csv_file)
6 df.columns = [col.strip().lower().replace(" ", "_") for col in df.columns]
7
8 # Upsample ke 5000 baris
9 desired_size = 20000
10 df = df.sample(n=desired_size, replace=True, random_state=42).reset_index(
    drop=True)
11
12 # Tambahkan ID unik
13 df.insert(0, 'id', range(1, len(df) + 1))
```

Gambar 3.11 Membuat kolom timestamp

5. Membuat *endpoint* untuk mengambil data sensor dan menambahkan data ke dalam json setiap 10 detik. Data akan terus menerus di-*generate* seperti ketika sensor mengambil data *real-time* dari aplikasi.

```

1 @app.route('/sensor', methods=['GET'])
2 def stream_data():
3     elapsed = (datetime.now() - server_start_time).total_seconds()
4     data_count = int(elapsed // 10)
5
6     # Loop data dari awal jika sudah habis
7     full_data = []
8     loop = data_count // len(original_data)
9     remainder = data_count % len(original_data)
10
11     # Tambahkan semua loop penuh
12     for _ in range(loop):
13         full_data.extend(original_data)
14
15     # Tambahkan sisanya
16     full_data.extend(original_data[:remainder])
17

```

Gambar 3.12 Membuat Endpoint API

6. Menjalankan server Flask dalam mode *debug* untuk mempermudah pengembangan karena *auto-reload* dan error ditampilkan.

```

1 if __name__ == '__main__':
2     app.run(debug=True)

```

Gambar 3.13 Menjalankan Server

3.2 Alat

Salah satu tujuan penelitian ini adalah untuk meninjau performa *rendering* dan penggunaan *resource* pada perangkat saat melakukan visualisasi data dari perangkat sensor IoT. Oleh karena itu, penelitian ini membutuhkan peralatan software dan hardware untuk menunjang penelitian.

3.2.1 Perangkat Lunak

Peralatan perangkat lunak yang digunakan dalam pengembangan sistem ini, meliputi sistem operasi, *library*, dan aplikasi yang digunakan, yang dijelaskan sebagai berikut :

Tabel 3.3 Spesifikasi Perangkat Lunak

Nama Perangkat Lunak	Versi	Keterangan
Visual Studio Code	1.97.0	Digunakan untuk membuat <i>website</i> , dari segi <i>Frontend</i> dan <i>Backend</i> .

Nama Perangkat Lunak	Versi	Keterangan
Python	3.11.4	Digunakan untuk membuat data <i>dummy</i> guna menguji cobakan <i>chart</i> visualisasi.
Chart.js	4.0	<i>Versi</i> Chart.js saat diuji cobakan. Digunakan untuk membuat visualisasi.
D3.js	7.9.0	<i>Versi</i> D3.js saat diuji cobakan. Digunakan untuk membuat visualisasi.
Highcharts	12.1.2	<i>Versi</i> Highcharts saat diuji cobakan. Digunakan untuk membuat visualisasi.

3.2.2 Perangkat Keras

Peralatan perangkat keras yang digunakan dalam pengembangan sistem ini, meliputi spesifikasi laptop yang digunakan untuk mengembangkan dan menjalankan *website*, yang dijelaskan sebagai berikut :

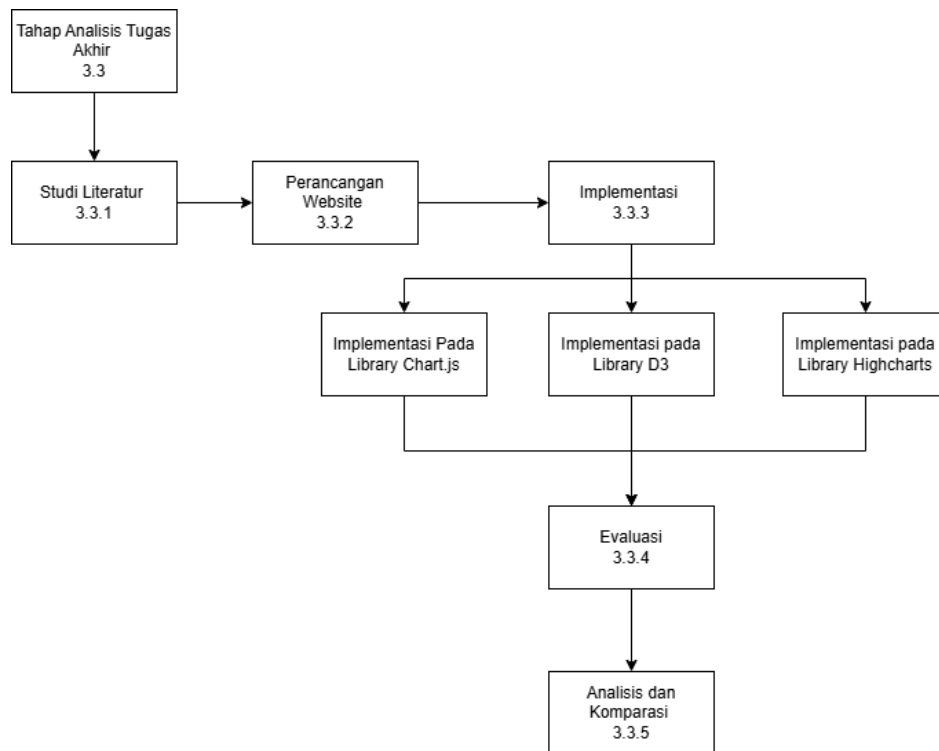
Tabel 3.4 Spesifikasi Perangkat Keras yang Digunakan

Spesifikasi	Keterangan
Operating System	Windows 11 64-bit
Processor	Intel(R) Core (TM) i5-10300H
CPU	2.5 GHz
Memory	8GB DDR4 SO-DIMM 2666MHz
Graphic	NVIDIA® GeForce® GTX 1650 4GB GDDR6

3.3 Tahapan Proyek Akhir

Dalam penelitian yang berjudul "Analisis Perbandingan Kinerja *Library* Chart.js, Highcharts, dan D3.js untuk Visualisasi Data Sensor IoT pada Dashboard Berbasis Laravel", penulis melalui beberapa tahapan penting untuk mendapatkan hasil yang komprehensif. Tahapan pertama adalah fase studi literatur, selanjutnya adalah pengembangan *website*, di mana penulis membangun sebuah dasbor berbasis Laravel yang mampu menampilkan data sensor IoT secara dinamis. Setelah itu, pada fase implementasi *library*, tiga *library* visualisasi Chart.js, Highcharts, dan D3.js

diimplementasikan ke dalam sistem untuk menampilkan data dalam format grafik *line chart*. Tahapan terakhir adalah fase komparasi, yang bertujuan untuk mengevaluasi kinerja masing-masing *library* berdasarkan berbagai parameter yang telah ditentukan sebelumnya. Melalui pendekatan ini, penelitian diharapkan dapat memberikan wawasan mendalam mengenai *library* yang paling optimal untuk memvisualisasikan data IoT. Alur penelitian ini dijelaskan dalam bagan berikut :



Gambar 3.14 Alur Penelitian

3.3.1 Studi Literatur

Tahap ini dilakukan untuk memperoleh landasan yang mendukung pelaksanaan penelitian. Literatur yang dikaji mencakup teori-teori dan penelitian terdahulu yang berkaitan dengan pendekatan dan metode yang digunakan dalam penelitian ini.

Penelitian dilakukan dengan menggunakan metode eksperimen dengan pendekatan kuantitatif. Pendekatan ini dilakukan secara runtut, di mana peneliti mengujicobakan variabel bebas untuk melihat dampaknya terhadap variabel yang dipengaruhi dalam situasi yang telah dikendalikan. Dengan metode ini, peneliti memiliki kontrol atas variabel bebas baik sebelum maupun saat proses eksperimen berlangsung. Proses ini juga disertai dengan pengukuran data secara kuantitatif guna mengetahui adanya hubungan sebab dan akibat antara variabel yang diteliti. Tujuan dari pendekatan ini adalah untuk menguji kebenaran hipotesis yang telah

dirumuskan, memperkirakan hasil dari perlakuan yang diberikan, serta menyimpulkan pola hubungan antar variabel secara umum [37]. Dalam konteks ini, variabel bebas yang dimaksud adalah Chart.js, D3, dan Highcharts, serta variabel terikat adalah waktu *render*, penggunaan CPU, dan penggunaan memori.

Terdapat beberapa studi literatur yang digunakan sebagai acuan pengukuran dalam penelitian ini. Berdasarkan rujukan jurnal yang dijelaskan pada bab sebelumnya, waktu *rendering*, penggunaan memori, dan alokasi CPU seringkali dijadikan sebagai indikator pengukuran performa suatu *website* visualisasi karena ketiganya memengaruhi alokasi sumber daya (*resources*) pada sistem komputer yang menentukan pengalaman pengguna (*user experience*) dan efisiensi sistem [4] [3] [35].

Fitur-fitur utama dari visualisasi data seperti interaktivitas, kompleksitas elemen grafik, jumlah data yang divisualisasikan, serta kemampuan dukungan *real-time* sangat memengaruhi performa *render*. Semakin tinggi tingkat interaktivitas serta semakin kompleks grafik, maka semakin besar pula beban terhadap CPU dan memori sistem, serta semakin lama waktu yang dibutuhkan untuk menampilkan visualisasi secara utuh. Merujuk pada *website* dari Chart.js [5], D3.js [6], dan Highcharts [23], perbandingan yang ditinjau dari aspek yang telah disebutkan dijelaskan seperti pada tabel berikut :

Tabel 3.5 Perbandingan Chart.js, D3.js, dan Highcharts

Aspek	Chart.js	D3.js	Highcharts
Ketersediaan <i>open source</i>	Tersedia	Tersedia	Tersedia
Dukungan Interaktif	Mendukung interaktivitas seperti <i>tooltip</i> , <i>zoom</i> , dan <i>hover</i> .	Interaksi kompleks termasuk <i>tooltip</i> , <i>zoom</i> , dan animasi.	Interaktif dengan built-in fitur <i>tooltip</i> , <i>drilldown</i> , <i>zoom</i> , <i>pan</i> , dan animasi.
Basis grafik	Menggunakan Canvas	Menggunakan SVG atau Canvas	Menggunakan SVG (default)
Dukungan data <i>real-time</i>	Didukung dengan update dinamis menggunakan <code>chart.update()</code>	Harus dikodekan manual	Mendukung data <i>real-time</i> dengan fitur bawaan <code>setData()</code>

3.3.2 Perancangan *Website*

Dalam prosesnya, pengembangan *website* ini dibagi menjadi dua bagian utama, yaitu *frontend* dan *backend*. *Frontend* merupakan sisi pengembangan yang berfokus pada tampilan dan interaksi pengguna. Sementara *backend* menangani manajemen *database* dan API. Penelitian ini secara khusus berfokus pada pengembangan *Frontend* yang mencakup implementasi desain serta analisis penggunaan *Library* visualisasi. Proses pengembangan *frontend* melibatkan berbagai teknologi seperti HTML, CSS, dan JavaScript. Dengan fokus pada *Frontend*, penelitian ini bertujuan untuk menghasilkan tampilan *website* yang responsif dan mudah digunakan oleh pengguna. Adapun aspek *Backend* yang berhubungan dengan manajemen data dan logika pemrosesan tidak dibahas dalam penelitian ini, sehingga integrasi dengan sistem *Backend* akan menjadi pertimbangan dalam implementasi lanjutan.

A. Perancangan Desain Sistem

Selain data uji, diperlukan pula perancangan sistem yang menjadi bahan untuk mendefinisikan struktur, alur kerja, dan interaksi sistem sebelum tahap implementasi dilakukan. Pada tahap ini, diperlukan pendefinisian kebutuhan, *use case diagram* dan *sequence diagram*, serta kebutuhan data pengujian. Kebutuhan fungsional yang dimaksud mencakup fitur-fitur utama yang harus ada dalam sistem agar sesuai dengan tujuan. Berikut adalah beberapa kebutuhan fungsional dalam *website* yang menampilkan data IoT:

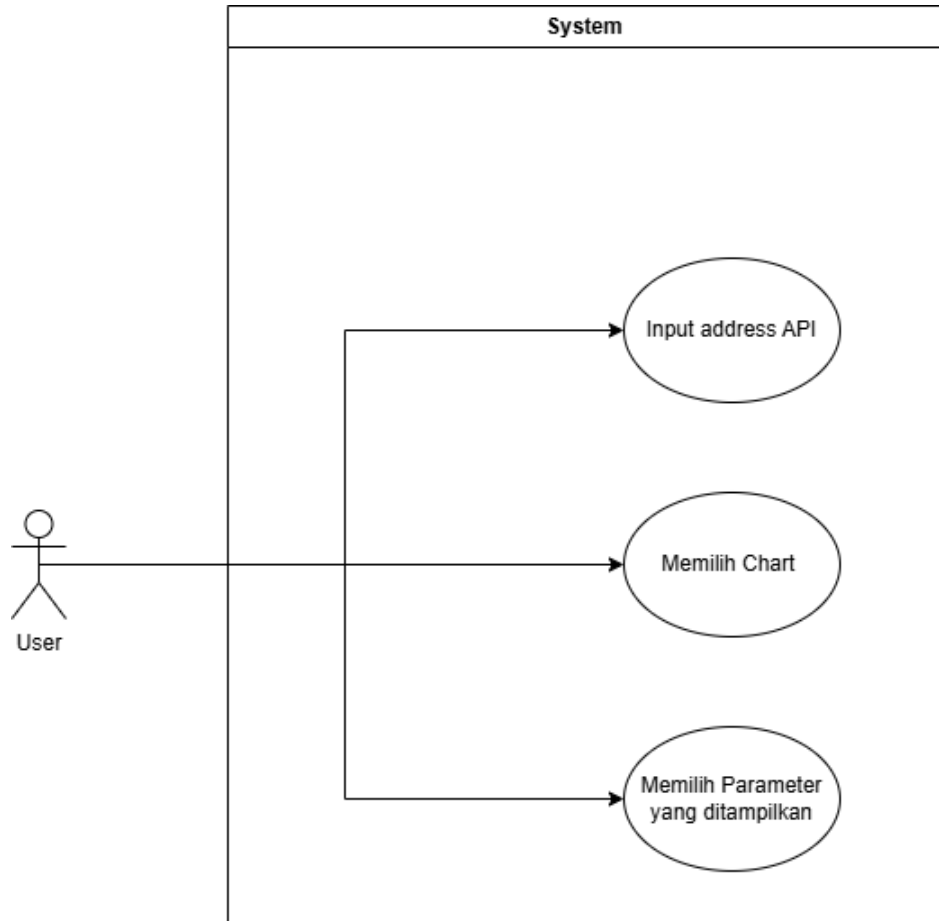
1. *Website* harus mampu mengambil data secara *real-time* melalui API.
2. *Website* mampu menambahkan lebih dari satu visualisasi dengan dukungan terhadap jenis grafik seperti *line chart*, *bar chart*, dan *scatter plot*.

Sedangkan kebutuhan non-fungsional yang dimaksud, berkaitan dengan kualitas sistem dan aspek teknis yang memastikan *website* berjalan dengan optimal. Kebutuhan tersebut meliputi:

1. Sistem mampu menangani volume data besar yang terus bertambah dari perangkat IoT.
2. Kode terdokumentasi dengan baik, sehingga mudah diperbarui atau diperbaiki.

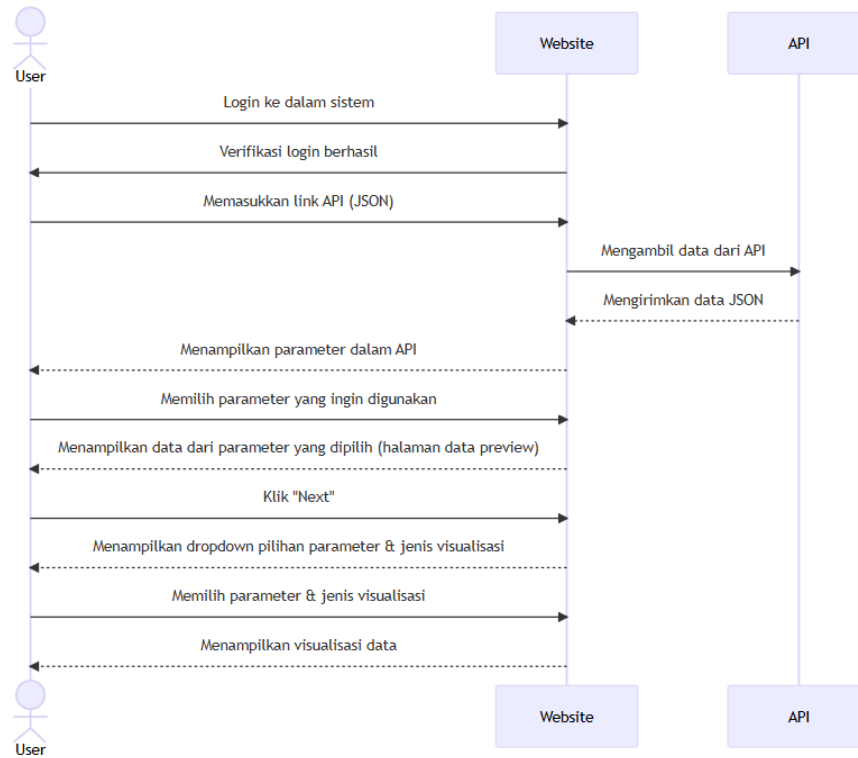
Untuk mengimplementasi kebutuhan fungsional dan non fungsional, diagram yang dibutuhkan antara lain adalah *use case diagram* dan *sequence diagram*. *Use Case Diagram* adalah diagram *Unified Modeling Language* (UML) yang menggambarkan interaksi antara pengguna (*actors*) dengan sistem melalui berbagai fitur penggunaan (*use cases*). *Use case diagram* dapat membantu pengembang akan

lebih mudah dalam melakukan analisis kebutuhan, serta merancang fitur yang akan dikembangkan [31]. Pada sistem yang sedang diteliti, *use case diagram* berikut menggambarkan fitur-fitur yang berkaitan dengan penelitian, seperti gambar berikut :



Gambar 3.15 use Case Diagram

Sequence Diagram merupakan diagram UML yang digunakan untuk menggambarkan bagaimana objek berinteraksi dan bertukar pesan seiring waktu. Diagram ini menunjukkan bagaimana pesan dikirim antara objek atau instance lainnya untuk menyelesaikan suatu tugas. *Sequence Diagram* digunakan pada tahap perancangan detail, dimana komunikasi antar proses harus ditetapkan secara tepat. *Sequence Diagram* untuk *website* ini digambarkan seperti berikut :



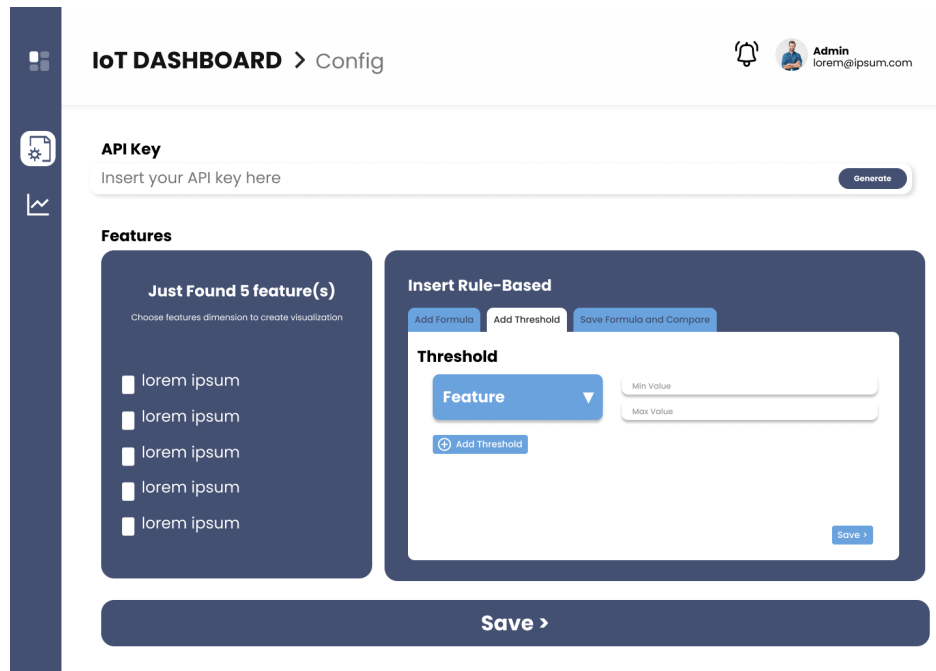
Gambar 3.16 *Sequence Diagram*

B. Perancangan UI

Sebelum tahap pengembangan *frontend*, diperlukan perancangan *User Interface* dengan Figma yang akan digunakan sebagai acuan untuk membuat kode *frontend*, rancangan *user interface* tersebut merupakan rancangan desain *high-fidelity* yang menggambarkan secara langsung bentuk dari *website* dari segala aspek, seperti warna, penataan, dan semua aspek visual yang ada di dalamnya. Desain UI dibagi menjadi dua halaman utama dan tiga komponen tambahan, seperti berikut :

1. Halaman Utama

Halaman ini merupakan halaman pertama yang akan dilihat pengguna setelah mengakses *website*. Pada bagian ini, pengguna dapat menginputkan API dari data IoT pada *field* input yang tersedia, kemudian setelah di klik *generate*, parameter pada API tersebut akan tertampil dalam bentuk *checkbox*.



Gambar 3.17 Halaman Utama

Adapun fitur yang ada di dalam halaman ini adalah kolom input alamat API untuk menginputkan alamat API dari pengguna, *Checkbox* untuk menampilkan parameter yang ada di dalam API, *Tab rules* untuk memberikan pengaturan terhadap data-data yang masuk. Seperti mengatur ambang batas, formula, dan komparasi formula.

2. Formula

Merupakan fitur yang digunakan untuk menyimpan rumus tertentu yang dapat difungsikan sebagai *threshold* atau *alert* saat data menyentuh suatu titik tertentu yang dianggap tidak wajar.

Insert Rule-Based

Add Formula Add Threshold Save Formula and Compare

Formula

Feature 1 * (feature 2 * (2.1 + 3)) - 8 / feature 3

- Feature 1
- Feature 2
- Feature 3
- Feature 4
- Feature 5
- Feature 6

Save >

Gambar 3.18 Menambahkan Formula

3. *Threshold*

Merupakan fitur yang digunakan untuk menyimpan ambang batas minimal dan maksimal dari suatu fitur data sehingga sistem akan memunculkan peringatan ketika data melampaui ambang batas.

Insert Rule-Based

Add Formula Add Threshold Save Formula and Compare

Threshold

Feature ▼

Min Value

Max Value

+ Add Threshold

Save >

Gambar 3.19 Menambahkan Ambang Batas

4. *Save Formula and Compare*

Digunakan untuk membandingkan antara *threshold* dan batasan tertentu. Sehingga ketika terdapat perbedaan, maka sistem akan menampilkan *alert*.

Pada tab ini, pengguna dapat memilih dua fitur untuk dibandingkan.

Insert Rule-Based

Add Formula Add Threshold **Save Formula and Compare**

Save Formula and Compare

Features 1 < Insert indicator Feature 1

Features 1 < Insert indicator Feature 1

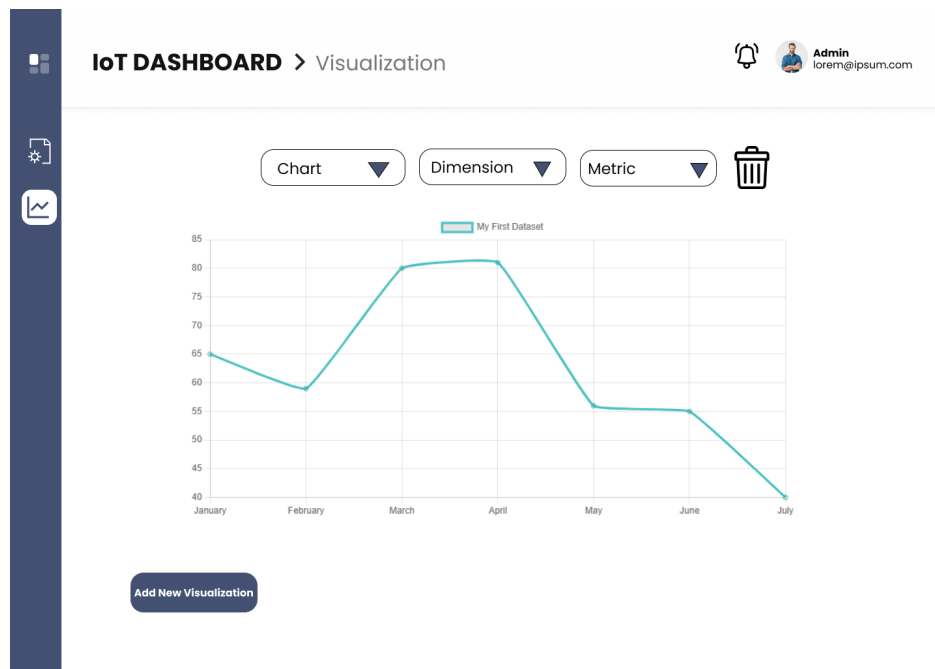
+ Add More

Save >

Gambar 3.20 Menambahkan Ambang Batas

5. Halaman visualisasi dan *Chart*

Halaman ini digunakan untuk memvisualisasikan perbandingan dari suatu fitur dengan fitur lainnya melalui *bar chart*, *line chart*, dan *scatter plot*.



Gambar 3.21 Menambahkan Ambang Batas

3.3.3 Implementasi

Setelah menyiapkan data, maka tahap selanjutnya adalah tahap implementasi. Tahap ini akan menjelaskan implementasi Chart.js, D3.js, dan Highcharts dalam menampilkan visualisasi. Implementasi dilakukan dengan menggunakan skrip Javascript yang sama dengan penyesuaian pada bagian fungsi `createChart()` sesuai dengan *library* JavaScript yang digunakan. Fungsi-fungsi lain yang digunakan dalam tahap ini telah dijelaskan pada sub-bab “Pembuatan Website”.

3.3.4 Evaluasi

Proses ini bertujuan untuk mengetahui perbedaan signifikan dalam waktu *rendering*, efisiensi pengolahan data visual, serta kemampuan masing-masing *library* dalam menangani beban visualisasi yang berbeda. Dalam prosesnya, semua pengujian dilakukan dalam lingkungan *hardware*, *software*, dan versi *browser* yang sama serta kondisi *browser* selalu dipastikan bersih sebelum melakukan pengujian lainnya dengan selalu memulai ulang kondisi *browser*. Dengan melakukan perbandingan ini, diharapkan dapat ditemukan *library* yang paling optimal dan sesuai untuk digunakan dalam pengembangan sistem visualisasi data. Setelah diimplementasikan, performa akan dievaluasi berdasarkan parameter-parameter yang telah ditentukan, meliputi :

1. Performa *rendering*

Performa akan diukur dengan fungsi dari Javascript [4] [3]. Pengukuran dilakukan dengan memanfaatkan fungsi dari JavaScript `console.time()` dan `performance.now()` yang ditempatkan di dalam fungsi `createChart()` untuk mengukur waktu *render*. Fungsi `performance.now()` adalah fungsi di JavaScript yang digunakan untuk mengukur waktu secara presisi sehingga dapat digunakan untuk mengetahui durasi eksekusi suatu proses dalam aplikasi web [38]. Pengukuran ini dilakukan dengan tahapan :

1. menjalankan server data *dummy* dan server *website* secara lokal,
2. menampilkan visualisasi pada *website*,
3. meninjau waktu *render* melalui file berformat `.txt` yang telah otomatis di-*generate* dari *console* menggunakan fungsi `performance.now()` yang memberikan nilai waktu relatif, yaitu berapa milidetik yang dihitung sejak fungsi dijalankan.

```
console.time(`Render Time Chart ${chartId}`);  
let startTime = performance.now();
```

Gambar 3.22 Mulai Pengukuran

```
console.time(`Render Time Chart ${chartId}`);  
let startTime = performance.now();
```

Gambar 3.23 Akhir Pengukuran

Setelah seluruh proses selesai dijalankan dan waktu *render* telah terekam, data tersebut akan digunakan untuk menghitung rata-rata waktu *render* di setiap titik yang telah ditentukan. Nilai rata-rata ini berperan penting dalam proses evaluasi, karena mampu mereduksi pengaruh *noise* atau *outlier* yang mungkin muncul selama pengujian. Dengan demikian, pengukuran performa *rendering website* menjadi lebih stabil, representatif, dan dapat mencerminkan tingkat skalabilitas sistem secara lebih akurat.

2. Alokasi *Footprint Memory*

Pengukuran penggunaan memori dilakukan dengan memanfaatkan fitur Chrome Task Manager, dengan cara mencatat nilai *Memory Footprint* pada tab Chrome tertentu. Nilai ini menggambarkan total memori yang digunakan oleh tab tersebut, mencakup memori yang dipakai untuk proses *rendering* halaman, eksekusi skrip JavaScript, serta pemuatan berbagai sumber daya yang ada di dalam halaman web. Pendekatan ini membantu memperoleh perkiraan konsumsi memori yang lebih terfokus pada aktivitas halaman yang diuji.

Task	Memory footprint	CPU	CPU Time	Network	Process ID
Se					
Sl	23,604K	0.0	0h 0m 1s	0	8492
Sl	23,672K	0.0	0h 0m 1s	0	29400
Ta	97,340K	1.2	0h 0m 34s	0	21432
Se	77,992K	0.0	0h 0m 12s	0	31148

End process

Gambar 3.24 Pengukuran Footprint Memory

3. Penggunaan CPU

Penggunaan CPU diukur dengan *library* psutil pada Python yang memungkinkan pengukuran pada penggunaan CPU dari task manager. Pengukuran ini merujuk pada penelitian yang dilakukan oleh Vayadande, dkk dalam karyanya yang berjudul “*Efficient System for CPU Metric Visualization*” [39]. Dalam penelitiannya, pengukuran performa CPU dilakukan secara menyeluruh terhadap seluruh sistem melalui *Task Manager*. Namun, dalam studi ini, pengukuran akan difokuskan secara lebih spesifik pada penggunaan CPU oleh proses tertentu, dengan merujuk pada PID (Process ID) dari tab atau aplikasi yang dimaksud. Pendekatan ini memungkinkan analisis performa yang lebih terarah dan akurat terhadap aktivitas CPU pada proses yang relevan.

4. Skalabilitas

Skalabilitas akan mengukur perubahan yang terjadi terhadap performa *rendering*, penggunaan memori, dan penggunaan CPU. Hasilnya, skalabilitas meninjau perubahan tersebut dan memvisualisasikan ke dalam bentuk diagram agar mudah dibandingkan. Pengukuran ini mengacu pada penelitian yang dilakukan oleh Persson dalam judul “*Scalability of JavaScript Libraries for Data Visualization*” [4]. Untuk meninjau hasil evaluasi performa dan skalabilitas sistem, pengambilan data hasil ukur akan dilakukan pada setiap kelipatan 100. Dalam konteks ini, ukuran data yang diuji dimulai dari 100, 200, 300, 400, 500, dan seterusnya, disesuaikan dengan kapasitas dan batas kemampuan alat uji. Pendekatan bertahap ini bertujuan untuk mengamati

sejauh mana sistem mampu mempertahankan performanya seiring dengan meningkatnya beban kerja secara sistematis.

3.3.5 Analisis dan Komparasi

Tahap analisis dan komparasi merupakan langkah di mana penulis akan mengevaluasi serta membandingkan kinerja dari berbagai *library* yang telah diimplementasikan dalam penelitian. Pada tahap ini, perbandingan akan dilakukan secara sistematis dengan memanfaatkan visualisasi grafik yang mencakup berbagai parameter sebagai dasar evaluasi. Parameter-parameter tersebut akan digunakan untuk mengukur sejauh mana masing-masing *library* dapat memenuhi kriteria yang telah ditetapkan, sehingga dapat diperoleh gambaran yang jelas mengenai keunggulan dan kelemahan dari setiap *library* yang diuji dengan parameter rata-rata waktu *render* di setiap titik tertentu, rata-rata alokasi memori *footprint* di titik tertentu, dan rata-rata penggunaan cpu di titik tertentu.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Perancangan *Frontend*

Pada tahap perancangan *frontend*, penulis menggunakan HTML, CSS, dan Javascript yang diimplementasikan pada *framework* Laravel. Dari rancangan desain yang telah dibuat, terdapat dua halaman utama dan beberapa *function* yang mengeksekusi program, seperti fetch API, menampilkan *dropdown* yang berisi parameter dari API yang di-*generate*, membuat visualisasi dengan *library*, dan lainnya.

4.1.1 Halaman Utama

Pada halaman ini, terdapat beberapa fitur dan fungsi Javascript untuk menjalankan fungsionalitas *website*. Potongan kode berikut berfungsi untuk men-*generate keys* yang ada di dalam API yang diinputkan dari tampilan yang dibuat menggunakan HTML.

```
1 <div class="input-wrapper">
2   <form action="" id="sensor-form" method="POST">
3     @csrf
4     <input id="api-input" class="input-text p-3" type="text" placeholder="Input Your API here">
5     <button type="submit" id="submit-api" onclick=" generateAPI()" class="submit"> Generate </button>
6   </form>
7 </div>
```

Gambar 4.1 Generate API

Fungsi yang ada pada potongan kode diatas adalah fungsi `generateAPI()`. Kode Javascript yang disusun untuk fungsi tersebut adalah seperti berikut :

```

1 function generateAPI() {
2   // Ambil URL dari input form
3   const apiUrl = document.getElementById("api-input").value;
4
5   if (!apiUrl) {
6     alert("Please input a valid API URL");
7     return;
8   }
9
10  // Simpan API URL ke localStorage
11  localStorage.setItem("apiUrl", apiUrl);
12
13  fetch(apiUrl)
14    .then(response => {
15      if (!response.ok) {
16        throw new Error("Network response was not ok");
17      }
18      return response.json();
19    })
20    .then(data => {
21      const keysDiv = document.getElementById("api-keys");
22      keysDiv.innerHTML = ""; // Hapus checkbox sebelumnya
23
24      let keys = [];
25
26      if (Array.isArray(data)) {
27        keys = Object.keys(data[0]);
28      } else if (typeof data === "object") {
29        const firstItem = Object.values(data)[0];
30        if (typeof firstItem === "object") {
31          keys = Object.keys(firstItem);
32        }
33      }
34
35      keys.forEach(key => {
36        const checkboxWrapper = document.createElement("div");
37        checkboxWrapper.className = "form-check";
38
39        const checkbox = document.createElement("input");
40        checkbox.type = "checkbox";
41        checkbox.className = "form-check-input";
42        checkbox.id = `checkbox-${key}`;
43        checkbox.value = key;
44
45        const label = document.createElement("label");
46        label.className = "form-check-label";
47        label.htmlFor = `checkbox-${key}`;
48        label.textContent = key;
49
50        checkboxWrapper.appendChild(checkbox);
51        checkboxWrapper.appendChild(label);
52        keysDiv.appendChild(checkboxWrapper);
53      });
54    })
55    .catch(error => {
56      console.error("Error fetching API:", error);
57      alert("Failed to fetch the API. Please check the URL.");
58    });
59 }

```

Gambar 4.2 Fungsi Generate API

Kode JavaScript di atas dibuat untuk mengambil data dari API yang dimasukkan oleh pengguna melalui kolom input, lalu menampilkan daftar *checkbox* berdasarkan kunci-kunci (*keys*) dari data yang diperoleh. Jika tidak ada input API pada kolom dengan id = "api-input", sistem akan menampilkan peringatan agar pengguna memasukkan URL yang valid.

Berikutnya, API akan diambil dengan menggunakan fungsi `fetch()`. *Fetch* sendiri merupakan sebuah fungsi bawaan Javascript yang digunakan mengambil data dari sumber eksternal, seperti API, file JSON, atau file lain yang tersedia di server. Jika respon API gagal, maka sistem akan menampilkan error dan menghentikan proses lebih lanjut. Namun, jika data berhasil diperoleh, sistem akan mengolahnya untuk mendapatkan daftar *keys*. Setelah daftar *keys* diperoleh, untuk setiap *key*, sistem akan menambahkan elemen input bertipe *checkbox* kemudian tiap *checkbox* diberi atribut *id* dan *value* sesuai dengan *key* yang bersangkutan sehingga data tersimpan sebagai pasangan *key-value*. Setelah itu, elemen-elemen ini akan ditempatkan dalam *div* utama yang memiliki *id*="api-keys", sehingga *checkbox* dapat ditampilkan di halaman *main page*. Jika terdapat kesalahan saat mengambil data dari API, error akan dicatat dan ditampilkan di *console*, dan sistem akan menampilkan peringatan bahwa pengambilan data gagal melalui *alert*. Dengan fitur ini, pengguna dapat melihat *keys* pada API secara langsung dan dapat memilih *key* yang dibutuhkan melalui *checkbox* yang disediakan.

```
1 <button id="addfeatures" class="position-absolute bottom-0 end-0 btn btn-primary m-3 px-2 py-2"
  style="width:95%; background-color:white; color:var(--blueiot); border:transparent">Use Features</button>
2
```

Gambar 4.3 Generate API

Setelah pengguna memilih *keys* dari *checkbox* dan menekan tombol “*use features*” dengan *id*="addfeatures", data akan disimpan di *localStorage* untuk ditampilkan pada halaman visualisasi.

4.1.2 Halaman Visualisasi

Pada halaman visualisasi, kode dari fitur dan fungsi digunakan untuk menampilkan dan menambahkan visualisasi secara dinamis dalam sebuah kontainer.

a. Fungsi *Log Render*

Kode berikut merupakan fungsi untuk mencatat dan menyimpan log waktu *rendering* setiap grafik yang dihasilkan oleh *library* visualisasi data. Fungsi ini memastikan bahwa setiap proses pembuatan dan pembaruan grafik terdokumentasi dengan baik, sehingga dapat digunakan untuk analisis performa dan evaluasi efisiensi *rendering* secara berkala.


```

1 //save file
2 let fileHandle; // Handle untuk file log
3 const logTimes = []; // Array untuk menyimpan log waktu render
4
5 // Fungsi untuk meminta izin menyimpan file
6 async function initFile() {
7     fileHandle = await window.showSaveFilePicker({
8         suggestedName: "render_times.txt",
9         types: [{ accept: { "text/plain": [".txt"] } }]
10    });
11 }
12
13 // Fungsi untuk menulis log ke file
14 async function saveLogToFile() {
15     if (!fileHandle) {
16         await initFile();
17     }
18
19     const logContent = logTimes.join("\n");
20     const writable = await fileHandle.createWritable();
21     await writable.write(logContent);
22     await writable.close();
23 }

```

Gambar 4.4 Fungsi Menyimpan Log Render

b. Fungsi Fetch Data()

Kode JavaScript pada gambar berikut berfungsi untuk mengambil data sensor dari API yang disimpan secara lokal secara berkala dan memperbarui tampilan grafik dengan data terbaru.

```

1 async function fetchSensorData() {
2     try {
3         // Ambil API dari localStorage
4         const Chartendpoint = localStorage.getItem("Chartendpoint");
5
6         if (!Chartendpoint) {
7             console.error("API endpoint belum diset!");
8             return;
9         }
10
11         const response = await fetch(Chartendpoint);
12         if (!response.ok) throw new Error("Gagal mengambil data!");
13
14         const data = await response.json();
15         jsonData = Object.values(data); // Ubah objek menjadi array
16         console.log("Data terbaru:", jsonData);
17
18         updateAllCharts(); // Update semua grafik setelah data baru masuk
19     } catch (error) {
20         console.error("Gagal mengambil data dari API:", error);
21     }
22 }
23
24 // Jalankan fetch setiap 10 detik
25 setInterval(fetchSensorData, 10000);
26 fetchSensorData();

```

Gambar 4.5 Fungsi Fetch Data

Data akan diperbarui dengan menggunakan fungsi

`setInterval(fetchSensorData, 10000)` yang akan memanggil `fetchSensorData()` setiap 10 detik. Pada konteks ini, `fetchSensorData()` bersifat asinkron untuk memastikan proses pengambilan data tidak menghambat eksekusi kode lainnya. Jika respons dari API tidak berhasil, fungsi akan menampilkan error. Jika data berhasil diambil, tampilan grafik akan diperbarui dengan pemanggilan fungsi `updateAllCharts()`. Fungsi ini akan memanggil fungsi `UpdateChart()` untuk memperbarui visualisasi pada id chart tertentu.

```
1 function updateAllCharts() {  
2   for (let i = 1; i <= chartCount; i++) {  
3     updateChart(i);  
4   }  
5 }
```

Gambar 4.6 Memperbarui Data pada Chart

c. fungsi `AddNewVisualization()`

Ketika tombol add new visualization pada halaman ditekan, fungsi `addNewVisualization()` akan menambahkan elemen baru ke dalam div dengan id='visualizations' berupa *template chart*.

```
1 <div id="" class="chartContainer container-fluid">  
2  
3 <div class="container-fluid overflow-auto my-4 mx-0">  
4  
5   <div id="visualizations" class="container-fluid overflow-auto">  
6     <!-- All visualizations will be dynamically added here -->  
7   </div>  
8  
9 <div class="controls">  
10   <button id="addChart" onclick="addNewVisualization()" class="btnaddChart m-5 mt-2 px-3">  
11     Add More visualization  
12   </button>  
13 </div>
```

Gambar 4.7 Potongan Kode Pada Button New Visualization

Tombol dengan id=addChart pada event diatas, akan mengeksekusi fungsi `addNewVisualization()` berikut :

```

1 function addNewVisualization() {
2   chartCount++;
3
4   const container = document.getElementById('visualizations');
5   const newChartDiv = document.createElement('div');
6   newChartDiv.classList.add('container-fluid', 'overflow-y-auto', 'm-3');
7   newChartDiv.id = `chart${chartCount}-container`;
8   newChartDiv.innerHTML = `
9     <div class="col-auto justify-content-center container-fluid overflow-auto">
10       <div class="flex flex-row justify-content-center">
11         <select id="chartType${chartCount}" class="styled-dropdown">
12           <option value="bar">Bar Chart</option>
13           <option value="line">Line Chart</option>
14           <option value="scatter">Scatter Plot</option>
15         </select>
16         <select id="xFeature${chartCount}" class="styled-dropdown"></select>
17         <select id="yFeature${chartCount}" class="styled-dropdown"></select>
18         <button id="deleteChart${chartCount}"
19           class="delete-btn"></button>
21       </div>
22       <div class="chart-container container-fluid overflow-auto whitespace-nowrap">
23         <div class="chart chartconfig" style="width:100%" id="chart${chartCount}"></div>
24       </div>
25     </div>
26
27     container.appendChild(newChartDiv);
28
29     // Isi dropdown dengan data jsonData
30     populateDropdowns(chartCount);
31
32     // Tambahkan event listener untuk elemen baru
33     document.getElementById(`chartType${chartCount}`).addEventListener('change', () => updateChart(
34       chartCount));
35     document.getElementById(`xFeature${chartCount}`).addEventListener('change', () => updateChart(
36       chartCount));
37     document.getElementById(`yFeature${chartCount}`).addEventListener('change', () => updateChart(
38       chartCount));
39     document.getElementById(`deleteChart${chartCount}`).addEventListener('click', () => deleteChart(
40       chartCount));
41   `;
42 }

```

Gambar 4.8 Fungsi untuk Membuat Visualisasi Baru

Setiap kali fungsi `addNewVisualization()` dipanggil, variabel `chartCount` akan menambahkan elemen baru pada wadah `div` dengan `id= \visualizations`". Di dalam elemen `div` yang baru dibuat terdapat dua *dropdown* yang memungkinkan pengguna memilih tipe grafik (*bar*, *line*, atau *scatter plot*) dan fitur yang akan divisualisasikan. Setiap *dropdown* memiliki `id` yang mengandung nilai `chartCount` agar tidak mengubah visualisasi yang lain ketika terdapat perubahan. Selain itu, terdapat tombol `delete` yang memungkinkan pengguna menghapus visualisasi tertentu dengan mengklik ikon `trash`.

Setelah struktur HTML untuk visualisasi baru selesai dibuat, elemen tersebut dimasukkan ke dalam container utama menggunakan `appendChild()`. Fungsi `populateDropdowns(chartCount)` kemudian dipanggil untuk mengisi *dropdown* dengan data yang sesuai.

d. Fungsi `PopulateDropdown()`

Untuk menampilkan *dropdown* yang datanya diambil dari data-data API,

diperlukan fungsi `populateDropdowns(chartId)` seperti pada gambar berikut :

```
1 function populateDropdowns(chartId) {
2   // Ambil data fitur yang dipilih dari localStorage
3   const selectedFeatures = JSON.parse(localStorage.getItem("selectedFeatures")) || [];
4
5   if (selectedFeatures.length === 0) {
6     console.error("Tidak ada fitur yang dipilih!");
7     return;
8   }
9
10  // Ambil elemen dropdown
11  const xFeatureDropdown = document.getElementById(`xFeature${chartId}`);
12  const yFeatureDropdown = document.getElementById(`yFeature${chartId}`);
13
14  if (!xFeatureDropdown || !yFeatureDropdown) {
15    console.error("Dropdown tidak ditemukan!");
16    return;
17  }
18
19  // Bersihkan dropdown sebelum diisi ulang
20  xFeatureDropdown.innerHTML = '';
21  yFeatureDropdown.innerHTML = '';
22
23  // Tambahkan opsi ke dropdown
24  selectedFeatures.forEach((feature) => {
25    const optionX = document.createElement("option");
26    optionX.value = feature;
27    optionX.textContent = feature;
28    xFeatureDropdown.appendChild(optionX);
29
30    const optionY = document.createElement("option");
31    optionY.value = feature;
32    optionY.textContent = feature;
33    yFeatureDropdown.appendChild(optionY);
34  });
35
36  console.log("Dropdown berhasil diisi dengan fitur:", selectedFeatures);
37 }
```

Gambar 4.9 Fungsi untuk Populate Dropdown

fungsi ini mengambil data fitur yang telah dipilih sebelumnya dari `localStorage.getItem("selectedFeatures")`. Jika tidak ada data yang tersimpan, maka variabel `selectedFeatures` akan berisi *array* kosong dan sistem akan menunjukkan eror pada *console*. Kemudian, fungsi ini akan mencocokkan elemen *dropdown* dengan *chartid*. Jika salah satu elemen tidak ditemukan, akan muncul pesan eror di *console*, dan fungsi akan berhenti. Jika elemen *dropdown* sesuai dengan *chartid* dan `selectedFeatures` tidak kosong, selanjutnya fungsi akan menambahkan *keys* ke dalam *dropdown* berdasarkan daftar `selectedFeatures`.

e. Fungsi `CreateChart()`

Fungsi `createChart()` merupakan fungsi yang digunakan untuk menambahkan kode visualisasi sesuai dengan *library* yang akan diujicobakan.

f. fungsi `UpdateChart()` Merupakan fungsi yang digunakan untuk memperbarui grafik secara dinamis berdasarkan data JSON yang ada atau saat fitur pada *dropdown* diubah. Fungsi ini juga disesuaikan dengan *library* yang

diuji cobakan. fungsi ini mengambil input dari elemen HTML untuk menentukan jenis grafik (`chartType`), fitur yang akan digunakan sebagai sumbu x (`xFeature`), dan fitur untuk sumbu y (`yFeature`) dari *array* JSON. Data untuk sumbu x disimpan dalam variabel `labels`, sedangkan data untuk sumbu y disimpan dalam `chartData`. Keduanya diperoleh menggunakan `map()`, yang mengambil nilai dari properti yang sesuai dalam objek JSON berdasarkan fitur yang dipilih. Setelah data diperoleh, fungsi `createChart(chartId, chartType, labels, chartData)` akan dipanggil untuk membuat atau memperbarui grafik dengan data terbaru.

g. Fungsi `DeleteChart()`

Fungsi ini akan memeriksa grafik yang akan dihapus berdasarkan id-nya. Kemudian, grafik akan dihapus dengan fungsi `destroy()`. Referensi terhadap grafik tersebut juga akan dihapus dari objek `chartInstances` menggunakan `delete chartInstances[chartId]`.

```
1 function deleteChart(chartId) {  
2   if (chartInstances[chartId]) {  
3     chartInstances[chartId].destroy();  
4     delete chartInstances[chartId];  
5   }  
6  
7   const chartContainer = document.getElementById(`chart${chartId}-container`);  
8   if (chartContainer) {  
9     chartContainer.remove();  
10  }  
11 }  
12
```

Gambar 4.10 Fungsi Delete Chart

4.2 Implementasi *Library*

Implementasi *library* dilakukan dengan menerapkan fungsi `createChart()` dan `updateChart()` pada kode awal sebagai bagian dari proses visualisasi data. Fungsi `createChart()` berperan untuk inisialisasi awal grafik, termasuk pengaturan konfigurasi, jenis grafik, skala sumbu, dan data awal yang akan ditampilkan. Sementara itu, fungsi `updateChart()` digunakan untuk memperbarui elemen-elemen grafik secara dinamis ketika terjadi perubahan data, baik karena pembaruan berkala maupun interaksi pengguna.

4.2.1 Implementasi Pada `Chart.js`

Tahap awal implementasi pada `Chart.js` dilakukan dengan pembuatan struktur `Chart.js` pada fungsi `createChart()`. Fungsi utama dalam kode ini adalah `createChart(chartId, chartType, labels, data)`, yang menerima empat parameter, yaitu `chartId` untuk memberi id grafik yang akan dibuat, `chartType` untuk mendefinisikan jenis chart yang akan dibuat, `labels` untuk

menandakan sumbu x, dan variabel data yang akan ditampilkan di dalam grafik sumbu y, Konfigurasi grafik mencakup tampilan responsif (`responsive: true`) agar grafik dapat menyesuaikan ukuran layar, skala sumbu x dan y (`scales`) yang mengatur tampilan label, rotasi teks, serta padding, layout (`layout`) untuk mengatur jarak padding grafik terhadap batas kanvas, dan Fitur tooltip untuk menampilkan informasi saat pengguna mengarahkan kursor ke grafik.

1. Inisialisasi Variabel.

```
1 async function createChart(chartId, chartType, labels, data) {
2   const canvas = document.getElementById(`chart${chartId}`);
3   if (!canvas) {
4     console.error(`Canvas untuk Chart ${chartId}
    tidak ditemukan di DOM!`);
5     return;
6   }
7
8   const ctx = canvas.getContext('2d');
9   console.time(`Render Time Chart ${chartId}`);
10  let startTime = performance.now();
11
12  let finalType;
13  let datasetData;
```

Gambar 4.11 Inisialisasi Variabel dan Pengukuran Performa

Pada bagian ini, canvas chart akan diinisialisasi dan dibuat agar dapat digunakan untuk menampilkan chart.

2. Menentukan Tipe *Chart*

```
1 if (chartType === 'scatter') {
2   finalType = 'scatter';
3   datasetData = labels.map((x, i) => ({
4     x: parseFloat(x),
5     y: parseFloat(data[i])
6   }));
7 } else if (chartType === 'bar') {
8   finalType = 'bar'; // Chart.js pakai 'bar' untuk vertikal
9   datasetData = data;
10 } else {
11   finalType = 'line';
12   datasetData = data;
13 }
```

Gambar 4.12 Potongan Kode untuk Menentukan Jenis Visualisasi

Potongan kode diatas berfungsi untuk menyesuaikan jenis grafik dan format data yang akan ditampilkan menggunakan Chart.js. Jenis grafik yang akan ditampilkan disesuaikan dengan tipe visualisasi yang dipilih. Pada kondisi ini,

data tetap digunakan dalam bentuk *array* numerik tanpa perlu mengubah strukturnya. Apabila jenis grafik yang dipilih bukan scatter maupun bar, maka secara bawaan tipe grafik diatur menjadi 'line' dengan data yang juga tetap berupa *array* numerik.

3. Pembuatan Chart

```
1 chartInstances[chartId] = new Chart(ctx, {
2   type: finalType,
3   data: {
4     labels: finalType === 'scatter' ? [] : labels,
5     datasets: [{
6       label: 'Data',
7       data: datasetData,
8       backgroundColor: 'rgba(75, 192, 192, 0.2)',
9       borderColor: 'rgb(192, 75, 143)',
10      borderWidth: 1,
11      showLine: finalType === 'scatter' ? false : true
12    }]
13  },
14  options: {
15    responsive: true,
16    maintainAspectRatio: false,
17    scales: {
18      x: finalType === 'scatter'
19        ? { type: 'linear', position: 'bottom' }
20        : {
21          offset: true,
22          ticks: { padding: 10 },
23          grid: { drawTicks: false }
24        },
25      y: {
26        beginAtZero: false,
27        ticks: { padding: 10 }
28      }
29    },
30    layout: {
31      padding: { left: 20, right: 20, top: 20, bottom: 20 }
32    },
33    plugins: {
34      tooltip: {
35        mode: 'index',
36        intersect: true,
37      }
38    }
39  }
40 });
```

Gambar 4.13 Potongan Kode untuk Pembuatan Chart

Potongan kode di atas merupakan bagian dari pembuatan objek grafik baru dengan Chart.js, yang disimpan di dalam 'chartInstances' menggunakan 'chartId' sebagai *identifier*. Pada bagian 'type', nilai 'finalType' digunakan untuk menentukan jenis grafik sesuai hasil logika sebelumnya.

Struktur data grafik juga menyesuaikan: jika tipe grafik *scatter*, maka `'labels'` dibiarkan kosong karena Chart.js tidak memerlukan sumbu kategori untuk *scatter plot*, sedangkan pada tipe lain seperti *bar* dan *line*, `'labels'` diisi dengan data kategori sumbu X. Selain itu, pengaturan dataset disesuaikan melalui properti `'showLine'`, yang akan bernilai `'false'` khusus untuk *scatter* agar titik data tidak dihubungkan oleh garis, dan `'true'` untuk jenis grafik lainnya. Pada bagian `'options'`, properti `'responsive'` dan `'maintainAspectRatio'` memastikan grafik tetap menyesuaikan ukuran layar dengan proporsi yang diatur. Bagian `'scales'` juga dibuat fleksibel: jika tipe *scatter*, maka sumbu X diatur sebagai sumbu linear pada posisi bawah. Sedangkan pada tipe lainnya, sumbu X diformat sebagai sumbu kategori dengan pengaturan tampilan tambahan seperti jarak dan *grid*. Bagian `'layout'` digunakan untuk memberikan jarak tepi di sekitar area grafik, sedangkan `'plugins.tooltip'` mengatur perilaku penampilan *tooltip* agar lebih informatif saat pengguna berinteraksi dengan grafik. Dengan rangkaian pengaturan ini, grafik yang ditampilkan akan memiliki format visual yang benar sesuai tipe data dan kebutuhan pengguna, serta tetap responsif terhadap perubahan tampilan layar.

4. Fungsi `updateChart`


```

1  async function updateChart(chartId) {
2      const chartType = document.getElementById(`chartType${chartId}`).
      value;
3      const xFeature = document.getElementById(`xFeature${chartId}`).value;
4      const yFeature = document.getElementById(`yFeature${chartId}`).value;
5
6      if (!jsonData || typeof jsonData !== "object" || Object.keys(jsonData)
      ).length === 0) return;
7
8      const dataArray = Object.values(jsonData);
9      const labels = dataArray.map(item => item[xFeature]);
10     const chartData = dataArray.map(item => item[yFeature]);
11
12     if (
13         previousChartData[chartId]?.labels?.toString() === labels.
      toString() &&
14         previousChartData[chartId]?.data?.toString() === chartData.
      toString() &&
15         previousChartData[chartId]?.type === chartType
16     ) return;
17
18     previousChartData[chartId] = { labels: labels, data: chartData, type:
      chartType };
19
20     if (!chartInstances[chartId]) {
21         await createChart(chartId, chartType, labels, chartData);
22     }
23     else if (chartInstances[chartId].config.type !== chartType) {
24         chartInstances[chartId].destroy();
25         delete chartInstances[chartId];
26         await createChart(chartId, chartType, labels, chartData);
27     }
28     else {
29         const chart = chartInstances[chartId];
30
31
32         const startTime = performance.now();
33
34         if (chartType === 'scatter') {
35             chart.data.datasets[0].data = labels.map((x, i) => ({
36                 x: parseFloat(x),
37                 y: parseFloat(chartData[i])
38             }));
39         } else {
40             chart.data.labels = labels;
41             chart.data.datasets[0].data = chartData;
42         }

```

Gambar 4.14 Potongan Kode untuk Pembuatan Chart

Fungsi `updateChart` ini pada dasarnya bertugas memastikan setiap grafik yang tampil selalu mengikuti data sensor terbaru dan pengaturan yang dipilih pengguna. Fungsi akan mengidentifikasi tipe grafik, fitur pada sumbu x, dan fitur pada sumbu y dari *dropdown* yang berkaitan dengan ID grafik tertentu. Jika grafik belum ada sama sekali, grafik baru dibuat. Jika tipe grafik diubah,

grafik lama dihapus dan diganti dengan grafik baru agar sesuai tipe, dan jika hanya datanya yang berubah, label dan isi datanya diperbarui tanpa perlu mengubah grafik dari awal.

4.2.2 Implementasi Pada D3

Implementasi pada D3.js dilakukan dengan mengubah fungsi `CreateChart()` pada kode yang telah dibuat sebelumnya. Penjelasan kode yang diubah adalah sebagai berikut :

1. Melakukan inisialisasi dan mengatur ukuran *chart*. Kemudian mengubah sumbu x menjadi *scale band* yang dapat menerima data kategorikal dan sumbu y menjadi linear agar dapat menampilkan data numerik kontinyu. Kemudian membuat sumbu x dan sumbu y dengan menerapkan beberapa atribut. Seperti pada sumbu x, atribut yang diterapkan adalah *translate* untuk memindahkan sumbu menjadi dibagian bawah, memutar teks pada sumbu x dan mengatur font, serta memberikan filter untuk menampilkan keterangan di sumbu x. Adapun atribut yang diterapkan di sumbu y adalah `d3.axisLeft tickValues(uniqueYValues)` membatasi label sumbu y hanya pada nilai-nilai tertentu.

```

1 const chartInstances = {};
2 async function createChart(chartId, chartType, labels, data) {
3   const svg = d3.select(`#chart${chartId}`);
4   svg.selectAll("*").remove(); // Bersihkan SVG
5
6   const width = window.innerWidth;
7   const height = 500;
8   const margin = { top: 20, right: 30, bottom: 80, left: 50 };
9
10  svg.attr("viewBox", `0 0 ${width} ${height}`)
11    .attr("preserveAspectRatio", "xMidYMid meet")
12    .style("width", "100%")
13    .style("height", "auto");
14
15
16
17  const x = d3.scaleBand()
18    .domain(labels)
19    .range([margin.left, width - margin.right])
20    .padding(0.2);
21
22  const y = d3.scaleLinear()
23    .domain([d3.min(data), d3.max(data)])
24    .range([height - margin.bottom, margin.top]);
25
26  const uniqueYValues = Array.from(new Set(data)).sort((a, b) => a - b);
27
28  const xAxis = g => {
29    g.attr("transform", `translate(0,${height - margin.bottom})`)
30      .call(d3.axisBottom(x).tickSizeOuter(0))
31      .selectAll("text")
32      .attr("transform", "rotate(90)")
33      .attr("x", 9)
34      .attr("y", 0)
35      .style("text-anchor", "start")
36      .style("font-size", "12px")
37      .filter((d, i) => i % 3 === 0); // Tampilkan setiap 3 label
38  };
39
40  const yAxis = g => {
41    g.attr("transform", `translate(${margin.left},0)`)
42      .call(d3.axisLeft(y).tickValues(uniqueYValues));
43  };
44
45  svg.append("g").call(xAxis);
46  svg.append("g").call(yAxis);

```

Gambar 4.15 Membuat Sumbu Pada D3

- Menyediakan *tooltip* yang akan ditampilkan saat kursor di-*hover* ke titik data pada *chart*.

```

1 // Tooltip
2 const tooltip = d3.select("body").append("div")
3   .attr("class", "tooltip")
4   .style("position", "absolute")
5   .style("visibility", "hidden")
6   .style("background-color", "rgba(0,0,0,0.7)")
7   .style("color", "white")
8   .style("padding", "8px")
9   .style("border-radius", "4px")
10  .style("font-size", "12px");
11

```

Gambar 4.16 Potongan Kode untuk Membuat *Tooltip*

- Menambahkan visualisasi berdasarkan jenis grafik yang akan dipilih (*line chart*, *bar chart*, atau *scatter plot*). Kode berikut membuat grafik garis (*line chart*). Pertama, dibuat jalur garis menggunakan `d3.line()` berdasarkan data dan

label yang dipetakan ke sumbu x dan y. Kemudian, titik-titik data ditampilkan sebagai titik merah. Setiap titik memiliki interaksi *tooltip* yang muncul saat *pointer* diarahkan ke titik dan akan menampilkan informasi sumbu x dan sumbu y.

```

1 } else if (chartType === "bar") {
2   svg.selectAll(".bar")
3     .data(data)
4     .enter()
5     .append("rect")
6     .attr("class", "bar")
7     .attr("x", (d, i) => x(labels[i]))
8     .attr("y", d => y(d))
9     .attr("width", x.bandwidth())
10    .attr("height", d => height - margin.bottom - y(d))
11    .attr("fill", "steelblue")
12    .on("mouseover", function(event, d) {
13      tooltip.style("visibility", "visible")
14      .text(`Waktu: ${labels[data.indexOf(d)]}, Nilai: ${d}`);
15    })
16    .on("mousemove", function(event) {
17      tooltip.style("top", (event.pageY + 5) + "px")
18      .style("left", (event.pageX + 5) + "px");
19    })
20    .on("mouseout", function() {
21      tooltip.style("visibility", "hidden");
22    });
23

```

Gambar 4.17 Potongan Kode untuk Mengimplementasikan *Line Chart*

4. Membuat grafik diagram batang dengan elemen `rect`. Pada kode berikut posisi batang diatur dengan `.attr("x", ...)` berdasarkan label dan skala data x. Tinggi batang dihitung dari selisih tinggi SVG dan posisi data pada sumbu y, sementara lebar batang diambil dari `x.bandwidth()`.

```

1 } else if (chartType === "bar") {
2   svg.selectAll(".bar")
3     .data(data)
4     .enter()
5     .append("rect")
6     .attr("class", "bar")
7     .attr("x", (d, i) => x(labels[i]))
8     .attr("y", d => y(d))
9     .attr("width", x.bandwidth())
10    .attr("height", d => height - margin.bottom - y(d))
11    .attr("fill", "steelblue")
12    .on("mouseover", function(event, d) {
13      tooltip.style("visibility", "visible")
14      .text(`Waktu: ${labels[data.indexOf(d)]}, Nilai: ${d}`);
15    })
16    .on("mousemove", function(event) {
17      tooltip.style("top", (event.pageY + 5) + "px")
18      .style("left", (event.pageX + 5) + "px");
19    })
20    .on("mouseout", function() {
21      tooltip.style("visibility", "hidden");
22    });
23

```

Gambar 4.18 Potongan Kode untuk Mengimplementasikan *Bar Chart*

5. Membuat grafik *scatter plot*, dengan menentukan posisi x dan y. Masing-masing titik digambar sebagai lingkaran berwarna kuning dengan garis tepi hitam.

Tooltip interaktif juga digunakan, sama seperti pada grafik batang, untuk menunjukkan informasi detail saat *pointer* diarahkan ke titik.

```
1 } else if (chartType === "scatter") {
2     svg.selectAll(".dot")
3         .data(data)
4         .enter()
5         .append("circle")
6         .attr("class", "dot")
7         .attr("cx", (d, i) => x(labels[i]) + x.bandwidth() / 2)
8         .attr("cy", d => y(d))
9         .attr("r", 6)
10        .attr("fill", "orange")
11        .attr("stroke", "black")
12        .attr("stroke-width", 1.5)
13        .on("mouseover", function(event, d) {
14            tooltip.style("visibility", "visible")
15            .text(`Waktu: ${labels[data.indexOf(d)]}, Nilai: ${d}`);
16        })
17        .on("mousemove", function(event) {
18            tooltip.style("top", (event.pageY + 5) + "px")
19            .style("left", (event.pageX + 5) + "px");
20        })
21        .on("mouseout", function() {
22            tooltip.style("visibility", "hidden");
23        });
24 }
```

Gambar 4.19 Potongan Kode untuk Mengimplementasikan *Scatter Plot*

6. Fungsi Update Pada D3

```

1  async function updateChart(chartId) {
2
3      const chartType = document.getElementById(`chartType${chartId}`).
value;
4      const xFeature = document.getElementById(`xFeature${chartId}`).value;
5      const yFeature = document.getElementById(`yFeature${chartId}`).value;
6
7      if (!jsonData || typeof jsonData !== "object" || Object.keys(jsonData
).length === 0) {
8          console.error("jsonData tidak tersedia atau bukan objek!",
jsonData);
9          return;
10     }
11
12     // Konversi JSON dari objek ke array
13     const dataArray = Object.values(jsonData);
14
15     if (dataArray.length === 0) {
16         console.error("Tidak ada data dalam JSON!", jsonData);
17         return;
18     }
19
20     // Ambil data untuk sumbu X dan Y
21     const labels = dataArray.map(item => item[xFeature]);
22     const chartData = dataArray.map(item => item[yFeature]);
23
24     if (!labels || !chartData || labels.length === 0 || chartData.length
=== 0) {
25         console.error("Data tidak ditemukan untuk fitur yang dipilih!",
xFeature, yFeature, jsonData);
26         return;
27     }

```

Gambar 4.20 Fungsi untuk Memperbarui Chart D3

Fungsi diatas akan mengambil informasi dari elemen HTML berupa tipe grafik, nama fitur untuk sumbu X, dan fitur untuk sumbu Y berdasarkan ID grafik yang sedang diproses. Setelah itu, fungsi memeriksa apakah jsonData yang berisi data sensor sudah valid, berupa objek, dan memiliki isi. Jika data tidak ada atau tidak sesuai format, fungsi akan menghentikan proses dan mencatat pesan kesalahan ke konsol untuk memudahkan pengecekan. Kemudian objek JSON diubah menjadi *array* agar lebih mudah diolah. Jika *array* ini kosong, berarti tidak ada data valid yang dapat divisualisasikan, sehingga fungsi kembali membatalkan proses dan menampilkan eror. Kemudian, data untuk sumbu X dan Y diambil masing-masing dengan melakukan mapping ke *array* fitur X menjadi label, dan fitur Y menjadi data nilai.

4.2.3 Implementasi Pada Highcharts

Implementasi Highcharts dilakukan pada fungsi CreateChart() dengan menambahkan kode berikut.

1. Implementasi Kontainer Grafik

Pertama, kode membuat id kontainer grafik. Jika grafik dengan `chartId` yang sama sudah pernah dibuat sebelumnya, maka grafik lama akan dihapus menggunakan fungsi `destroy()` untuk mencegah duplikasi.

```
1  const chartInstances = {};  
2  
3  async function createChart(chartId, chartType, labels, data) {  
4      const containerId = `chart${chartId}`;  
5      const container = document.getElementById(containerId);  
6  
7      if (!container) {  
8          console.error(`Container untuk Chart ${chartId}  
tidak ditemukan di DOM!`);  
9          return;  
10     }  
11  
12     console.time(`Render Time Chart ${chartId}`);  
13     let startTime = performance.now();  
14  
15     // Hapus chart sebelumnya kalau ada  
16     if (chartInstances[chartId]) {  
17         chartInstances[chartId].destroy();  
18     }
```

Gambar 4.21 Potongan Kode untuk Kontainer Grafik

2. Implementasi ChartOptions

Selanjutnya, objek `chartOptions` disusun sebagai konfigurasi grafik. Pada bagian `chart`, properti `renderTo` menunjukkan elemen HTML yang menjadi wadah grafik, sementara `type` diisi dengan tipe grafik dari variabel `highchartsType` yang sudah ditetapkan.

```

1  const highchartsType =
2      chartType === "scatter"
3      ? "scatter"
4      : (chartType === "bar" ? "column" : chartType);
5
6  const chartOptions = {
7      chart: {
8          renderTo: containerId,
9          type: highchartsType,
10         zoomType: 'xy'
11     },
12     title: { text: `Chart ${chartId}` },
13     xAxis: {
14         categories: chartType === "scatter" ? null : labels,
15         title: { text: document.getElementById(`xFeature${chartId}`).
value }
16     },
17     yAxis: {
18         title: { text: document.getElementById(`yFeature${chartId}`).
value }
19     },
20     series: [{
21         name: 'Data',
22         data: chartType === "scatter"
23             ? labels.map((x, i) => [parseFloat(x), parseFloat(data[i
24             ])]))
25             : data,
26     }],
27     responsive: {
28         rules: [{
29             condition: { maxWidth: 600 },
30             chartOptions: { legend: { enabled: false } }
31         }]
32     }
33 };
34
35 const chart = Highcharts.chart(chartOptions);
36 chartInstances[chartId] = chart;

```

Gambar 4.22 Potongan Kode untuk Membuat Grafik Highcharts

Properti `zoomType` diaktifkan agar pengguna dapat melakukan pembesaran tampilan grafik pada sumbu horizontal dan vertikal. Untuk sumbu X (`xAxis`), apabila grafik berjenis *scatter*, properti `categories` diatur `null` karena *scatter plot* memanfaatkan nilai numerik langsung, bukan kategori. Sebaliknya, untuk jenis grafik lain, `labels` digunakan sebagai penanda kategori. Judul pada sumbu X dan Y diatur berdasarkan input fitur X dan Y yang dipilih pengguna melalui elemen HTML.

Bagian `series` berfungsi menentukan data yang akan divisualisasikan. Jika grafik bertipe *scatter*, data diolah menjadi pasangan nilai `[x, y]` dengan format numerik melalui `parseFloat`. Untuk tipe grafik lain, data digunakan

sebagaimana adanya. Di bagian *responsive*, terdapat aturan tambahan yang akan menyembunyikan legenda grafik secara otomatis jika lebar layar di bawah 600 piksel, sehingga tampilan tetap rapi pada perangkat berlayar kecil. Kemudian, perintah `Highcharts.chart(chartOptions)` akan me-*render* grafik sesuai pengaturan tersebut ke dalam halaman. Hasil grafik ini kemudian disimpan ke dalam `chartInstances[chartId]` agar dapat diakses dan diperbarui.

3. Update Highcharts

Fungsi `updateChart` yang berfungsi untuk memperbarui data grafik berdasarkan fitur yang dipilih. Langkah kerjanya dimulai dengan mengambil jenis grafik serta nama fitur untuk sumbu X dan Y dari elemen HTML. Kemudian, fungsi akan memvalidasi `jsonData`. Selanjutnya, data JSON dikonversi menjadi *array*, lalu dipecah menjadi data sumbu X (`labels`) dan data sumbu Y (`chartData`). Jika hasil ekstraksi data kosong atau gagal, fungsi juga akan berhenti sambil menampilkan pesan error. Terakhir, jika semua valid, data tersebut akan dipakai untuk membuat atau memperbarui grafik.

```

1  async function updateChart(chartId) {
2
3      const chartType = document.getElementById(`chartType${chartId}`).
value;
4      const xFeature = document.getElementById(`xFeature${chartId}`).value;
5      const yFeature = document.getElementById(`yFeature${chartId}`).value;
6
7      if (!jsonData || typeof jsonData !== "object" || Object.keys(jsonData
).length === 0) {
8          console.error("jsonData tidak tersedia atau bukan objek!",
jsonData);
9          return;
10     }
11
12     // Konversi JSON dari objek ke array
13     const dataArray = Object.values(jsonData);
14
15     if (dataArray.length === 0) {
16         console.error("Tidak ada data dalam JSON!", jsonData);
17         return;
18     }
19
20     // Ambil data untuk sumbu X dan Y
21     const labels = dataArray.map(item => item[xFeature]);
22     const chartData = dataArray.map(item => item[yFeature]);
23
24     if (!labels || !chartData || labels.length === 0 || chartData.length
=== 0) {
25         console.error("Data tidak ditemukan untuk fitur yang dipilih!",
xFeature, yFeature, jsonData);
26         return;
27     }
28
29

```

Gambar 4.23 Potongan Kode untuk Memperbarui Grafik Highcharts

4.3 Pengukuran Performa Rendering

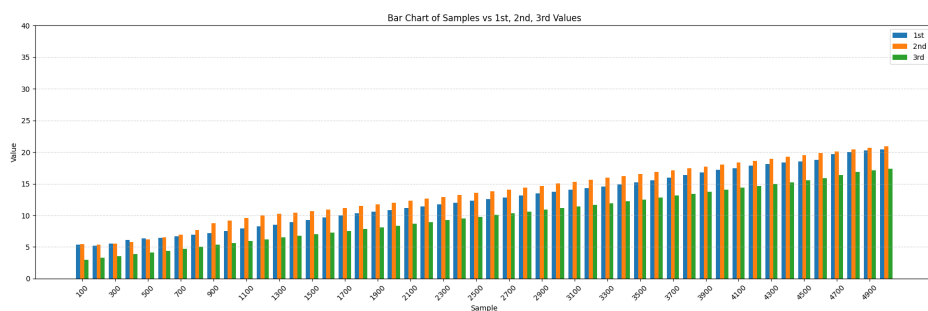
Pada bagian ini, dilakukan pengukuran performa *rendering* untuk menilai seberapa efisien dan responsif *library* dalam menampilkan data visualisasi. Pengujian dilakukan dengan menggunakan beberapa skenario jumlah data yang ditampilkan melalui *line chart* untuk mengamati waktu yang dibutuhkan dalam me-render grafik ke layar. Proses pengukuran dilakukan dengan mencatat waktu *rendering* sejak inisialisasi data hingga grafik ditampilkan secara penuh di DOM. Analisis ini bertujuan untuk memberikan gambaran kuantitatif mengenai kemampuan *library* dalam menangani proses render, serta menjadi dasar perbandingan dengan *library* visualisasi data lainnya. Pengujian dilakukan sebanyak tiga kali eksperimen untuk menentukan *library* paling optimal dan guna menghilangkan bias dalam pengukuran. Hasil pengujian kemudian dibandingkan dalam tabel yang berisi jumlah sampel, hasil eksperimen pertama (1st), hasil eksperimen kedua (2nd) dan hasil eksperimen ketiga (3rd).

4.3.1 Chart.js

Pengukuran ini dilakukan untuk mengevaluasi performa Chart.js dalam *me-render* berbagai jenis grafik dengan jumlah data yang bervariasi. Waktu *rendering* dihitung mulai dari proses inisialisasi chart hingga seluruh elemen visual berhasil ditampilkan pada halaman. Hasil *rendering* dengan *library* Chart.js sebanyak tiga kali eksperimen adalah sebagai berikut :

Tabel 4.1 Komparasi Render *Chart.js*

Samples	1 st	2 nd	3 rd
100	5.36	5.46	3.00
200	5.19	5.35	3.27
300	5.56	5.54	3.58
400	6.10	5.77	3.86
500	6.40	6.20	4.13
600	6.42	6.50	4.41
700	6.66	6.92	4.72
...	...	...	...
4400	18.33	19.23	15.25
4500	18.55	19.51	15.55
4600	18.78	19.81	15.86
4700	19.72	20.10	16.40
4800	20.02	20.44	16.87
4900	20.23	20.66	17.12
5000	20.43	20.91	17.37



Gambar 4.24 Perbandingan Rendering Chart.js

Hasil pengujian waktu *render* Chart.js pada tiga kali percobaan (1st, 2nd, dan 3rd) menunjukkan pola pertumbuhan yang konsisten seiring bertambahnya jumlah sampel data, dari 100 hingga 5000. Pada sampel terkecil, yaitu 100 data, waktu *render* pada percobaan pertama tercatat sebesar 5.36 ms, percobaan kedua 5.46 ms,

dan percobaan ketiga 3.00 ms. Seiring bertambahnya jumlah data, tren kenaikan pada ketiga percobaan tampak serupa. Misalnya pada 1000 data, waktu *render* untuk percobaan pertama naik menjadi 7.49 ms, percobaan kedua 9.19 ms, dan percobaan ketiga 5.65 ms. Pola yang sama terus berlanjut hingga data terbesar, yaitu 5000 sampel, di mana percobaan pertama mencapai 20.43 ms, percobaan kedua 20.91 ms, sedangkan percobaan ketiga tetap lebih rendah di angka 17.37 ms.

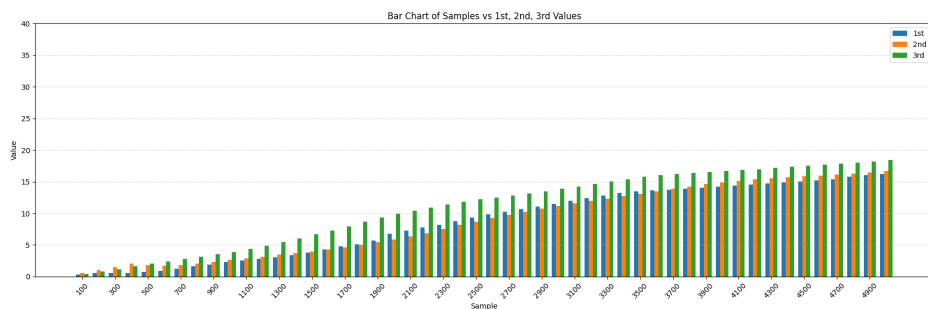
Data ini menunjukkan bahwa meskipun kondisi *environment* dan skrip yang digunakan sama, selalu muncul deviasi antar percobaan, terutama antara percobaan pertama dan kedua yang cenderung memiliki selisih mendekati stabil di rentang 0.5–1 ms, sedangkan percobaan ketiga relatif lebih cepat dengan selisih 2–3 ms lebih rendah dibanding dua percobaan lainnya. Fenomena ini mengindikasikan bahwa Chart.js mempertahankan konsistensi skalabilitas *render* di setiap percobaan meskipun ada variasi antar iterasi. Kenaikan waktu *render* mendekati linier sesuai penambahan beban data, yang wajar terjadi karena semakin banyak elemen visual dan kalkulasi yang harus diolah pada browser. Perbedaan antar percobaan kemungkinan besar berkaitan dengan variasi eksekusi di tingkat proses atau pengelolaan memori internal browser, meskipun secara teknis tidak ada perubahan pada sumber kode dan tidak dilakukan optimasi tambahan.

4.3.2 D3

Pengujian ini bertujuan untuk menilai kinerja D3 dalam menampilkan kemampuan *rendering library* D3. Proses pengukuran dimulai dari saat chart diinisialisasi hingga seluruh komponen visual sepenuhnya muncul di layar. Untuk memperoleh hasil yang konsisten, proses *rendering* dilakukan sebanyak tiga kali dengan D3.js. Berikut merupakan grafik perbandingan hasil dari ketiga percobaan tersebut:

Tabel 4.2 Komparasi Render D3

Samples	1 st	2 nd	3 rd
100	0.35	0.58	0.40
200	0.62	1.05	0.79
300	0.59	1.50	1.13
400	0.60	2.03	1.66
500	0.73	1.81	2.03
600	0.93	1.72	2.39
700	1.26	1.80	2.77
...	...	...	...
4400	14.91	15.69	17.38
4500	15.08	15.84	17.56
4600	15.25	15.99	17.72
4700	15.44	16.14	17.89
4800	15.81	16.31	18.06
4900	16.03	16.48	18.23
5000	16.23	16.66	18.41

**Gambar 4.25** Perbandingan Rendering D3

Berdasarkan hasil pengamatan terhadap data *rendering* D3.js, ketiga percobaan memperlihatkan pola kenaikan waktu *rendering* yang konsisten seiring bertambahnya jumlah sampel data. Hal ini wajar mengingat D3.js memanfaatkan manipulasi DOM dan SVG secara intensif, sehingga makin banyak data yang di-*render*, makin besar pula beban prosesnya. Nilai pada percobaan pertama memperlihatkan pola pertumbuhan yang konsisten dan stabil, naik secara bertahap dari 0,35 ms hingga

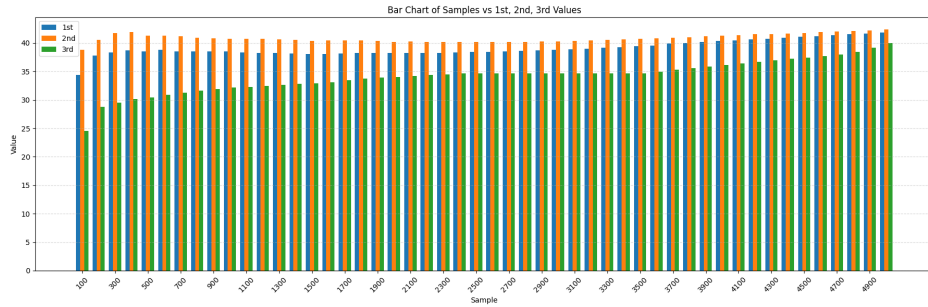
mencapai 16,20 di akhir rentang data, tanpa penurunan. Nilai percobaan kedua juga mengalami peningkatan yang stabil, bahkan lebih cepat dibandingkan pertama dalam beberapa titik data. Dimulai dari 0,58 ms, nilainya terus meningkat hingga menjadi yang tertinggi di titik akhir, yaitu 16,66ms. Nilai 3rd menunjukkan tren yang sedikit berbeda. Nilai ini tumbuh paling pesat dan mendominasi mencapai puncaknya di angka 18,41ms. Meskipun masing-masing memiliki laju pertumbuhan yang berbeda dan pola yang sedikit bervariasi, terutama pada nilai percobaan ketiga yang sempat menurun di akhir, secara umum tren ketiganya tetap mengarah naik.

4.3.3 highcharts

Untuk menilai kemampuan *render* Highcharts dalam menampilkan berbagai jenis grafik dengan variasi jumlah data, dilakukan proses pengujian dengan mengukur waktu yang dibutuhkan sejak inisialisasi grafik hingga seluruh komponen visual sepenuhnya muncul di layar. Evaluasi ini dilakukan sebanyak tiga kali untuk menampilkan data dengan *line chart*, dan berikut merupakan hasil yang diperoleh dari penggunaan Highcharts dalam tiga eksperimen tersebut.

Tabel 4.3 *Komparasi* Render Highcharts

Samples	1 st	2 nd	3 rd
100	34.35	38.80	24.57
200	37.81	40.50	28.81
300	38.37	41.77	29.50
400	38.67	41.93	30.17
500	38.55	41.27	30.40
600	38.79	41.32	30.86
700	38.53	41.20	31.26
...	...	...	...
4400	40.91	41.68	37.20
4500	41.06	41.77	37.44
4600	41.22	41.89	37.73
4700	41.40	41.99	37.98
4800	41.52	42.10	38.44
4900	41.64	42.21	39.17
5000	41.82	42.35	39.95



Gambar 4.26 Perbandingan Rendering Highcharts

Ditinjau dari ketiga eksperimen yang dilakukan, waktu *render* dengan *library* Highcharts meningkat di awal dan cenderung stabil di data dalam jumlah lebih dari 1000. Hal ini dikarenakan sebuah mekanisme TurboThreshold yang menjadi fitur default di dalam Highcharts yang belum berjalan. Fitur TurboThreshold ini adalah sebuah konfigurasi yang memungkinkan Highcharts untuk memvalidasi setiap titik yang di-*render*. Defaultnya, fitur ini akan mulai dijalankan secara otomatis saat jumlah data lebih dari 1000. Hal ini menjadi alasan yang valid karena setelah data ke 1000, tidak ada kenaikan waktu *render* yang signifikan dan kenaikan cenderung landai.

4.4 Pengukuran Penggunaan CPU

Pengukuran penggunaan CPU ditinjau melalui Process ID (PID) terspesifik pada halaman web. Fungsi dari pengukuran CPU ini adalah untuk mengukur rata-rata penggunaan CPU yang digunakan untuk me-*render* jumlah data tertentu. Tahapan yang dilakukan dalam pengukuran ini antara lain:

1. Membuka Task Manager pada Chrome untuk melihat PID dari tab yang digunakan.
2. Menuliskan PID pada kode yang telah dibuat. Kode ini berfungsi untuk mengambil nilai CPU yang digunakan pada suatu tab setiap detik.

```

1 import psutil
2 import time
3 from datetime import datetime
4
5 pid = 20320
6 p = psutil.Process(pid)
7
8 with open('highchart_4th.csv', 'w') as f:
9     f.write('Timestamp,CPU Usage (%)\n')
10
11 print("Monitoring CPU usage dari tab Chrome (PID:", pid, ")")
12
13 try:
14     while True:
15         cpu = p.cpu_percent(interval=1) # Ambil CPU usage dari proses ini
16         timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
17
18         with open('highchart_4th.csv', 'a') as f:
19             f.write(f"{timestamp},{cpu}\n")
20
21         print(f"[{timestamp}] CPU: {cpu}%")
22         # time.sleep()
23 except KeyboardInterrupt:
24     print("Selesai")

```

Gambar 4.27 Potongan Kode untuk Mengukur Penggunaan CPU

3. Nilai CPU tersebut kemudian akan disimpan dalam file csv.

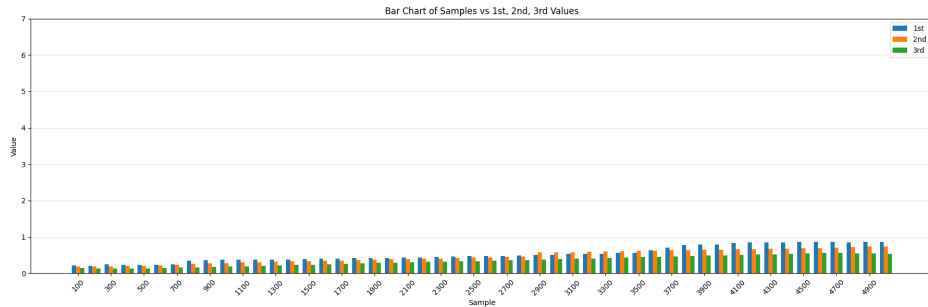
4.4.1 Chart.js

Pengukuran penggunaan CPU pada *library* Chart.js untuk menampilkan data IoT dalam bentuk *line chart* menunjukkan hasil seperti berikut :

Tabel 4.4 Komparasi CPU *Chart.js*

Samples	1 st	2 nd	3 rd
100	0.22	0.18	0.15
200	0.21	0.19	0.13
300	0.25	0.19	0.13
400	0.23	0.20	0.14
500	0.24	0.21	0.14
600	0.24	0.22	0.15
700	0.25	0.23	0.16
...	...	...	...
4400	0.85	0.68	0.53
4500	0.86	0.69	0.55
4600	0.87	0.70	0.56
4700	0.86	0.71	0.56
4800	0.85	0.72	0.55

Samples	1 st	2 nd	3 rd
4900	0.86	0.73	0.54
5000	0.86	0.74	0.54



Gambar 4.28 Grafik Komparasi Chart.js

Dari ketiga eksperimen yang dilakukan, penggunaan CPU pada *library* Chart.js untuk menampilkan data IoT memiliki kemiripan. Grafik kenaikan di semua iterasi pun terlihat rendah dan landai. Bahkan dari ketiga iterasi, hasil konsumsi CPU tetap menunjukkan peningkatan namun hanya berkisar pada 0-1% dan menyentuh titik paling tinggi ada di 0.85 ms. Fenomena ini dapat dikarenakan Chart.js menggunakan *canvas-based rendering* yang lebih menggunakan sumber daya GPU dibanding CPU sehingga alokasi CPU jauh lebih rendah dibandingkan dengan D3 ataupun Highcharts.

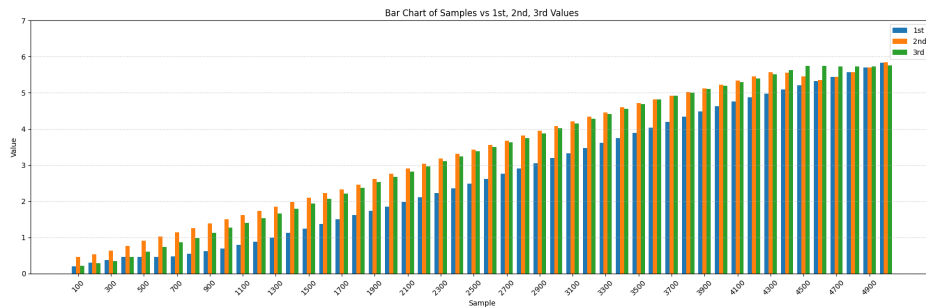
4.4.2 D3

Pengukuran penggunaan CPU pada *library* D3 untuk menampilkan data IoT dalam bentuk line chart menunjukkan hasil seperti berikut :

Tabel 4.5 Komparasi CPU *Chart.js*

Samples	1 st	2 nd	3 rd
100	0.20	0.46	0.22
200	0.16	0.53	0.28
300	0.14	0.64	0.35
400	0.13	0.77	0.46
500	0.14	0.91	0.61
600	0.16	1.03	0.74
700	0.19	1.14	0.86
...	...	...	...
4400	5.10	5.56	5.64

4500	5.21	5.46	5.75
4600	5.33	5.36	5.74
4700	5.45	5.44	5.73
4800	5.57	5.57	5.73
4900	5.70	5.71	5.74
5000	5.83	5.84	5.76



Gambar 4.29 Grafik Komparasi D3

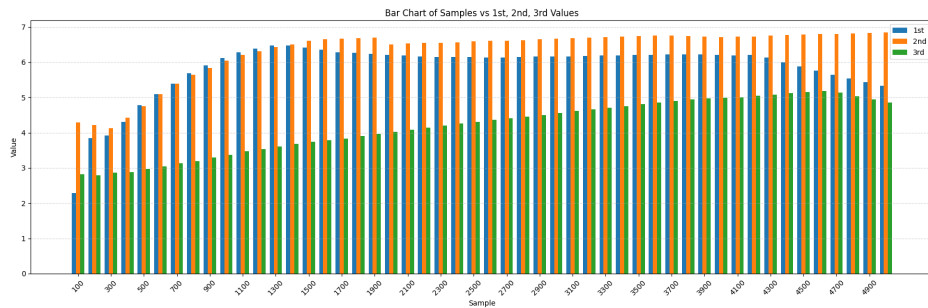
Ketiga grafik memiliki pola kenaikan yang sama karena peningkatan ukuran data, peningkatan *render* elemen DOM, dan *rendering* grafik yang semakin kompleks. Kenaikan CPU pada proses ini mencapai titik tertinggi di 5,8 ms dan tren ini terjadi pada ketiga eksperimen yang dilakukan.

4.4.3 Highcharts

Pengukuran penggunaan CPU pada *library* Highcharts untuk menampilkan data IoT dalam bentuk line chart menunjukkan hasil seperti berikut :

Tabel 4.6 Komparasi CPU pada *Highcharts*

Samples	1 st	2 nd	3 rd
100	2.28	4.30	2.83
200	3.84	4.22	2.79
300	3.92	4.13	2.86
400	4.31	4.43	2.89
500	4.78	4.76	2.97
600	5.09	5.10	3.04
700	5.40	5.39	3.13
...	...	...	...
4400	6.00	6.77	5.13
4500	5.88	6.78	5.16
4600	5.76	6.80	5.19
4700	5.65	6.81	5.14
4800	5.54	6.82	5.04
4900	5.44	6.83	4.94
5000	5.34	6.84	4.85



Gambar 4.30 Grafik Komparasi Highcharts

Dari ketiga eksperimen yang dilakukan, secara konsisten terdapat peningkatan penggunaan CPU saat data kurang dari 1500 kemudian setelahnya data menjadi lebih stabil atau mengalami penurunan. Hal ini dapat disebabkan oleh fungsi *TurboThreshold* yang juga turut mempengaruhi performa *render* dan penggunaan memori.

4.5 Pengukuran *Footprint Memory*

Pengukuran penggunaan memori dintinjau dari penggunaan *footprint memory* pada sebuah halaman website spesifik yang mengukur *private memory*, *heap memory* untuk menyimpan objek di dalam website termasuk data dari json yang ditampilkan,

dan stack memory untuk menyimpan variabel lokal dan kontrol eksekusi fungsi. Langkah yang dilakukan pada tahap ini adalah sebagai berikut :

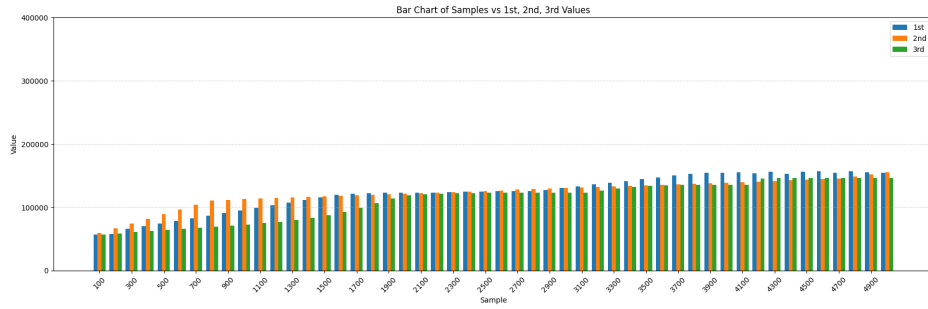
1. Menampilkan visualisasi pada tab incognito
2. Meninjau penggunaan *memory footprint* yang dapat diakses melalui *Task Manager* pada Chrome
3. Mencatat performa di tiap titik yang telah ditentukan, yaitu 100, 200, 300, 400, dan seterusnya hingga data ke 5000 Langkah tersebut dilakukan sebanyak tiga kali eksperimen di tiap-tiap *library* untuk mengukur penggunaan memorinya.

4.5.1 Chart.js

Pengukuran memori pada Chart.js dalam tiga eksperimen menghasilkan grafik perbandingan seperti berikut :

Tabel 4.7 Komparasi Memori *Chart.js*

Samples	1 st	2 nd	3 rd
100	56,730	59,600	56,883
200	57,460	66,959	58,921
300	66,104	74,317	60,734
400	70,215	81,676	62,489
500	74,326	89,035	64,120
600	78,437	96,394	65,871
700	82,548	103,752	67,534
...	...	...	...
4400	152,967	142,615	146,051
4500	155,905	143,490	146,128
4600	157,111	144,365	146,152
4700	154,744	145,240	146,062
4800	156,783	148,564	146,067
4900	155,316	151,889	146,070
5000	154,872	155,213	146,072



Gambar 4.31 Grafik Komparasi Chart.js

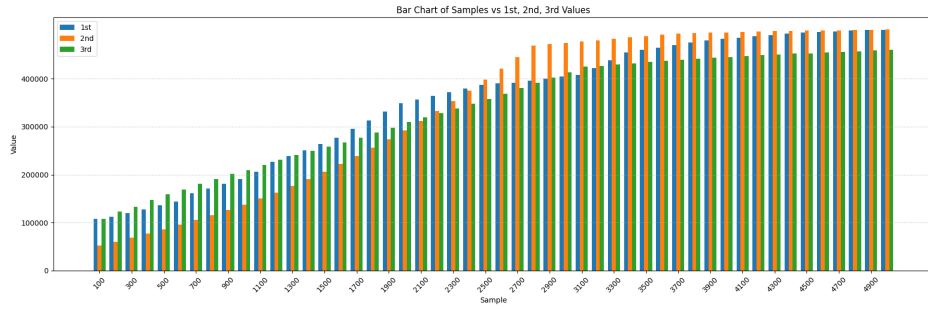
Dari ketiga eksperimen yang tertampil pada grafik, dapat diamati bahwa proses *rendering* menggunakan *library* Chart.js menunjukkan peningkatan konsumsi *footprint memory* yang bertahap di setiap pertambahan jumlah data. Hal ini menunjukkan adanya akumulasi beban *render* pada tiap eksperimen yang disebabkan oleh peningkatan kompleksitas visualisasi dan jumlah elemen yang di-*render* ulang untuk menampilkan data dengan jumlah data yang selalu meningkat.

4.5.2 D3

Pengukuran memori pada D3 dalam tiga eksperimen menghasilkan grafik perbandingan seperti berikut :

Tabel 4.8 Komparasi Memori *D3.js*

Samples	1 st	2 nd	3 rd
100	108,144	52,290	108,203
200	112,211	60,310	112,289
300	120,125	68,940	120,145
400	128,039	77,520	128,070
500	135,953	86,380	136,005
600	143,883	95,670	143,920
700	161,812	105,290	161,850
...	...	...	...
4400	498,702	494,992	511,000
4500	499,774	473,502	515,200
4600	500,344	473,790	518,000
4700	500,812	474,080	519,000
4800	501,496	487,760	519,600
4900	501,855	494,600	519,900
5000	502,214	501,440	520,120



Gambar 4.32 Grafik Komparasi D3

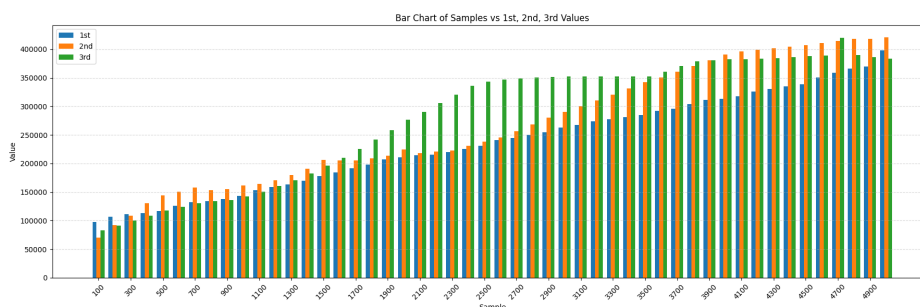
Terdapat selisih yang cukup besar pada eksperimen ketiga jika dibandingkan dengan eksperimen 1 dan eksperimen 2. Hal ini mengindikasikan terdapat kemungkinan DOM yang tidak dibersihkan. Namun, ditinjau dari tren dan hasil pengukuran dua eksperimen lainnya, pola tren penggunaan memori dari ketiga eksperimen tetap menunjukkan konsistensi. Ketiganya mencerminkan peningkatan *footprint memory* secara bertahap seiring bertambahnya jumlah data yang di-render. Hal ini menjelaskan bahwa D3 secara umum menunjukkan karakteristik peningkatan konsumsi memori yang linier terhadap penambahan kompleksitas dan jumlah data yang divisualisasikan.

4.5.3 Highcharts

Pengukuran memori pada Highcharts dalam tiga kali eksperimen menunjukkan hasil seperti berikut :

Tabel 4.9 Komparasi Memori *Highcharts*

Samples	1 st	2 nd	3 rd
100	97,183	70,350	83,214
200	106,565	92,120	91,500
300	111,724	108,760	100,200
400	113,001	131,100	108,600
500	116,409	143,870	117,800
600	126,316	150,550	124,300
700	132,553	157,700	130,800
...	...	...	...
4400	335,295	404,300	386,200
4500	338,330	407,250	388,000
4600	350,158	410,600	390,200
4700	358,557	414,200	420,268
4800	365,698	417,800	390,084
4900	369,709	418,450	381,600
5000	398,125	420,510	383,616



Gambar 4.33 Grafik Komparasi Highcharts

Grafik yang ditampilkan menunjukkan perbandingan penggunaan memori dari *library* Highcharts selama proses pengujian dengan kondisi lingkungan dan kode yang digunakan sama persis untuk ketiga skenario. Karena variabel-variabel eksternal dikendalikan secara ketat, perbedaan konsumsi memori yang muncul kemungkinan besar disebabkan oleh perilaku internal Highcharts saat runtime, seperti mekanisme Highcharts dalam mengelola objek grafik, menyimpan *cache*, atau menjalankan proses pembaruan visual. Pada pengujian pertama, terlihat pola kenaikan memori yang cenderung stabil dan terukur. Hal ini mengindikasikan bahwa proses pembersihan memori atau pengelolaan ulang elemen-elemen grafik berjalan cukup baik. Sebaliknya, pada pengujian kedua, lonjakan memori terjadi cukup drastis saat jumlah

eksperimen bertambah. Hal ini mengarah pada kemungkinan terjadinya penumpukan elemen atau data yang tidak segera dibersihkan, Sementara itu, hasil pengujian ketiga memperlihatkan lonjakan penggunaan memori yang cukup cepat di awal, tetapi kemudian stagnan di pertengahan grafik. Hal ini menunjukkan bahwa pada titik tertentu, Highcharts mulai melakukan optimasi agar konsumsi memori tidak terus naik. Namun, di bagian akhir grafik, nilainya kembali naik atau justru menurun, yang bisa mengindikasikan adanya proses pembersihan memori oleh sistem.

4.6 Analisis Skalabilitas

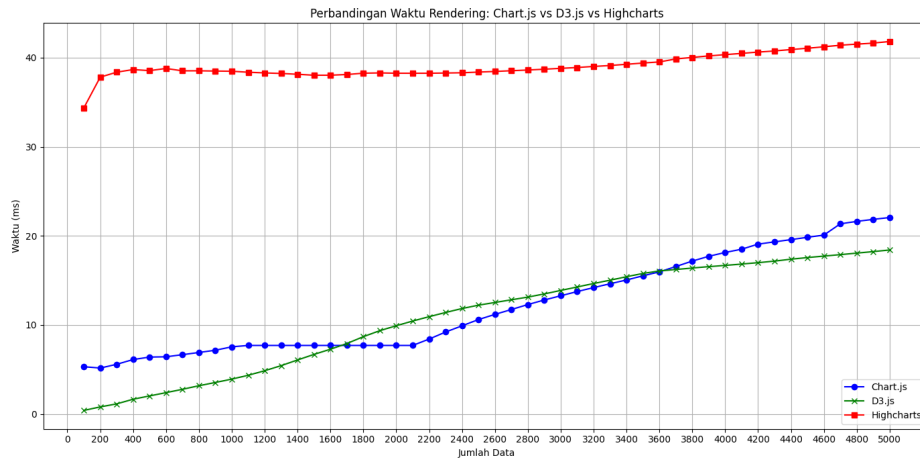
Analisis skalabilitas dan komparasi digunakan untuk membandingkan setiap *library* dan pengaruhnya terhadap parameter-parameter yang diukur dengan kuantitas data tertentu dalam jumlah yang sama pada lingkungan dan kondisi yang seragam. Pengujian dilakukan secara berulang guna memperoleh nilai yang representatif untuk kemudian dibandingkan satu sama lain.

4.6.1 Performa *Rendering*

Analisis performa *rendering* dilakukan dalam tiga kali eksperimen untuk mengevaluasi efisiensi masing-masing *library* visualisasi data, yakni Chart.js, D3.js, dan Highcharts dengan hasil seperti berikut

Tabel 4.10 Komparasi Rendering Library Chart.js, D3.js, dan Highcharts

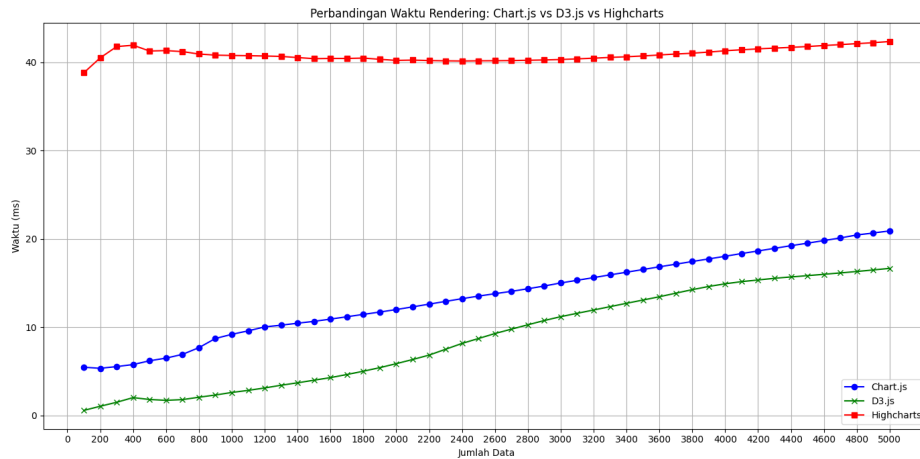
Sample	Chart.js			D3.js			Highcharts		
	1 st	2 nd	3 rd	1 st	2 nd	3 rd	1 st	2 nd	3 rd
100	5.36	5.46	3.00	0.35	0.58	0.40	34.35	38.80	24.57
200	5.19	5.35	3.27	0.62	1.05	0.79	37.81	40.50	28.81
300	5.56	5.54	3.58	0.59	1.50	1.13	38.37	41.77	29.50
400	6.10	5.77	3.86	0.60	2.03	1.66	38.67	41.93	30.17
500	6.40	6.20	4.13	0.73	1.81	2.03	38.55	41.27	30.40
600	6.42	6.50	4.41	0.93	1.72	2.39	38.79	41.32	30.86
700	6.66	6.92	4.72	1.26	1.80	2.77	38.53	41.20	31.26
...	...	...	...	...	...	...	...	...	...
4400	18.33	19.23	15.25	14.91	15.69	17.38	40.91	41.68	37.20
4500	18.55	19.51	15.55	15.08	15.84	17.56	41.06	41.77	37.44
4600	18.78	19.81	15.86	15.25	15.99	17.72	41.22	41.89	37.73
4700	19.72	20.10	16.40	15.44	16.14	17.89	41.40	41.99	37.98
4800	20.02	20.44	16.87	15.81	16.31	18.06	41.52	42.10	38.44
4900	20.23	20.66	17.12	16.03	16.48	18.23	41.64	42.21	39.17
5000	20.43	20.91	17.37	16.23	16.66	18.41	41.82	42.35	39.95



Gambar 4.34 Grafik Komparasi Render Pertama

Pada eksperimen pertama, ditemukan perbedaan yang cukup mencolok antara Chart.js, D3.js, dan Highcharts dalam hal kecepatan *rendering*. Meskipun Chart.js dan D3 masih dapat bersaing dengan performa yang cukup baik, Chart.js menunjukkan waktu *render* yang fluktuatif dibandingkan D3 yang menunjukkan laju perubahan yang cukup stabil. Sementara itu, Highcharts memiliki waktu *render* yang paling tinggi. Saat jumlah data meningkat hingga mencapai 5000 entri, baik Chart.js maupun D3.js masih mampu merespon dengan waktu *rendering* yang relatif stabil dan tidak menunjukkan lonjakan performa yang drastis. Hal ini menunjukkan bahwa keduanya cukup andal untuk penggunaan 5000 data. Selain itu, Highcharts juga menunjukkan penurunan waktu *render* dan lebih stabil setelah melewati ambang 1000 data. Penurunan ini disebabkan oleh fitur `turboThreshold` bawaan Highcharts yang secara otomatis membatasi kompleksitas *rendering* demi menjaga kestabilan.

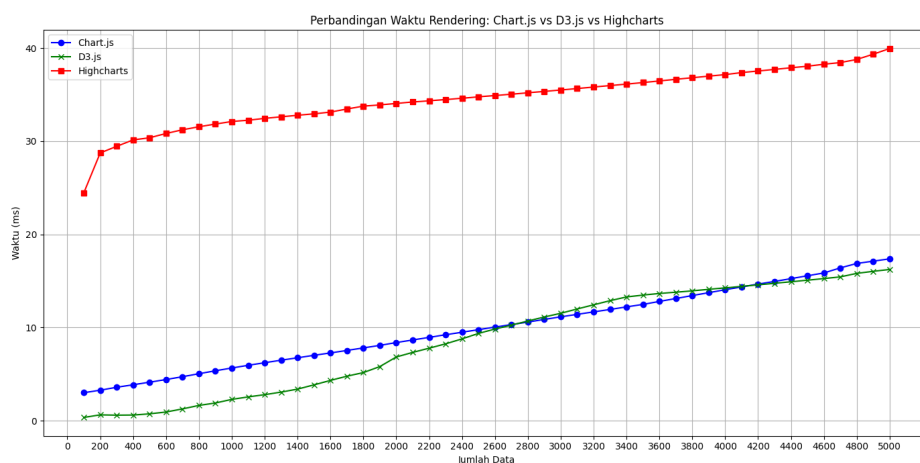
Pada eksperimen kedua, ketiga *library* menunjukkan tren yang sama. Dari grafik tersebut, terlihat bahwa D3 memiliki performa terbaik secara konsisten diikuti oleh Chart.js dan Highcharts yang cenderung konstan namun memiliki waktu *render* yang cukup tinggi.



Gambar 4.35 Grafik Komparasi Render Kedua

D3.js menunjukkan peningkatan waktu eksekusi yang linear namun tetap paling rendah dibandingkan dua *library* lainnya. Chart.js juga menunjukkan peningkatan linear yang tidak berbeda jauh dengan D3. Kedua *library* ini mengalami peningkatan di setiap kenaikan jumlah data dengan selisih waktu *render* kurang dari 5ms. Sementara itu, Highcharts menunjukkan karakteristik *rendering* seperti pada eksperimen pertama. Highcharts memiliki waktu *rendering* yang cukup lama dan mengalami peningkatan namun cenderung konstan setelah jumlah data lebih dari 1000.

Pada eksperimen ketiga, terdapat beberapa perbedaan dari eksperimen sebelumnya. Pada eksperimen ini, D3 memiliki waktu *render* yang lebih tinggi dari Chart.js pada titik 3400 hingga 4600 dengan selisih data tidak lebih dari 1ms.



Gambar 4.36 Grafik Komparasi Render ketiga

Namun, secara keseluruhan, pola tren pada eksperimen ini serupa dengan

eksperimen sebelumnya yang menunjukkan D3 masih menjadi *library* dengan waktu *render* paling rendah.

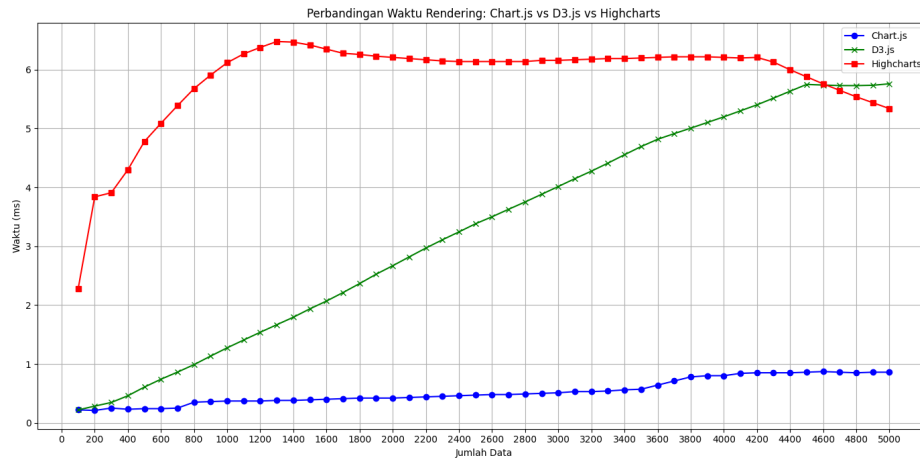
4.6.2 Alokasi CPU

Analisis alokasi CPU dilakukan dalam tiga kali eksperimen untuk mengevaluasi efisiensi masing-masing *library* visualisasi data, yakni Chart.js, D3.js, dan Highcharts.

Tabel 4.11 Komparasi Penggunaan CPU

Sample	Chart.js			D3.js			Highcharts		
	1 st	2 nd	3 rd	1 st	2 nd	3 rd	1 st	2 nd	3 rd
100	0.22	0.18	0.15	0.22	0.46	0.20	2.28	4.30	2.83
200	0.20	0.19	0.13	0.28	0.50	0.36	1.84	4.22	2.32
300	0.25	0.19	0.13	0.28	0.53	0.19	2.28	4.42	2.89
400	0.24	0.23	0.14	0.26	0.64	0.26	2.42	4.44	3.00
500	0.24	0.21	0.14	0.29	0.65	0.28	1.78	4.76	2.97
600	0.24	0.23	0.16	0.30	0.72	0.40	1.94	5.06	3.27
700	0.25	0.22	0.15	0.34	0.80	0.49	2.10	5.39	3.13
...	...	...	...	...	...	...	...	...	...
4400	0.85	0.68	0.53	5.64	5.60	5.10	6.77	5.13	4.93
4500	0.86	0.69	0.56	5.75	5.36	5.14	5.88	6.80	5.15
4600	0.87	0.70	0.56	5.74	5.36	5.33	5.76	6.80	5.19
4700	0.86	0.71	0.56	5.73	5.44	5.45	6.55	6.81	5.14
4800	0.85	0.72	0.54	5.73	5.57	5.54	6.34	6.94	4.85
4900	0.86	0.73	0.54	5.74	5.70	5.54	6.34	6.84	4.93
5000	0.86	0.74	0.54	5.76	5.84	5.33	6.34	6.84	4.85

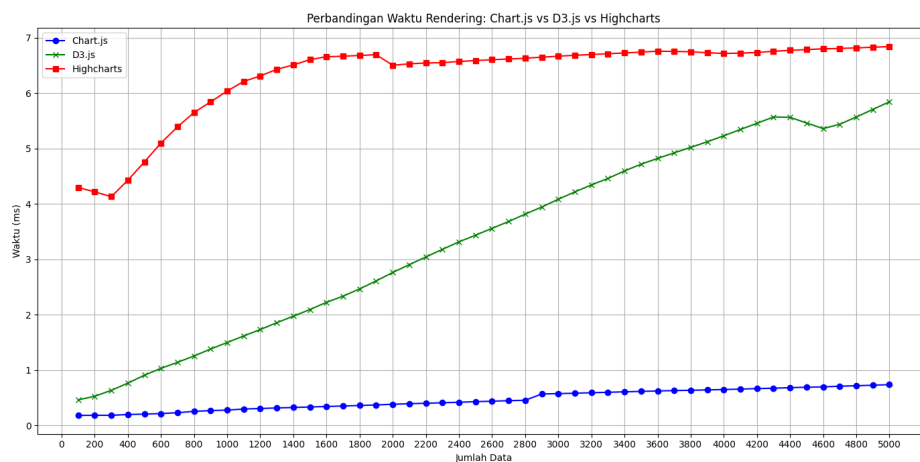
Pada eksperimen pertama, Chart.js menunjukkan performa yang sangat efisien dengan penggunaan CPU yang konsisten rendah, D3 mengalami kenaikan yang tinggi dan highcharts mengalami kenaikan di awal dan penurunan pada titik 4200 data.



Gambar 4.37 Grafik Komparasi CPU Pertama

Chart.js menunjukkan waktu *render* mulai dari sekitar 0.2% hingga hanya sekitar 0.7% pada 5000 titik data. D3.js menunjukkan pola kenaikan yang linier dan lebih signifikan dibanding Chart.js. D3 terus mengalami peningkatan penggunaan CPU hingga mencapai hampir 5.9% pada titik data maksimum. Hal ini menunjukkan D3 membutuhkan lebih banyak *resource* saat menangani dataset besar. Sementara itu, Highcharts menunjukkan penggunaan CPU yang tinggi sejak awal, mulai dari 2.3% dan melonjak hingga 6.5%, sebelum akhirnya menurun sedikit di titik akhir. Penurunan ini bisa terjadi karena mekanisme internal optimasi seperti *threshold* dan simplifikasi *rendering*, serta pembatasan otomatis dari browser yang bertujuan menjaga stabilitas dan mencegah aplikasi *crash*.

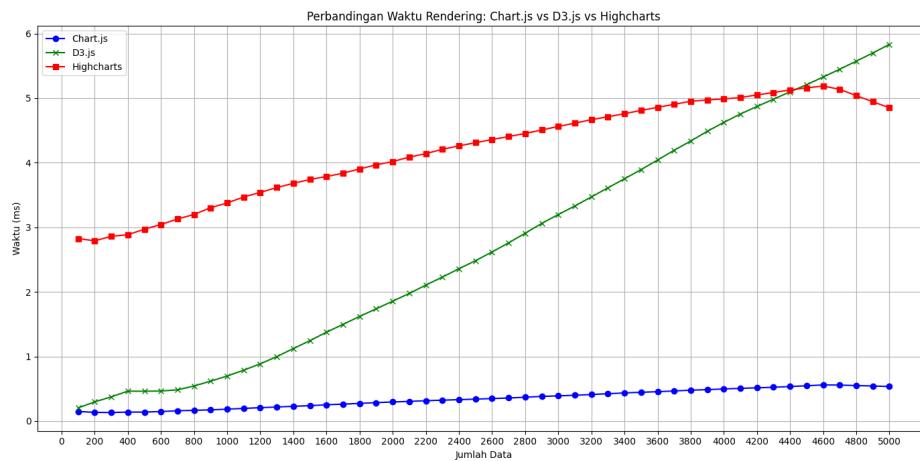
Pada eksperimen kedua, pola yang terjadi mirip dengan eksperimen pertama. Nilai Chart.js cenderung stabil, D3 yang meningkat secara signifikan, dan Highcharts yang memiliki waktu *render* tertinggi.



Gambar 4.38 Grafik Komparasi CPU Kedua

Performa Chart.js tetap konsisten dengan hasil sebelumnya, kembali menunjukkan penggunaan CPU yang rendah dan stabil, bahkan nyaris tidak terpengaruh oleh peningkatan jumlah data. D3 juga menunjukkan tren linier yang mirip dengan sebelumnya, mencapai hampir 5.8% di 5000 titik data tanpa fluktuasi yang signifikan. Highcharts pada eksperimen ini mengalami penurunan penggunaan CPU di akhir pengujian setelah mencapai puncaknya sekitar 5.2%, turun menjadi sekitar 4.9%. Hal ini mengindikasikan adanya stabilisasi pada penggunaan CPU, namun tetap lebih tinggi dibanding dua *library* lainnya.

Pada eksperimen ketiga, Chart.js masih menunjukkan penggunaan CPU paling kecil, D3 mengalami kenaikan signifikan dan Highcharts memiliki nilai penggunaan CPU tertinggi.



Gambar 4.39 Grafik Komparasi CPU Ketiga

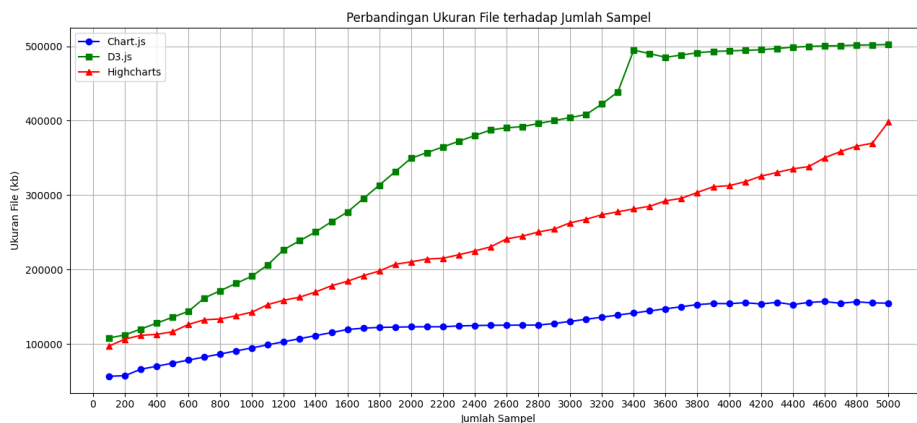
Penggunaan CPU oleh Chart.js berkisar di antara 0.2% hingga 0.6% secara konsisten. D3.js menunjukkan hasil yang sangat mirip dengan dua eksperimen sebelumnya di mana penggunaan CPU naik secara linier dan mencapai sekitar 5.8% di titik akhir. Highcharts, memiliki konsumsi CPU yang lebih rendah pada eksperimen ini. Penggunaan CPU berada pada 2,9% dan mencapai hampir 5,2% pada 4500 titik data sebelum mengalami penurunan hingga 4,9%. Hal ini menunjukkan untuk menampilkan data dengan D3 dan Highcharts, diperlukan resource CPU yang besar pula.

4.6.3 Alokasi Memori

Tabel 4.12 Perbandingan Penggunaan Memori

Samples	Chart.js			D3.js			Highcharts		
	1st	2nd	3rd	1st	2nd	3rd	1st	2nd	3rd
100	56,730	59,600	56,883	108,144	52,290	108,203	97,183	70,350	83,214
200	57,460	66,959	58,921	112,211	60,310	112,289	106,565	92,021	91,500
300	66,104	74,317	60,734	120,125	68,940	120,145	111,724	108,760	100,200
400	70,215	81,676	62,489	128,039	73,620	128,065	114,003	134,870	117,800
500	74,326	89,033	64,217	132,745	86,380	136,005	116,479	140,800	120,433
600	78,437	96,394	65,871	143,883	95,670	143,920	126,316	150,550	124,300
700	82,548	103,752	67,534	161,812	105,290	161,850	135,623	157,700	130,400
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
4400	152,967	142,615	146,051	498,702	494,992	511,000	335,295	404,300	386,200
4500	155,905	143,490	146,128	499,774	473,502	515,200	338,330	407,250	388,000
4600	157,111	144,365	146,152	500,344	473,790	518,000	350,158	416,000	389,200
4700	154,744	145,204	146,070	500,812	474,080	519,000	358,557	417,440	420,268
4800	156,783	148,564	146,067	501,496	487,760	519,600	365,698	417,800	390,084
4900	155,316	151,889	146,070	501,855	494,600	519,900	369,709	418,450	385,700
5000	154,872	155,213	146,072	502,214	501,440	520,120	398,125	420,510	383,616

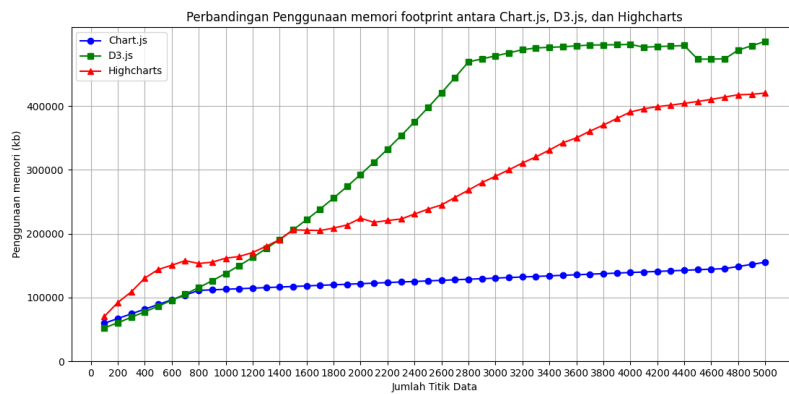
Pada eksperimen pertama, perbandingan penggunaan memory ditampilkan seperti berikut



Gambar 4.40 Grafik Komparasi Memori Pertama

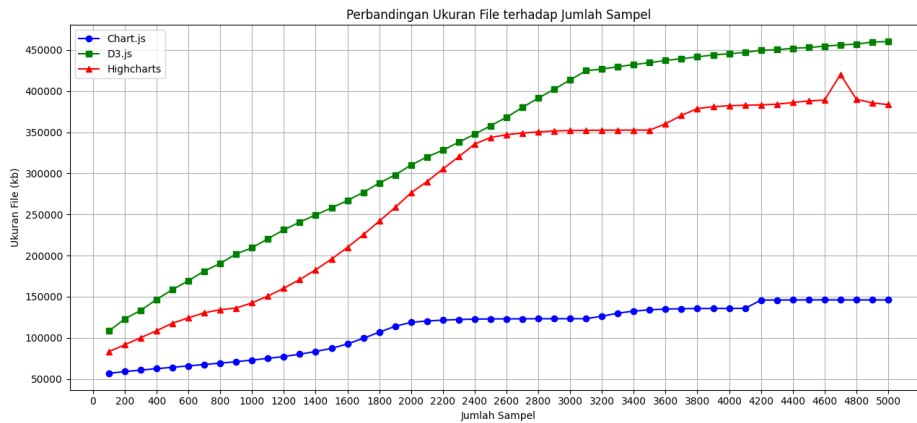
Dari grafik tersebut dapat ditinjau bahwa D3 memiliki penggunaan memori paling tinggi di antara ketiganya dan terus meningkat secara signifikan seiring bertambahnya jumlah titik data. Highcharts berada di posisi tengah, dengan tren

peningkatan memori yang konsisten namun tidak setinggi D3.js. Sementara itu, Chart.js menunjukkan penggunaan memori paling efisien, dengan peningkatan yang relatif kecil meskipun jumlah titik data bertambah hingga lebih dari 5000. Hal ini mengindikasikan bahwa Chart.js lebih optimal untuk aplikasi yang membutuhkan efisiensi memori, terutama ketika menangani dataset yang besar. D3.js meskipun sangat fleksibel, cenderung memakan lebih banyak memori, kemungkinan karena struktur dan proses *rendering* yang lebih kompleks. Pada eksperimen kedua, perbandingan memori ditampilkan seperti berikut



Gambar 4.41 Grafik Komparasi Memori Kedua

Chart.js masih menunjukkan penggunaan memori yang paling stabil dan efisien dibandingkan dua *library* lainnya. Meskipun jumlah titik data meningkat hingga 5000, kenaikan memori sangat lambat, dengan rata-rata penggunaan memori di bawah 150 mb. Ini menjadikan Chart.js sangat cocok untuk aplikasi dengan keterbatasan memori atau kebutuhan visualisasi ringan. Pada eksperimen ini penggunaan memori oleh D3 dan Highcharts meningkat secara signifikan. Pada eksperimen ketiga, Chart.js secara konsisten menghasilkan file dengan ukuran paling kecil di antara ketiga *library*.



Gambar 4.42 Grafik Komparasi Memori Ketiga

Pada percobaan ketiga, dengan data Chart.js lebih dari 5000 sampel, *footprint memory* yang dihasilkan tidak melebihi 150.000 Kb. Pola kenaikannya juga cukup stabil dan linear yang menunjukkan efisiensi dalam penyimpanan dan *rendering*. D3 menghasilkan file dengan ukuran paling besar dan terus meningkat hingga lebih dari 500.000 kb saat mencapai 5000 data. Ini menunjukkan bahwa meskipun D3 menawarkan fleksibilitas dan kemampuan kustomisasi yang tinggi, D3 juga memiliki *overhead* signifikan dalam penyimpanan file. Highcharts berada di posisi tengah antara D3.js dan Chart.js. *Memory footprint* yang dihasilkan meningkat secara eksponensial hingga sekitar 400.000 kb, dan kemudian mengalami sedikit fluktuasi di titik-titik tertentu, seperti pada sekitar 4700 sampel. Hal ini menunjukkan bahwa meskipun Highcharts memberikan hasil visualisasi yang kompleks dan interaktif, penggunaan sumber daya dalam hal *memory footprint* masih lebih rendah dibandingkan D3.js, meskipun tetap lebih tinggi dari Chart.js.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan analisis dan pembahasan data hasil pengujian, diperoleh beberapa kesimpulan, yaitu:

1. Chart.js menunjukkan kinerja *rendering* yang sangat baik pada jumlah data kecil hingga menengah dengan waktu pemrosesan yang cepat. Namun, saat jumlah data meningkat signifikan, performanya mulai melambat. D3 memiliki performa *rendering* yang kurang stabil pada data berjumlah besar karena memanipulasi elemen DOM secara langsung. Highcharts mampu menjaga kecepatan dan kestabilan *rendering* meskipun dihadapkan pada beban data besar karena fitur optimasi internal yang dimilikinya.
2. Chart.js cukup efisien dalam penggunaan memori. D3.js cenderung memakan lebih banyak memori karena setiap elemen visualnya dirender sebagai objek SVG individual, yang menyebabkan beban memori meningkat pesat saat dataset bertambah besar. Sebaliknya, Highcharts menunjukkan konsumsi memori yang stabil karena memiliki pengaturan bawaan untuk membatasi jumlah elemen yang aktif secara bersamaan, sehingga cocok digunakan untuk aplikasi dengan skala data yang tinggi.
3. Pemantauan penggunaan CPU menunjukkan Chart.js bekerja ringan dan tidak membebani sistem ketika data masih dalam jumlah wajar. Namun, performa CPU mulai menurun seiring bertambahnya data yang divisualisasikan. D3.js, karena kompleksitas render-nya, membutuhkan lebih banyak tenaga pemrosesan CPU, terutama saat memproses elemen yang banyak dalam waktu bersamaan. Highcharts lebih efisien karena mampu mendistribusikan beban pemrosesan dengan baik dan melakukan update data secara dinamis tanpa harus me-render ulang seluruh elemen *chart*.
4. Dari seluruh indikator yang diuji, Chart.js menunjukkan performa yang paling konsisten dibandingkan dengan D3.js dan Highcharts. *Library* ini cenderung menghasilkan nilai yang stabil tanpa fluktuasi ekstrem dalam berbagai aspek pengujian seperti *rendering*, penggunaan memori, dan konsumsi CPU. Meskipun tidak selalu menjadi yang tercepat atau paling efisien dalam setiap kategori, Chart.js tetap memberikan hasil yang dapat diprediksi dan tidak

mengalami penurunan performa yang signifikan ketika skenario pengujian diperluas. Stabilitas ini menjadikannya pilihan yang andal untuk aplikasi visualisasi data IoT berskala kecil hingga menengah, terutama ketika konsistensi performa diutamakan.

5.2 Saran

Berdasarkan hasil penelitian yang dilakukan, ditemukan beberapa kelemahan dalam pelaksanaannya, sehingga diperlukan upaya pengembangan lebih lanjut untuk memperbaiki sistem yang ada. Berikut ini adalah saran-saran yang dapat digunakan sebagai panduan untuk pengembangan penelitian selanjutnya:

1. Penelitian dapat diperluas dengan meneliti *library* visualisasi JavaScript lain. Dengan membandingkan lebih banyak *library*, peneliti dapat memperoleh gambaran yang lebih komprehensif mengenai kelebihan dan kekurangan masing-masing *library*.
2. Eksperimen dapat dilakukan pada perangkat lain guna memperoleh hasil yang lebih representatif dan aplikatif di berbagai lingkungan sistem.
3. Penelitian selanjutnya juga disarankan untuk memperluas pengujian terhadap jenis visualisasi lain yang belum dibahas dalam studi ini, seperti *bar chart*, *scatter plot*, *area chart*, atau *heatmap*. Dengan menguji berbagai tipe visualisasi, peneliti dapat mengevaluasi bagaimana setiap *library* menangani kompleksitas data dalam berbagai bentuk tampilan.
4. Pengimplementasikan Highcharts dan D3 harus disertai dengan optimalisasi CPU dan memori untuk mengurangi potensi *memory leak* atau kenaikan konsumsi CPU yang berlebihan.

DAFTAR PUSTAKA

- [1] R. Y. Endra, Y. Aprilinda, Y. Y. Dharmawan, and W. Ramadhan, “Analisis perbandingan bahasa pemrograman php laravel dengan php native pada pengembangan website,” *EXPERT: Jurnal Manajemen Sistem Informasi dan Teknologi*, vol. 11, p. 48, 6 2021.
- [2] Z. K. Mundargi, K. Patel, A. Patel, R. More, S. Pathrabe, and S. Patil, “Plotplay: An automated data visualization website using python and plotly,” in *2023 International Conference for Advancement in Technology, ICONAT 2023*. Institute of Electrical and Electronics Engineers Inc., 2023.
- [3] F. Boström, A. Dahlberg, W. Linderöth, M. Lamb, and J. Steinhauer, “A performance investigation into javascript visualization libraries with the focus on render time and memory usage: A performance measurement of different libraries and statistical charts,” Tech. Rep., 2022.
- [4] J. Persson, “Scalability of javascript libraries for data visualization bachelor degree project in information technology basic level 30 ects spring term 2021,” Tech. Rep., 2021.
- [5] Chart.js, “Chart.js samples,” pp. 1–1, 2 2025.
- [6] D3, “What is d3?”
- [7] ossinsight, “Javascript charting - ranking.”
- [8] A. H. Renear, S. Sacchi, and K. M. Wickett, “Definitions of dataset in the scientific and technical literature,” in *Proceedings of the ASIST Annual Meeting*, vol. 47, 11 2010.
- [9] M. D. Khairy, A. N. Defitri, A. Hadi, A. Nuzuli, A. L. Dyakiyyah, and A. Heryanto, “Netplg journal of network and computer applications pengolahan data sensor iot dengan apache spark menggunakan metode batch processing,” *NetPLG*, vol. 2, 10 2023. [Online]. Available: <https://jurnal.netplg.com/jnca>
- [10] S. Vučetić, M. Vulović, D. Radosavljević, P. Milić, J. Lekić, and B. Milosavljević, “Open data visualization by using javascript libraries,” Tech. Rep., 2023. [Online]. Available: <https://data.gov.rs/sr/datasets/ustanove-kulture-biblioteke-1>

- [11] S. Manna and A. Banerjee, "Experimental study on mqtt protocol based home automation system with enhancement," in *Proceedings - 2024 4th International Conference on Computer, Communication, Control and Information Technology, C3IT 2024*. Institute of Electrical and Electronics Engineers Inc., 2024.
- [12] P. Toppany, D. Kurnia, Janizal, and P. Aji, "Integrasi framework bootstrap dan chart.js untuk visualisasi data sensor pada sistem hidroponik berbasis internet of things (iot)," 2023.
- [13] L. Rothenhäusler, "d3.js and its potential in data visualization creating a diagram showcase using ukrainian refugee data," 8 2022.
- [14] M. Triawan, H. Husnawati, and F. Humam, "Sistem pemantauan lingkungan menggunakan sensor bme280 berbasis internet of things," *Generic*, vol. 15, pp. 37–41, 6 2023.
- [15] G. C. G. D. Melo, I. C. Torres, Ícaro Bezzerá Queiroz De Araújo, D. B. Brito, and E. D. A. Barboza, "A low-cost iot system for real-time monitoring of climatic variables and photovoltaic generation for smart grid application a low-cost iot system for real-time monitoring of climatic variables and photovoltaic generation for smart," *Grid Application. Sensors*, 5 2021.
- [16] G. Danielyan, "Analysis of data visualization tools and approaches," Tech. Rep., 2018.
- [17] L. A. Gokhale, B. Vidyapeeth, K. Nilesh, M. B. Vidyapeeth, . Kirti, N. Mahajan, . Leena, and A. Gokhale, "Comparative study of data visualization tools," Tech. Rep., 2020. [Online]. Available: <https://www.researchgate.net/publication/344425307>
- [18] S. Benbba, "Comparison of d3.js and chart.js as visualisation tools," Tampere University, Tech. Rep., 4 2021.
- [19] H. M. Millqvist, N. Bolin, A. Márki, and J. Steinhauer, "En jämförelse av prestanda och skalbarhet för grafgenerering i datavisualiserande javascript-bibliotek a comparision of performance and scalability of chart generation for javascript data visualisation libraries," Tech. Rep., 2022.
- [20] N. W. Kim, G. Ataguba, S. C. Joyner, C. Zhao, and H. Im, "Beyond alternative text and tables: Comparative analysis of visualization tools and accessibility methods," *Computer Graphics Forum*, vol. 42, pp. 323–335, 6 2023.

- [21] Laravel, “Why laravel?”
- [22] w3schools.com, “Javascript introduction.”
- [23] Highcharts, “Highcharts documentation.” [Online]. Available: <https://www.highcharts.com/docs/index>
- [24] A. Unwin, “Why is data visualization important? what is important in data visualization?” *Harvard Data Science Review*, 1 2020.
- [25] L. Mykhailo and S. Yevgeniya, “Real-time data visualization for iot network systems: Challenges and strategies for performance optimization,” *System technologies*, vol. 5, pp. 52–61, 3 2024.
- [26] S. Egger-Lampl, T. Hossfeld, R. Schatz, and M. Fiedler, “Waiting times in quality of experience for web based services,” in *2012 4th International Workshop on Quality of Multimedia Experience, QoMEX 2012*, 6 2012.
- [27] D. J. Paradis and B. Segee, “Remote rendering and rendering in virtual machines,” 2016.
- [28] A. Yıldız, H. F. Ugurdag, B. Aktemur, D. İskender, and S. Gören, “Cpu design simplified,” in *2018 3rd International Conference on Computer Science and Engineering (UBMK)*, 2018, pp. 630–632.
- [29] A. B. Bondi, “Characteristics of scalability and their impact on performance,” Tech. Rep., 2000. [Online]. Available: <http://www.whatis.com/scalabil.htm>.
- [30] I. H. Nurwarsito, M. K. Barlian, H. P. St, M. T. Eko, S. Pramukantoro, S. Kom, and M. Kom, “Arsitektur dan organisasi komputer i internal memori,” Tech. Rep.
- [31] E. I. Wahyuni, S. A. Gani, H. Aryanto, A. K. Siregar, and Q. Aini, “Analisis perancangan sistem informasi pendaftaran siswa baru tk putiek nanggroe berbasis web menggunakan unified modeling language,” Tech. Rep.
- [32] S. Al-Fedaghi, “Uml sequence diagram: An alternative model,” Tech. Rep. [Online]. Available: www.thesai.org
- [33] R. K. Kasera and T. Acharjee, “A comprehensive iot edge based smart irrigation system for tomato cultivation,” *Internet of Things (Netherlands)*, vol. 28, 12 2024.
- [34] T. Suryana, “Membangun stasiun cuaca dengan bme 280 untuk monitoring suhu, kelembaban,tekanan udara dan ketinggian,” Tech. Rep., 2022. [Online]. Available: <https://github.com/nodemcu/nodemcu-devkit>

- [35] Flask, “Flask documentation.”
- [36] S. Diantika, “Penerapan teknik random oversampling untuk mengatasi imbalance class dalam klasifikasi website phishing menggunakan algoritma lightgbm,” Tech. Rep., 2023.
- [37] R. Akbar, R. A. Siroj, M. W. Afgani, and U. I. N. R. F. P. Abstract, “Experimental reseacrch dalam metodologi pendidikan,” *Jurnal Ilmiah Wahana Pendidikan, Januari*, vol. 2023, pp. 465–474.
- [38] Mozilla, “Performance: now() method.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Performance/now>
- [39] K. Vayadande, A. Raut, R. Bhonsle, V. Pungliya, A. Purohit, and S. Pate, “Efficient system for cpu metric visualization,” *3C TIC: Cuadernos de desarrollo aplicados a las TIC*, vol. 11, pp. 239–250, 12 2022.
- [40] S. Febriani, Astuti, K. Ediputra, and Zulfah, “Anova dan tukey hsd analisis kesalahan siswa dalam menjawab soal cerita matematika berdasarkan kriteria watson,” *Jurnal Pengabdian Masyarakat dan Riset Pendidikan*, vol. 2, pp. 183–188, 8 2023.
- [41] M. Ljubojevic and M. Mitrea, “Interactive media planning data visualization,” pp. 361–362, 2024. [Online]. Available: <https://hal.science/hal-04914047v1>
- [42] K. Basques and S. Emelianova, “Referensi fitur performa.”

LAMPIRAN A

A Lembar Perbaikan Proyek Akhir

Nama : Ailsa Isnani Anubhawa
NIM : 21/474745/SV/19024
Prodi : Teknologi Rekayasa Perangkat Lunak
Judul PA : ANALISA PERBANDINGAN KINERJA LIBRARY CHART.JS, D3 DAN
Waktu Pendadaran : Tanggal/bulan/tahun

Dosen Penguji	BAB	Saran Perbaikan	Keterangan	Halaman Perbaikan

LAMPIRAN B

B Dokumentasi